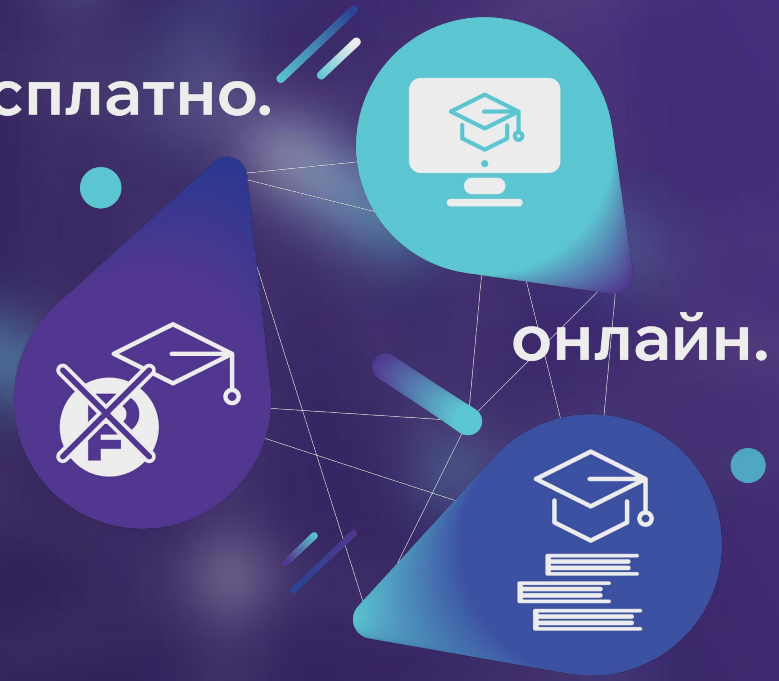
Deep Learning School



DLS

Diffusion for 3D and 3D in diffusion



Nina Konovalova

@AfeliaN

@ReadingAI

@READING_AI

DLS

Generating 3D worlds



Lecture plan, part 1: 3D generation

00

Representations

Meshes, points, voxels, SDF, NeRF, Gaussians: what exactly do we generate in 3D?

01

First attempts

Diffusion straight on 3D data, and why it was hard

02

2D for 3D

Score distillation: optimize 3D so its renders look right to a 2D model

03

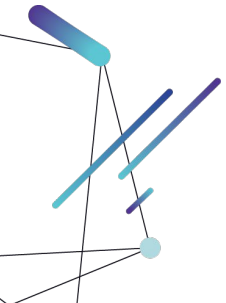
Native 3D after all?

What if we generate 3D directly: 3D latents and much more data

04

...or 2D after all?

What image and video models already know about geometry



Lecture plan, part 2: building a 3D world

DLS

01

Video generators

They already look real, but we can't steer them

☐ LOOKS REAL

02

Add a camera

Make the video follow a given trajectory

☐ FOLLOWS THE CAMERA

03

Add memory

The same place when we come back

☐ SAME WHEN WE RETURN

04

Fight drift

Keep small errors from piling up on a long walk

☐ NO DRIFT

05

Here is a 3D world!

From a long walk to a 3D scene

☐ REAL 3D

DLS

PART 0

Task & representations

A bit about 3D: what do we actually want to generate?



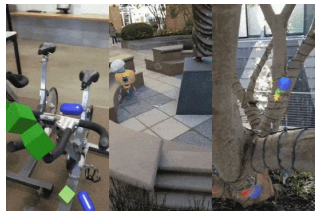
DLS

Why 3D?

An image is one view of an object. **A 3D model** is the object itself: render it from any side



Games & film
assets are still made by hand: slow and costly



AR / VR
objects you can walk around



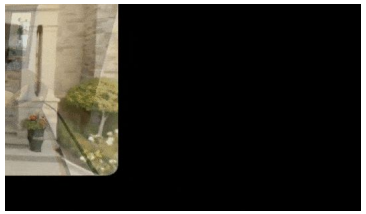
Online shopping
a product you can spin



Design & 3D printing
from an idea to a printable model



Architecture
buildings and interiors before they exist



Robotics & simulation
thousands of objects and scenes to train agents

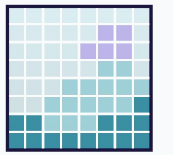
WHAT 3D GIVES

the same from every view · put into a scene · relight · animate · edit · simulate physics

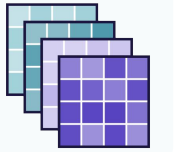
DLS

One format for 2D, many for 3D

2D IMAGES



Pixels
H × W × 3 numbers



Latent
a smaller grid from a VAE

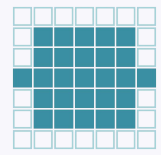
One natural format: a grid

VS

3D OBJECTS



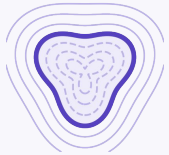
Point cloud
no surface, no connectivity



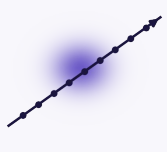
Voxels
memory grows as N^3



Mesh
an irregular graph of any size



Implicit surface
the surface must be extracted



Neural field
slow to fit and to render

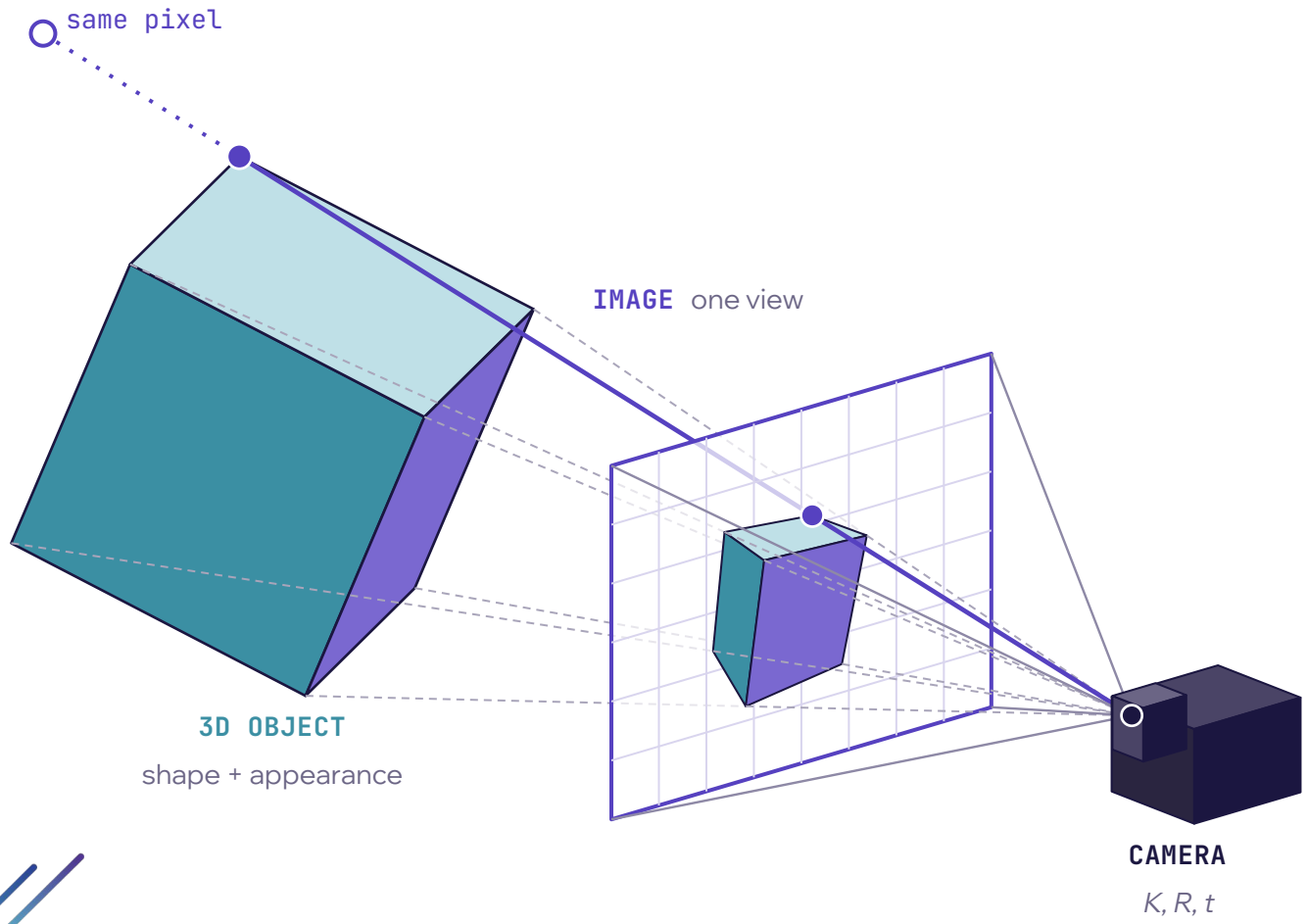


3D Gaussians
millions of unordered blobs

Many formats, each with its own problems

DLS

3D and 2D: two sides of one link



3D → 2D

Rendering

Place a virtual camera and project the object onto the image. A forward problem with one right answer

2D → 3D

Reconstruction

Ambiguous: one photo does not show the back, the missing part has to be imagined

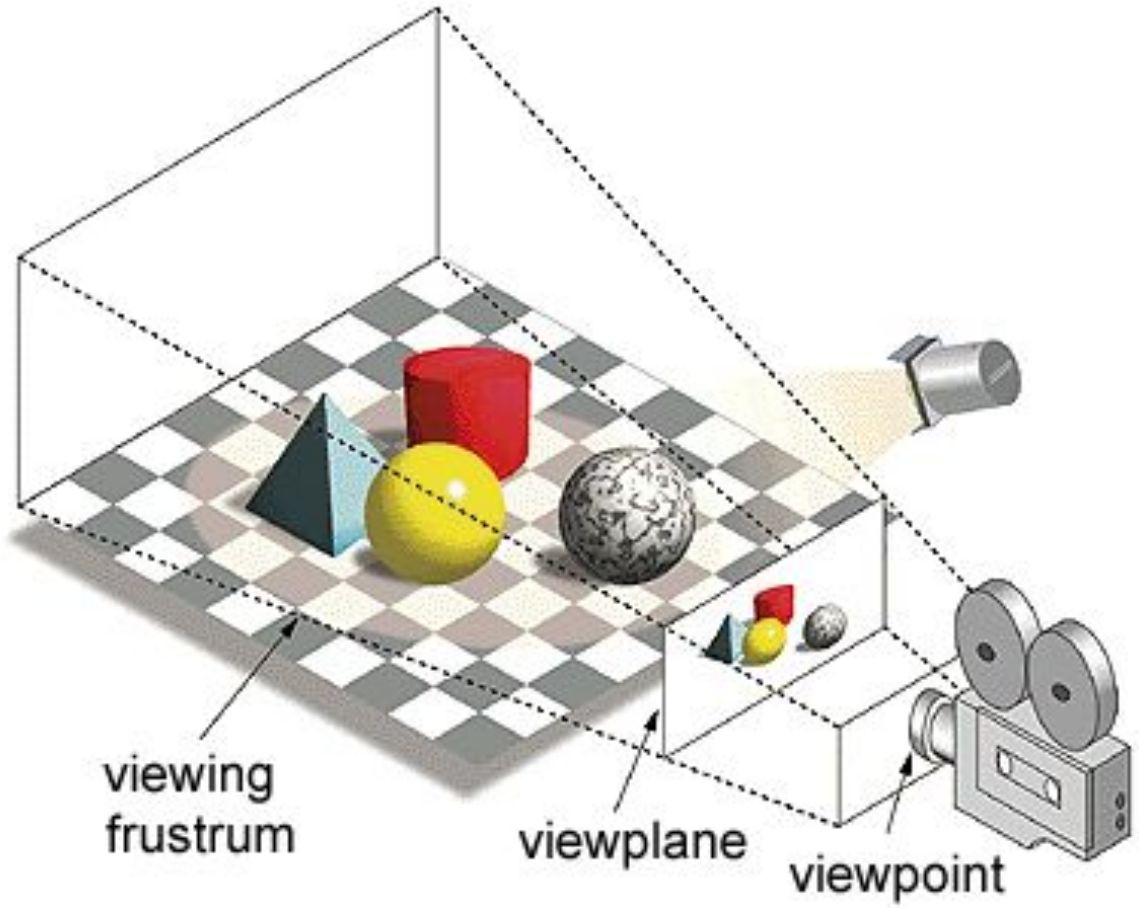
text or image → 3D

Generation

The same backward step, but everything has to be imagined

DLS

3D and 2D: two sides of one link



3D → 2D

Rendering

Place a virtual camera and project the object onto the image. A forward problem with one right answer

3D and 2D: two sides of one link



3D → 2D

Rendering

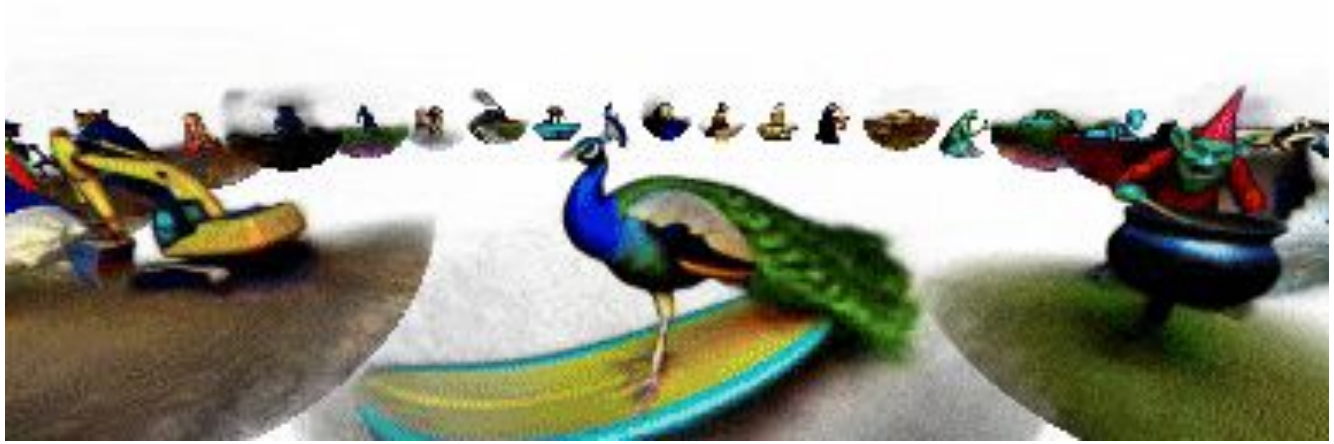
Place a virtual camera and project the object onto the image. A forward problem with one right answer

2D → 3D

Reconstruction

Ambiguous: one or photo does not show the back, the missing part has to be imagined

3D and 2D: two sides of one link



3D → 2D

Rendering

Place a virtual camera and project the object onto the image. A forward problem with one right answer

2D → 3D

Reconstruction

Ambiguous: one photo does not show the back, the missing part has to be imagined

text or image → 3D

Generation

The same backward step, but everything has to be imagined

NeRF · 2020 → 3DGS · 2023 →

DLS

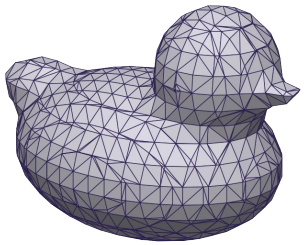
How to define a 3D shape

1 **Point cloud**



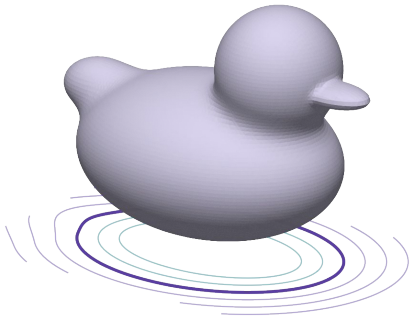
points (x, y, z)

2 **Mesh**



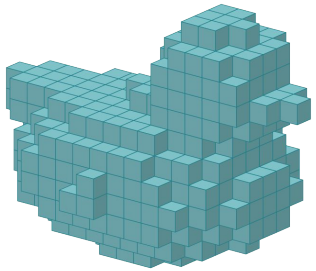
vertices + faces

3 **Implicit surface**



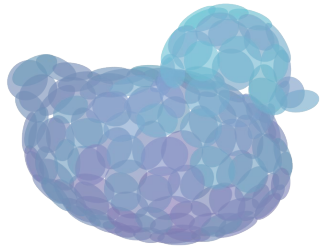
f(x, y, z) = 0

4 **Voxels**

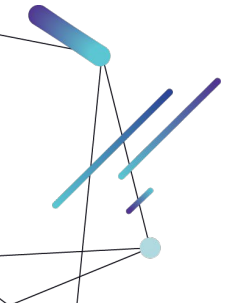


occupied cells

5 **3D Gaussians**



μ, Σ, α, color



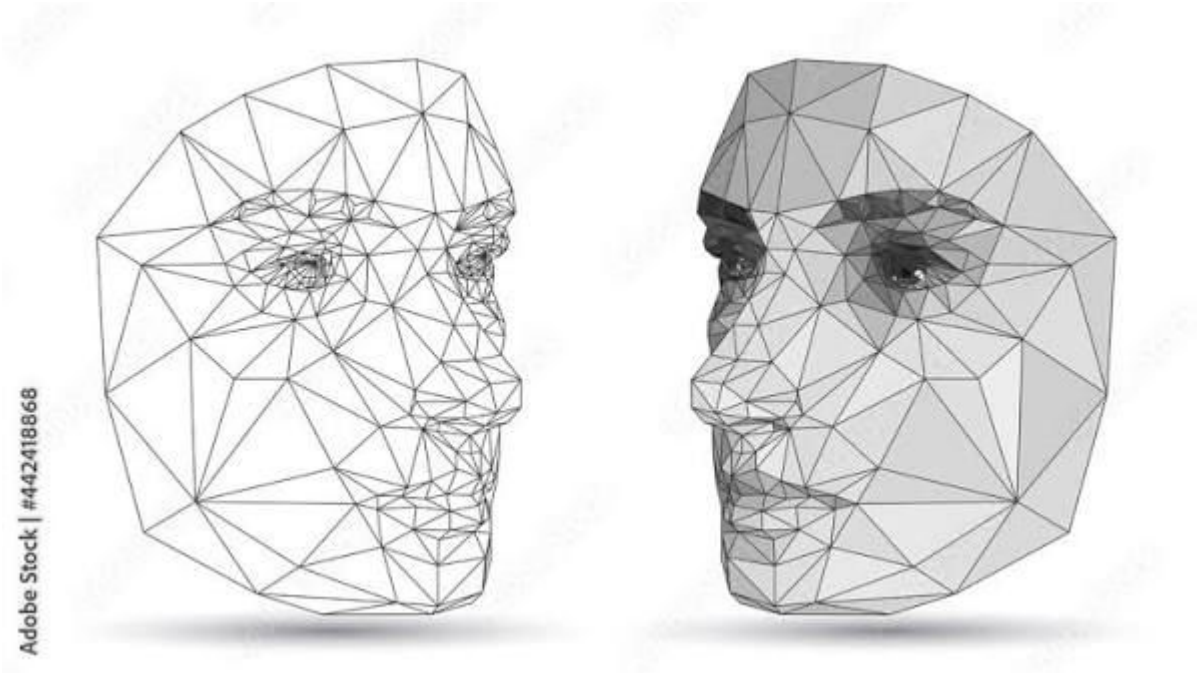
DMTet · 2021 →

FlexiCubes · 2023 →

nvdiffrast · 2020 →

DLS

Mesh representations



Explicit surface

Vertices V + faces F as index triples

Differentiable rendering

Rasterizers such as nvdiffrast pass gradients to vertex positions and textures

Hard to generate directly

Discrete, variable topology

NeRF · 2020 →

Instant-NGP · 2022 →

DLS

NeRF: a field of density and color

An MLP color and density at a point x seen from direction d:

$$F_\theta(\mathbf{x}, \mathbf{d}) \rightarrow (\mathbf{c}, \sigma)$$

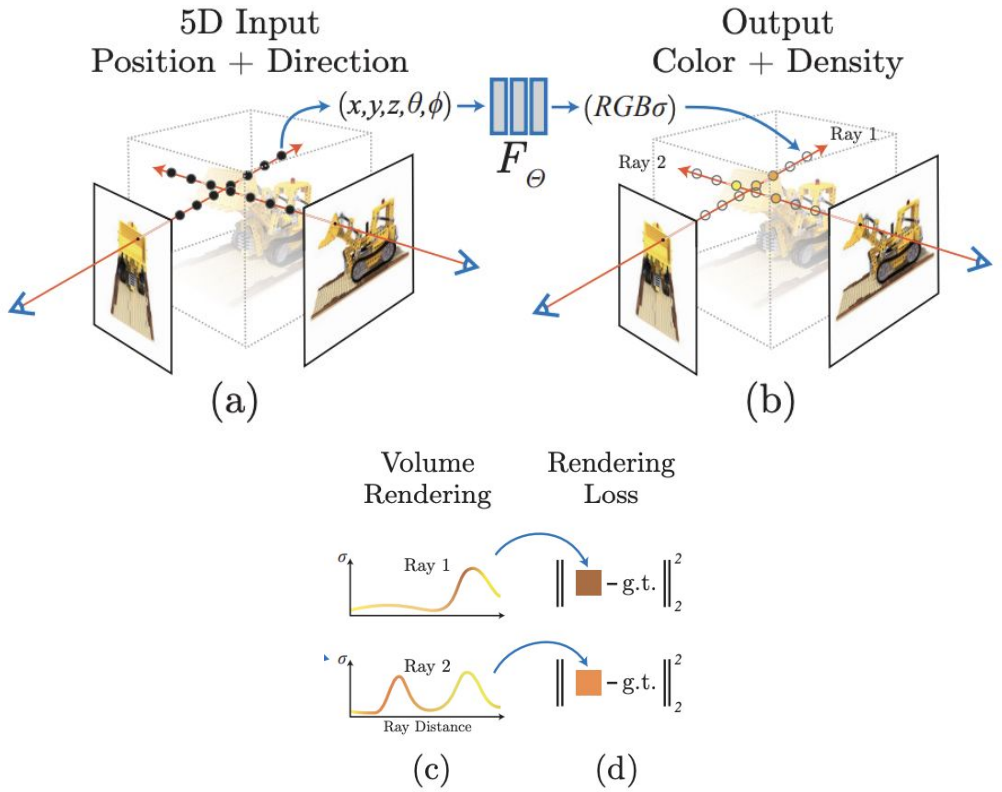
$$C(\mathbf{r}) = \sum_i T_i \left(1 - e^{-\sigma_i \delta_i}\right) \mathbf{c}_i, \qquad T_i = \exp\left(-\sum_{j<i} \sigma_j \delta_j\right)$$

Differentiable

Trained straight from photos with a rendering loss

Hard to generate

Slow: hundreds of samples per ray



Rays → samples → MLP → volume rendering → loss against the photo

3DGS · 2023 →

3D Gaussian Splatting

A scene = millions of 3D Gaussians. Each one stores:

$\mu \in \mathbb{R}^3$ — 3

Position

Where its center is

$\Sigma = R\,S\,S^{\mathrm{T}}\,R^{\mathrm{T}}$ — 3 + 4

Shape

3 scales + a rotation

$\alpha \in [0, 1]$ — 1

Opacity

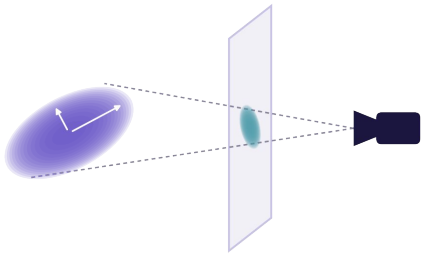
How much light it blocks

$\mathbf{c}(\mathbf{d}) = \mathrm{SH}(\mathbf{d})$ — 48

Color

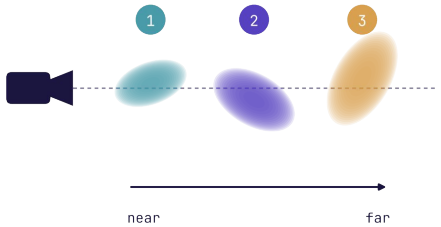
Spherical harmonics: color depends on view direction

1 · PROJECT



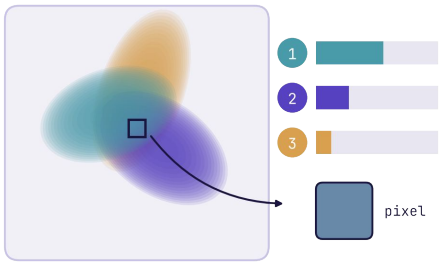
each 3D Gaussian → a 2D ellipse on screen

2 · SORT BY DEPTH



nearest splat first

3 · BLEND



add up colors until the pixel is opaque

Color of one pixel

$$C = \sum_i \mathbf{c}_i\,\alpha_i\,T_i, \qquad T_i = \prod_{j<i}(1-\alpha_j)$$

- $\mathbf{c}_i$ color of splat i, seen from this view
- α_i its opacity × the 2D Gaussian at this pixel
- T_i light left after the splats in front

DLS

3D generation: representation × backbone

3D REPRESENTATION what the model outputs



Point cloud
N points: xyz, optionally color



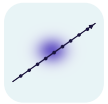
Voxels
occupancy or features on a 3D grid



Mesh
vertices + triangle faces



Implicit surface
SDF or occupancy; surface at the zero level



Neural field
MLP: point → density + color, as in NeRF



3D Gaussians
position, shape, opacity, color

×

GENERATIVE BACKBONE how it is learned

OTHER FAMILIES · NOT TODAY

VAE · GAN · VAE-GAN · Normalizing flow · Autoregressive transformer

Diffusion on 3D data
Point-E → · DiffRF → · PolyDiff →

denoise points, voxels or triangles directly

Latent diffusion
Shap-E → · 3DShape2VecSet →

encode 3D into a latent, diffuse there, decode a field

Diffusion transformer
Hunyuan3D 2.0 → · TRELLIS →

flow matching over a set of latent tokens

2D diffusion prior
DreamFusion → · DreamGaussian →

no 3D training data: optimize 3D so its renders look right

DLS

PART 1

3D generation

Can we generate 3D directly with diffusion?



DLS

3D diffusion, version 1

Point cloud generation

DPM · PVD · LION · Point-E

Denoise the coordinates (and colors) of N points directly

Voxel grid generation

DiffRF

A 3D U-Net denoises a voxel radiance field

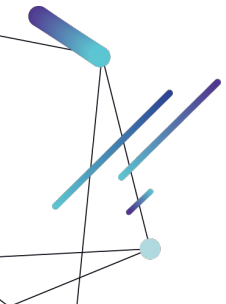
Implicit representation generation

NFD · 3DShape2VecSet · Shap-E

Diffuse triplanes or latent codes that decode to an SDF / NeRF

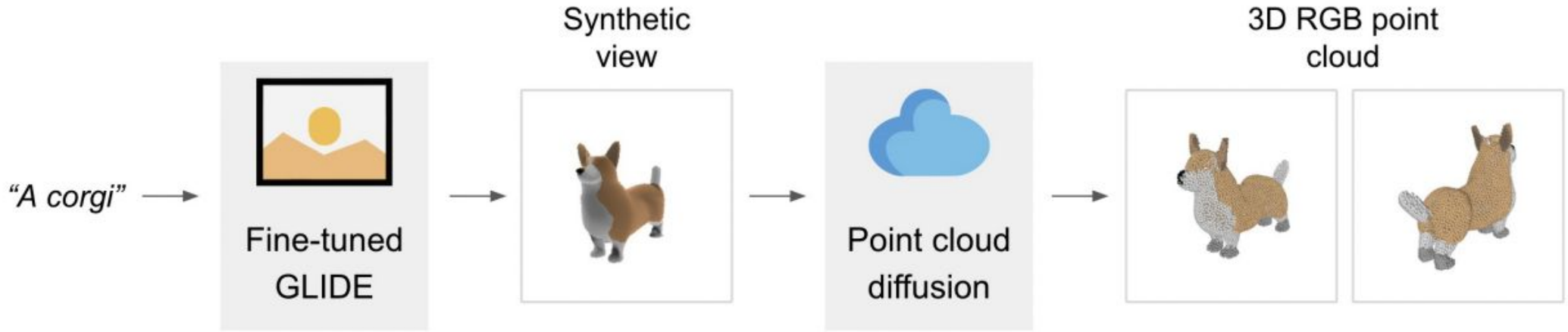
Mesh generation

PolyDiff



Point-E · 2022 →

Point-E

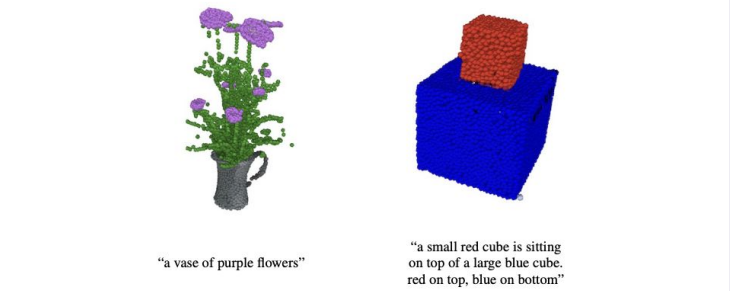


Points as tokens

K × 6 values (xyz + RGB), a transformer without positional encodings, CLIP image tokens as context

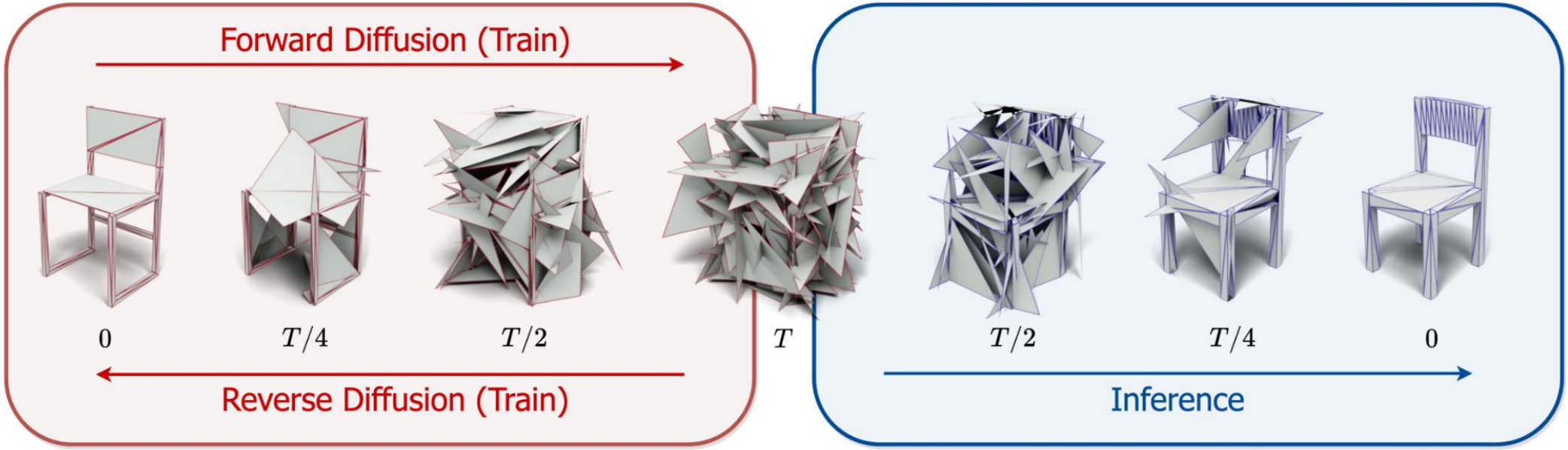
Fast, but rough

1–2 min on one GPU, quality well below optimization-based methods



PolyDiff · 2023 →

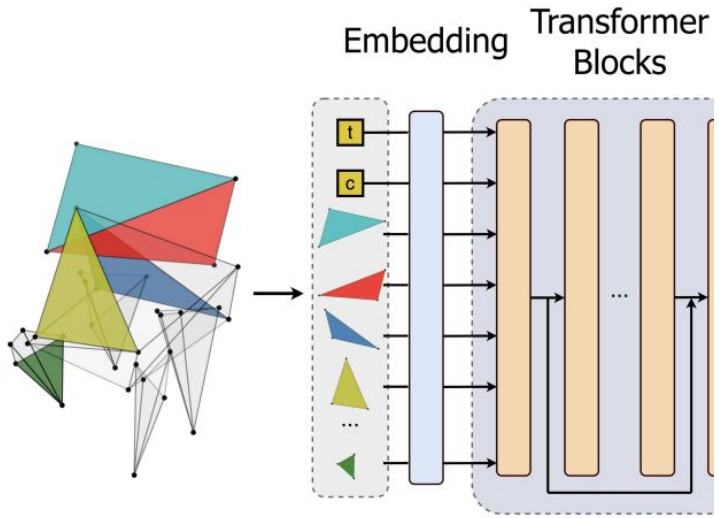
PolyDiff



The forward process scrambles triangles; the reverse process denoises them into a mesh

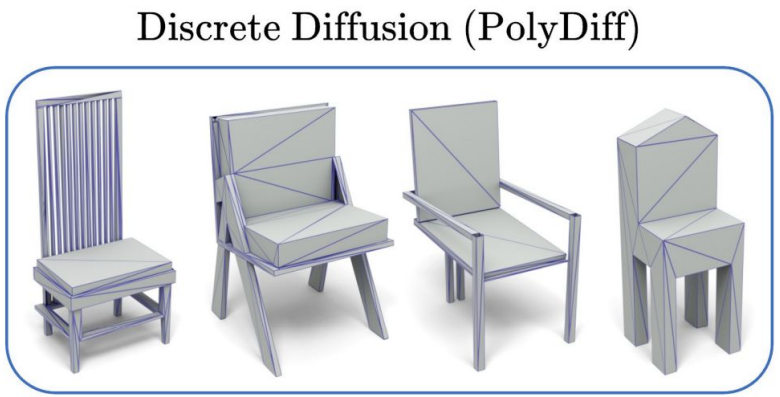
PolyDiff ·

PolyDiff



Denoiser: triangle embeddings → transform





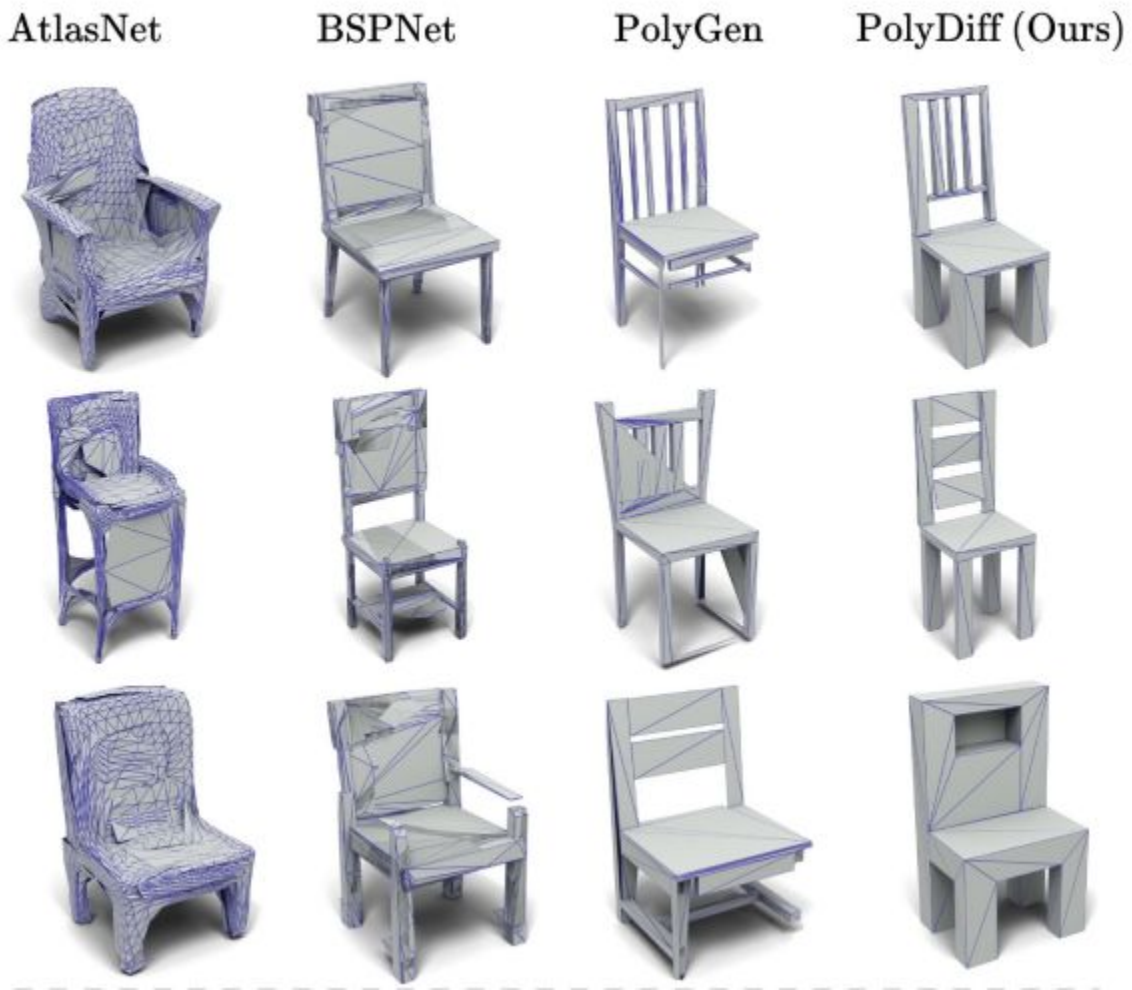
Unconditional samples

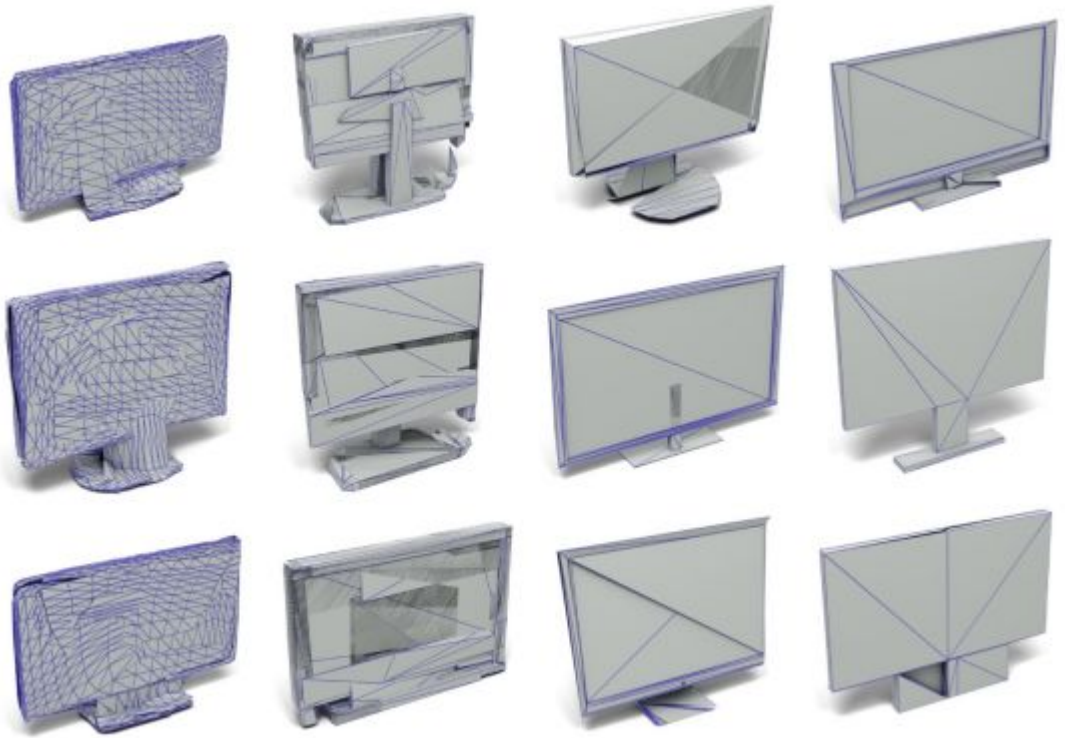
Results: per-category models on ShapeNet (c

nd JSD than PolyGen and BSPNet

PolyDiff · 2023 →

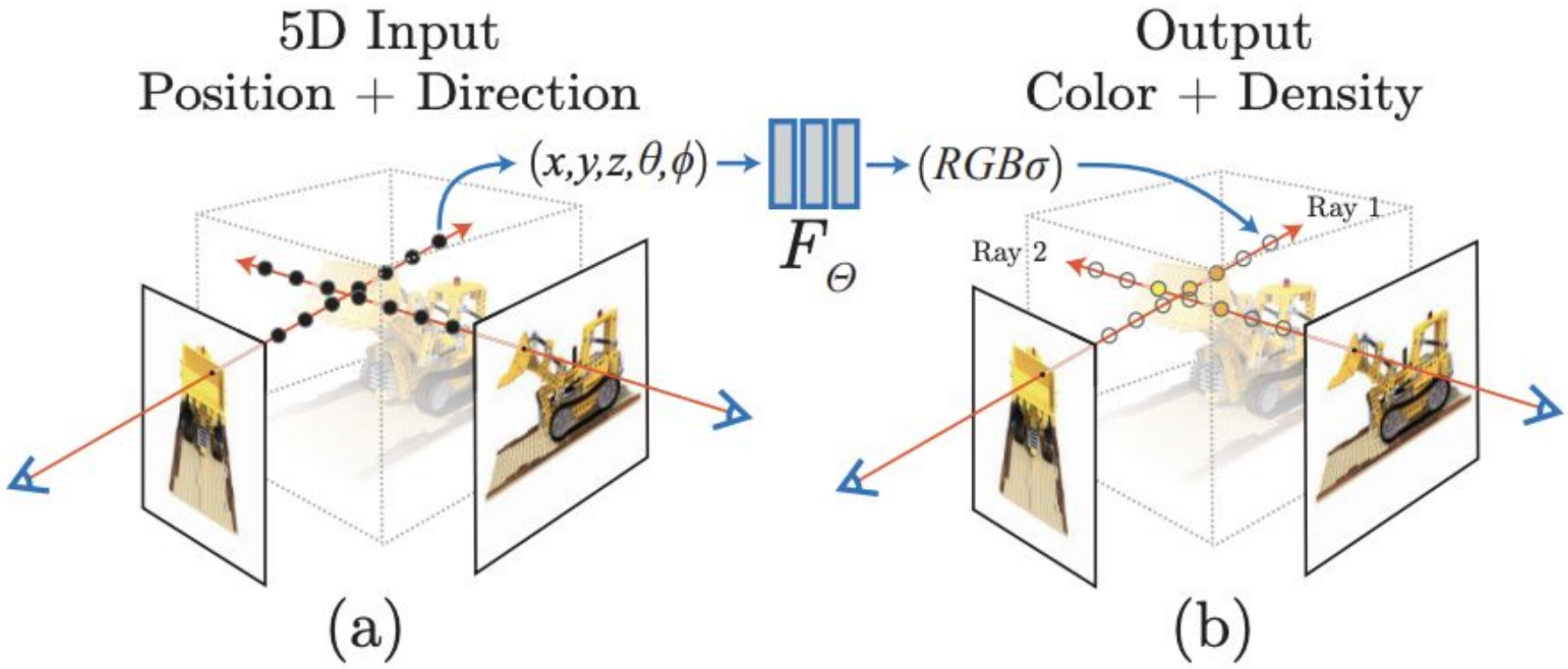
PolyDiff





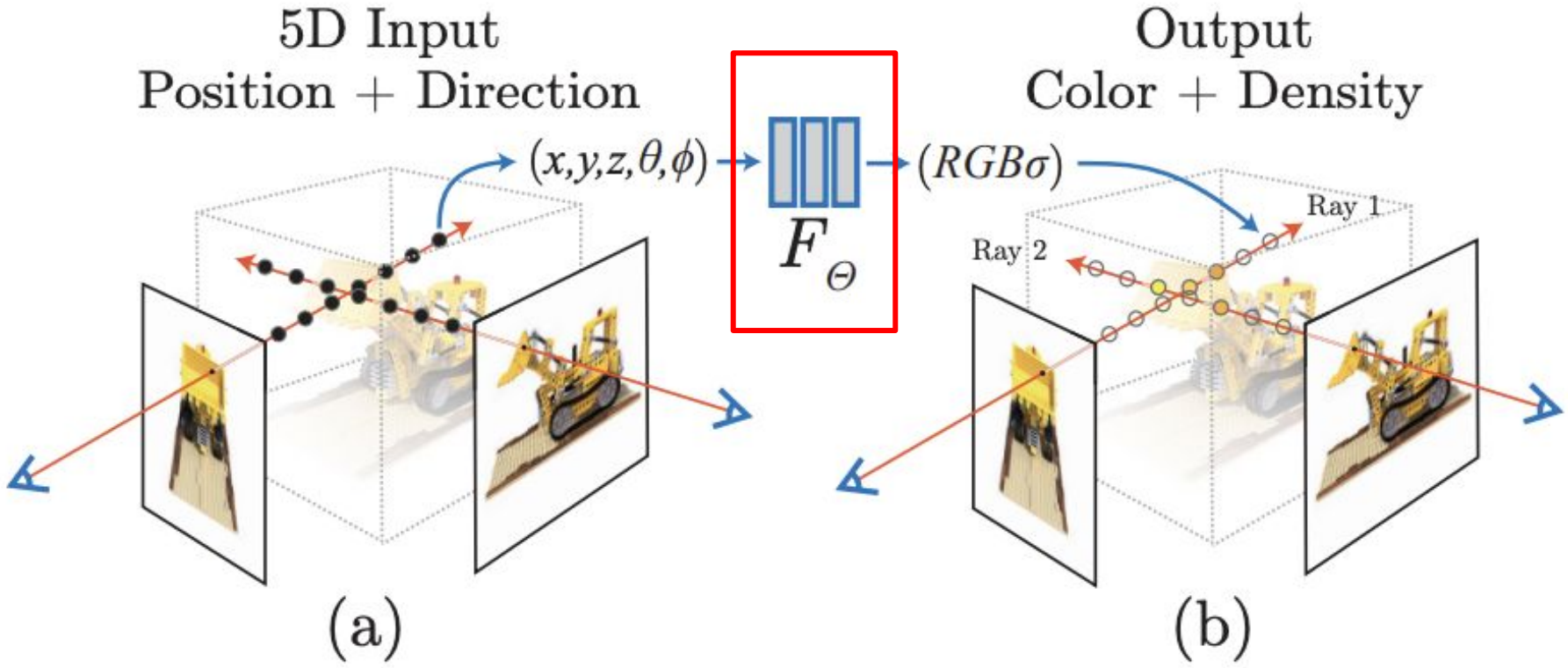
Shap-E · 2023 →

Shap-E



Shap-E · 2023 →

Shap-E



Shap-E · 2023 →

Shap-E

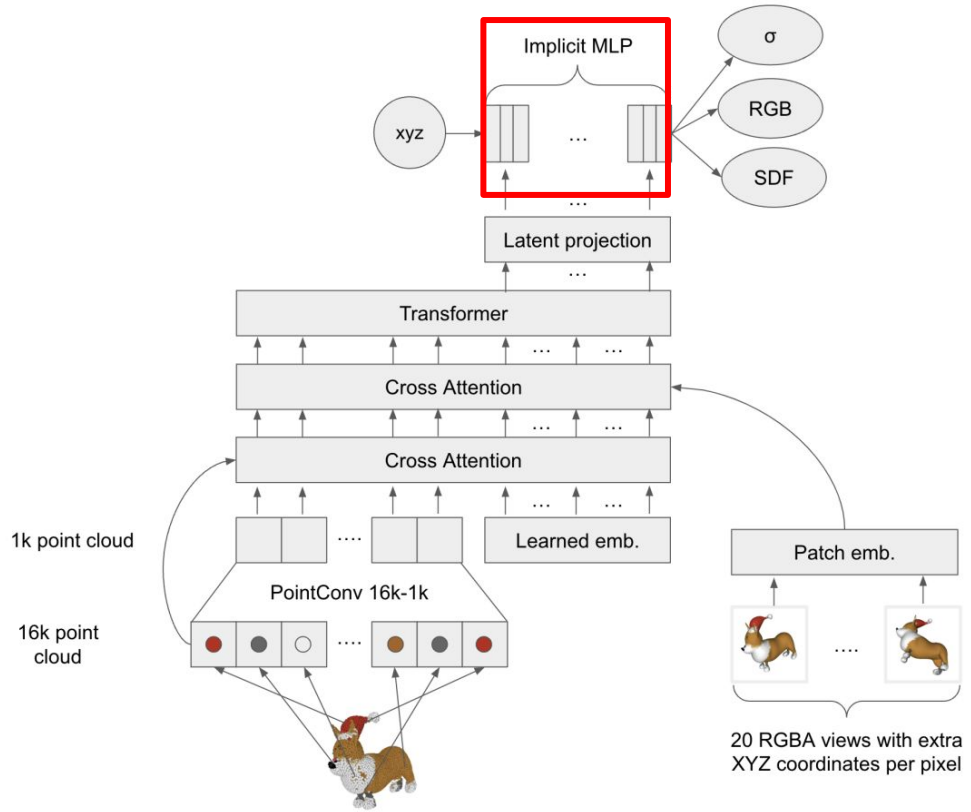


Figure 2: An overview of our encoder architecture. The encoder ingests both 16k resolution RGB point clouds and rendered RGBA images with augmented spatial coordinates for each foreground pixel. It outputs parameters of an MLP, which then acts as both a NeRF and a signed texture field (STF).

Inputs

16K-point RGB cloud + 20 rendered views

Latent = weights

Each latent vector is one row of an MLP layer: 4 layers of 256 × 256

Shap-E · 2023 →

Shap-E

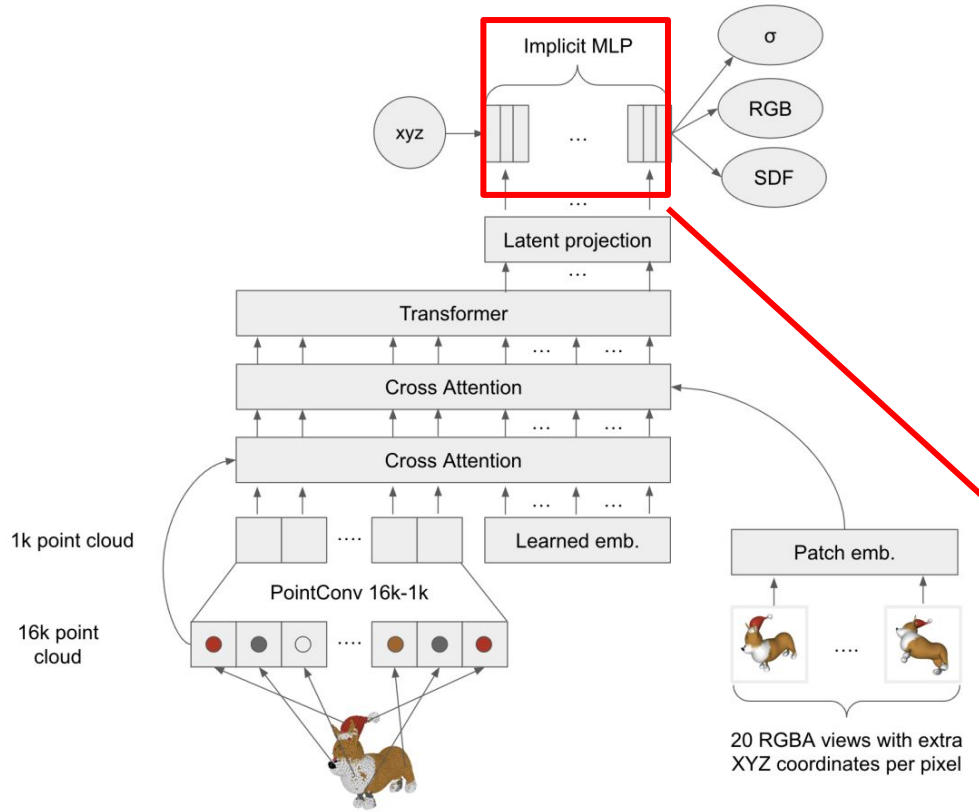


Figure 2: An overview of our encoder architecture. The encoder ingests both 16k resolution RGB point clouds and rendered RGBA images with augmented spatial coordinates for each foreground pixel. It outputs parameters of an MLP, which then acts as both a NeRF and a signed texture field (STF).

Inputs

16K-point RGB cloud + 20 rendered views

Latent = weights

Each latent vector is one row of an MLP layer: 4 layers of 256 × 256

Train diffusion to predict the weights

Shap-E · 2023 →

Shap-E

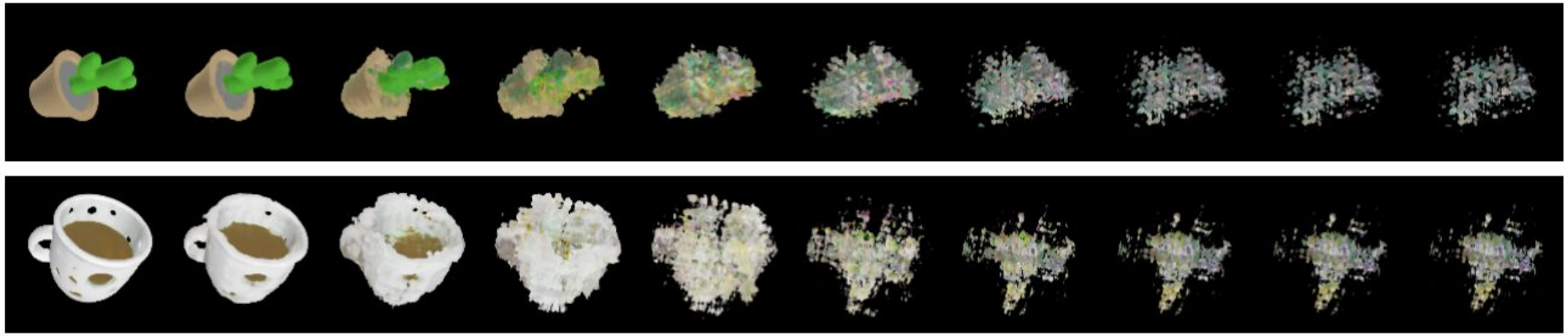


Figure 8: NeRF reconstructions of noised latents using our diffusion schedule $\bar{\alpha}_t = e^{-12t}$. The timestep t is linearly swept from 0 to 1 from left to right.

Shap-E · 2023 →

Shap-E

Stage 1 · encoder

3D asset → 1024 latent vectors = the weights of an implicit MLP

Stage 2 · diffusion

Point-E's transformer over the 1024 latent tokens, conditioned on CLIP text or image and predicts x_0

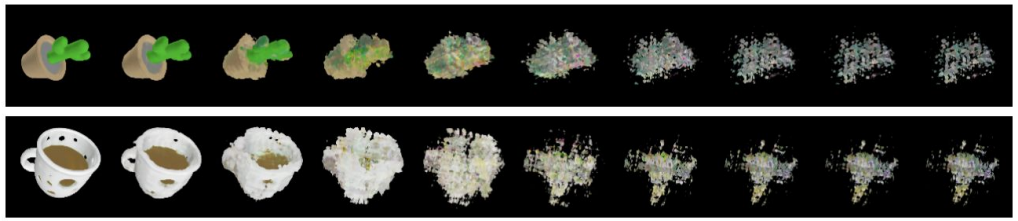


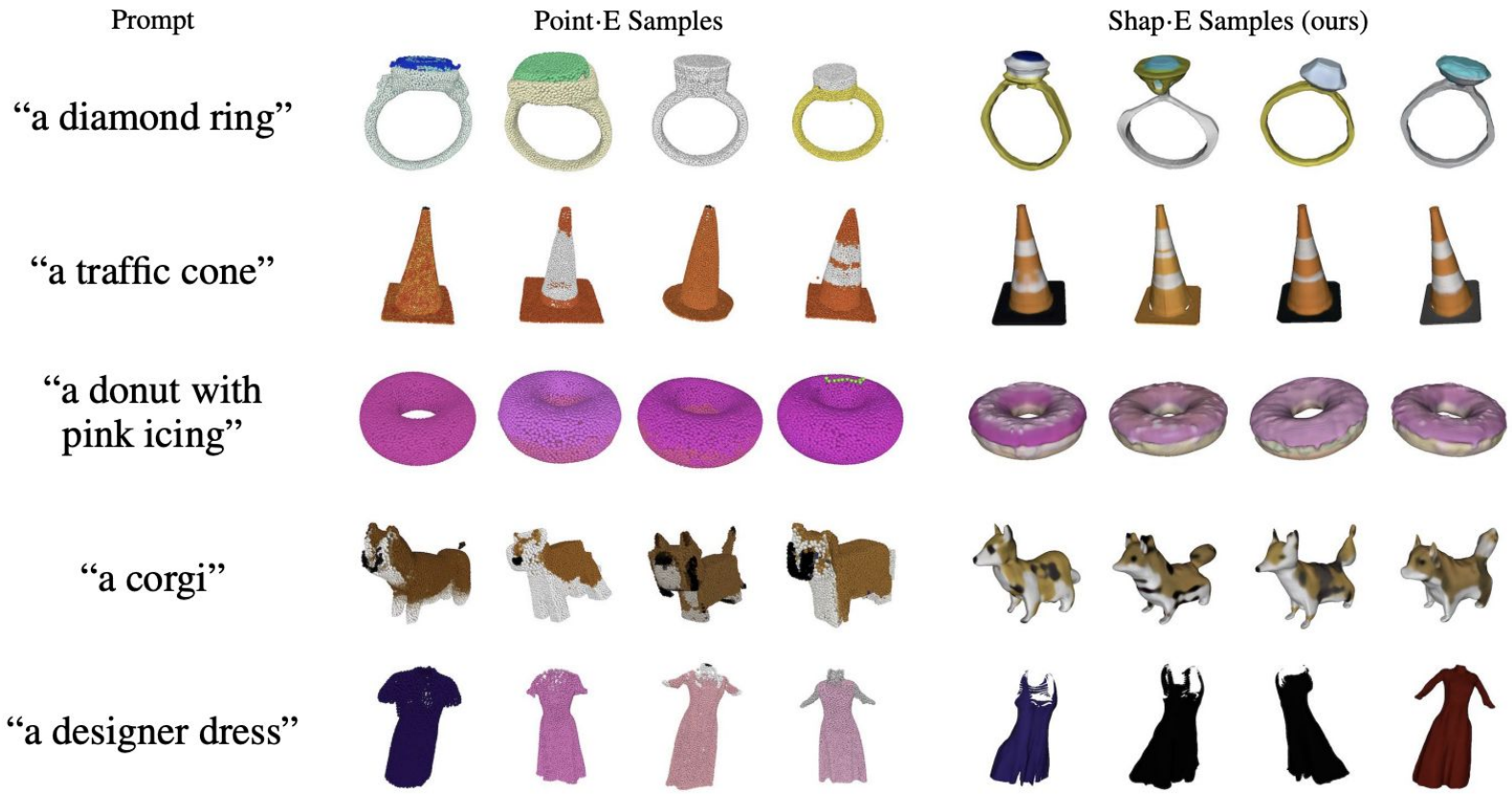
Figure 8: NeRF reconstructions of noised latents using our diffusion schedule $\bar{\alpha}_t = e^{-12t}$. The timestep t is linearly swept from 0 to 1 from left to right.



Text-conditional samples

Shap-E · 2023 →

Shap-E vs Point-E





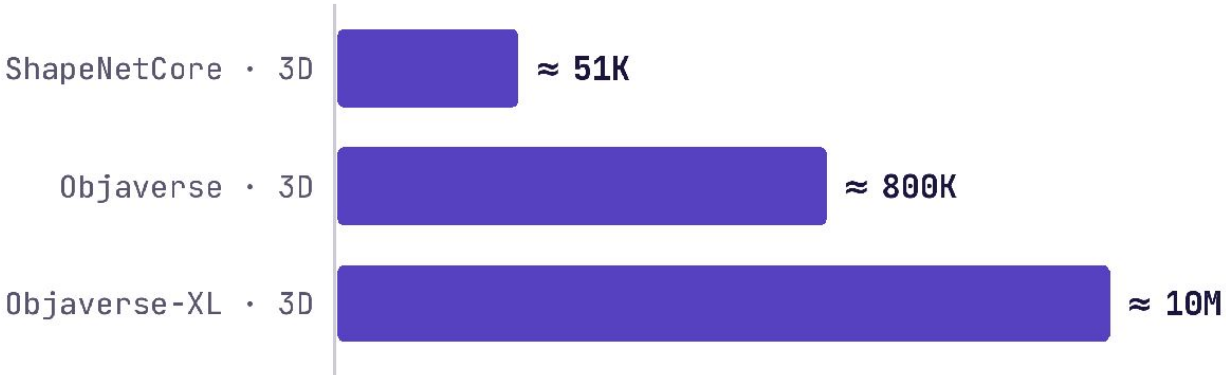
ShapeNet → Objaverse → Objaverse-XL →

DLS

Problems

- Lack of 3D data
- Unstable and difficult training





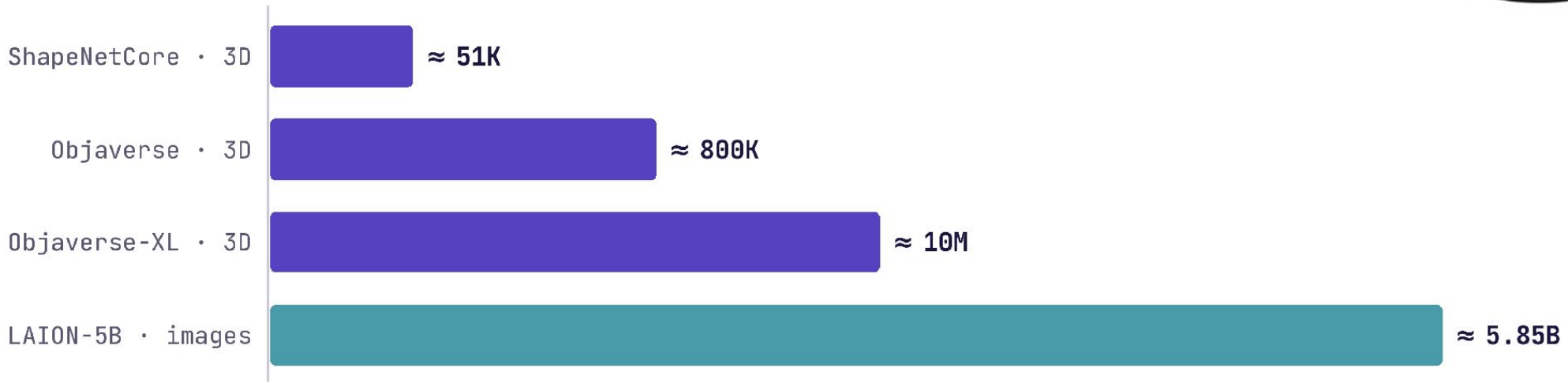
ShapeNet → Objaverse → Objaverse-XL → LAION-5B →

DLS

Problems

- Lack of 3D data
- Unstable and difficult training





ShapeNet → Objaverse → Objaverse-XL → LAION-5B →

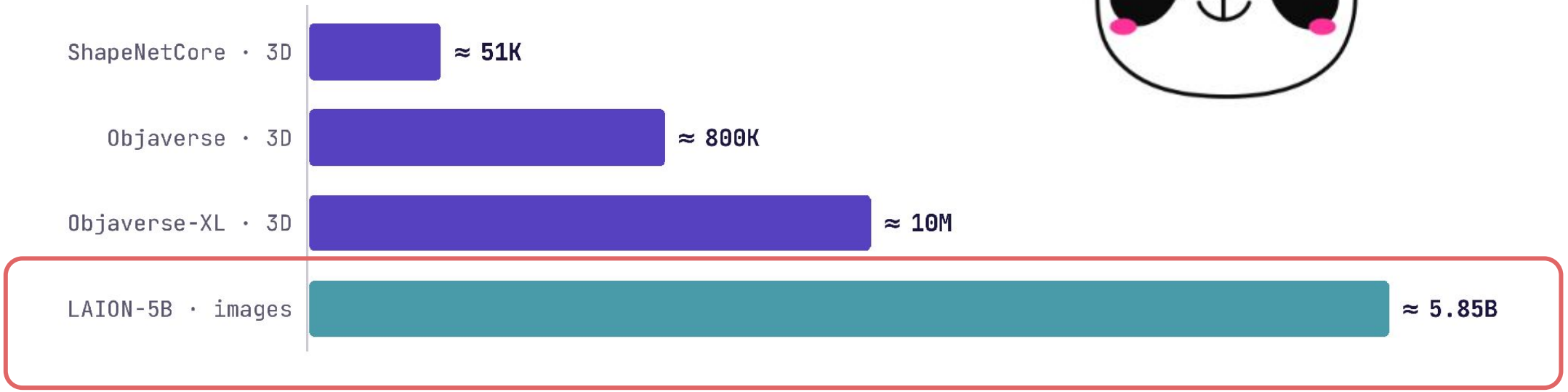
DLS

Problems

- Lack of 3D data
- Unstable and difficult training







DLS

PART 2

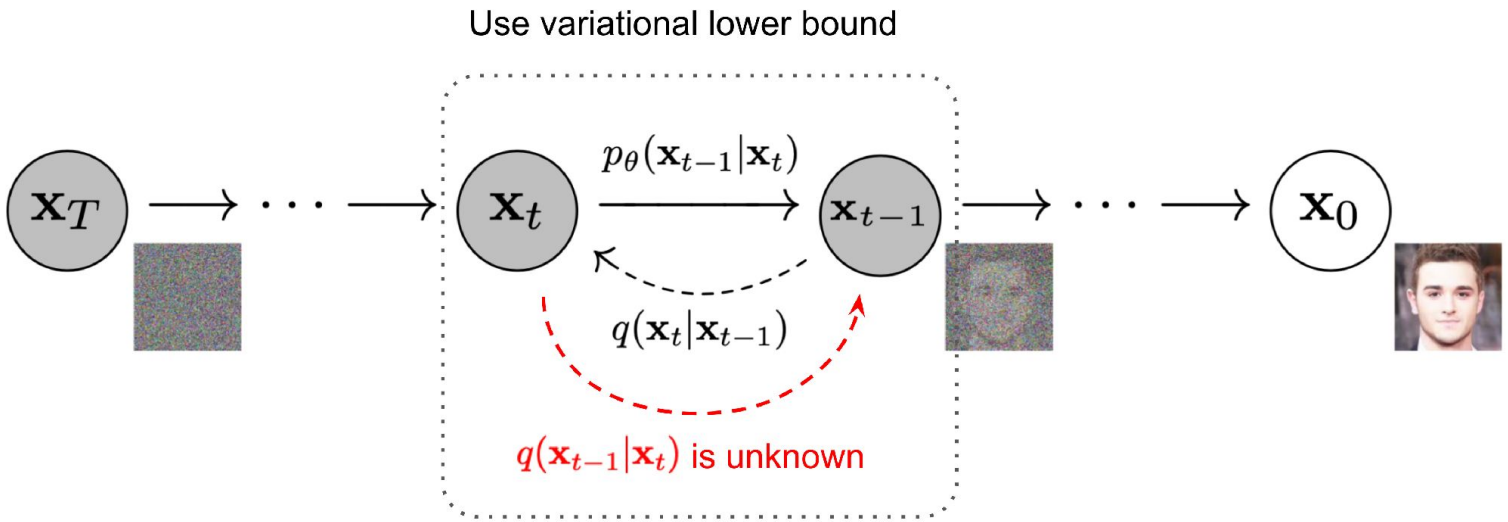
2D diffusion as a generative prior

Score distillation: optimize 3D so its renders look right to a 2D model

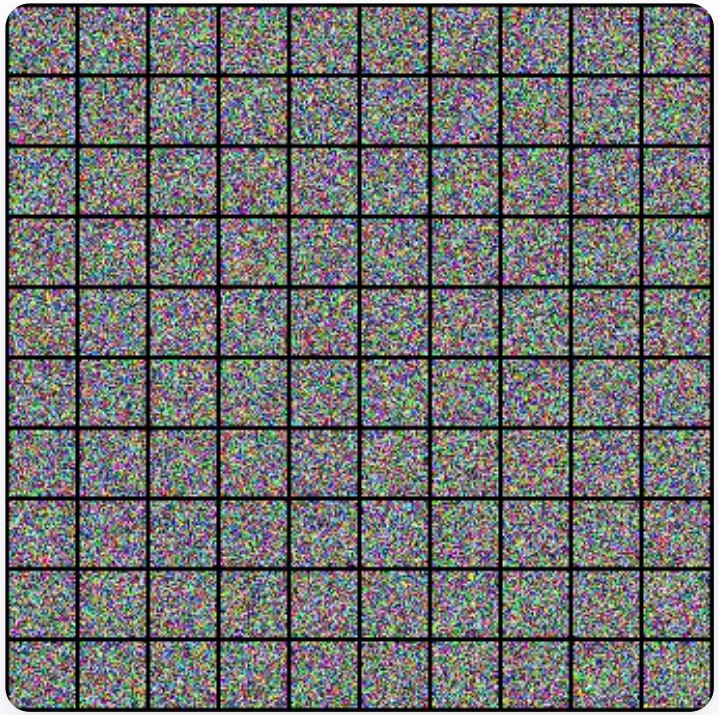


DDPM · 2020 →

2D diffusion



$$L_{simple,t} = \mathbb{E}_{x_0,\epsilon}\left[\|\epsilon_t - \epsilon_\theta(x_t, t)\|^2\right]$$



DDPM: learn to reverse a fixed noising process with a simple denoising loss

2D diffusion

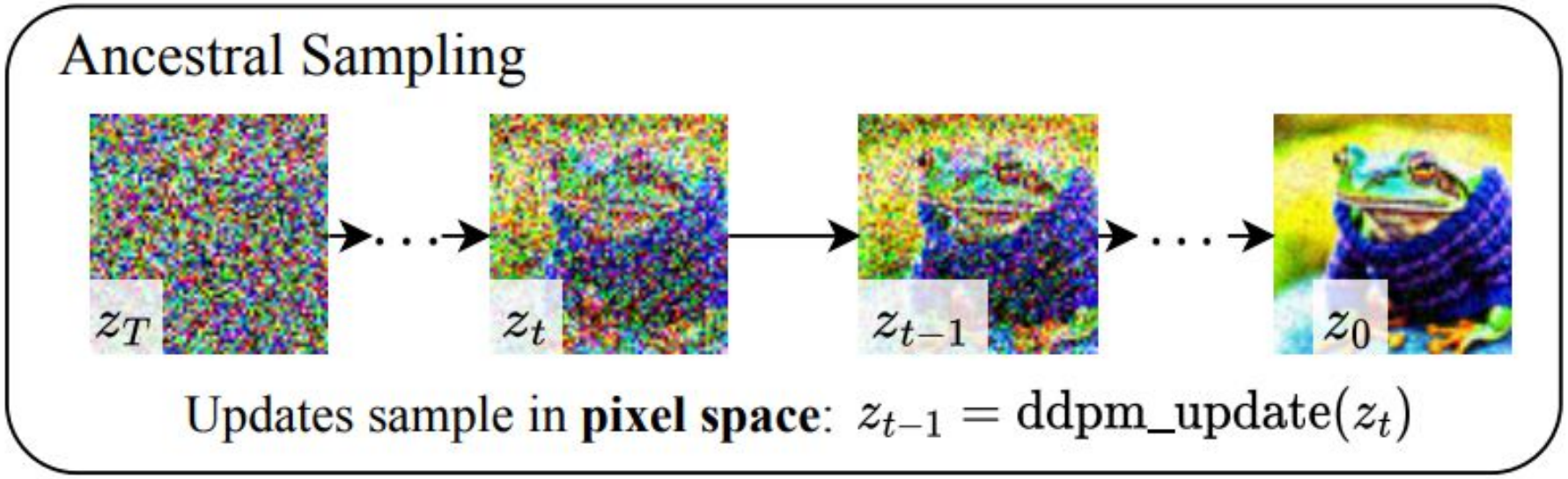




DLS

2D diffusion

Usual diffusion sampling



Generates a sample of the same type and dimensionality as the training data

DreamFusion · 2022 →

DLS

Text-to-3D: DreamFusion

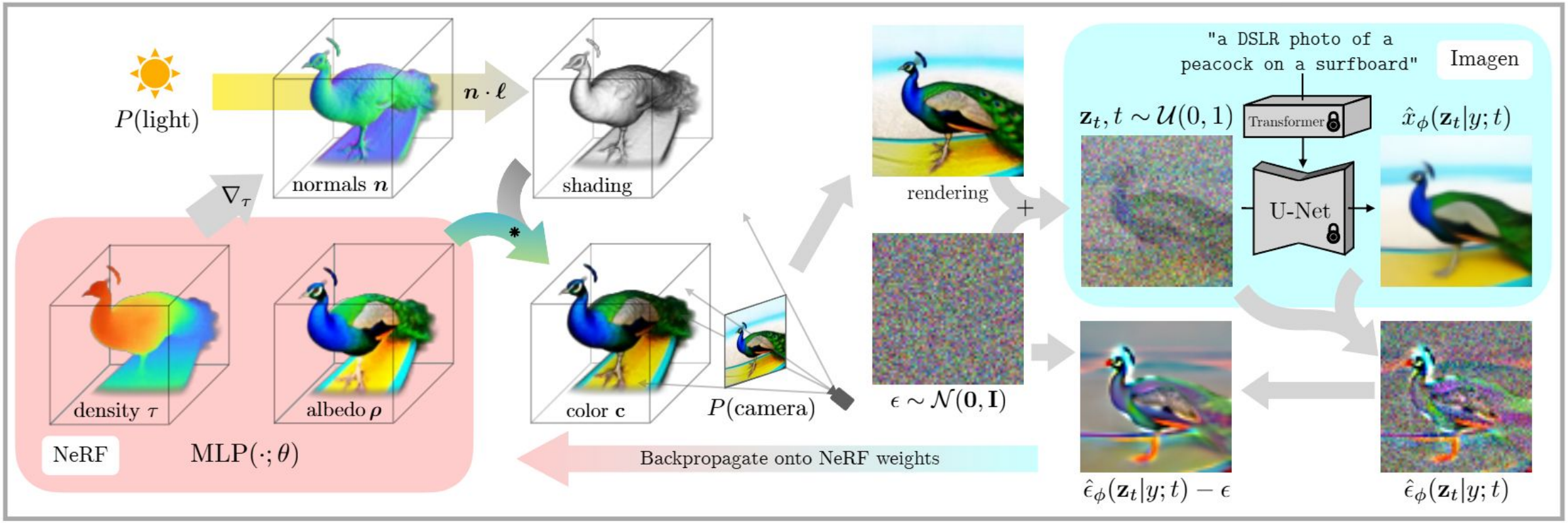
We want 3D models that look like good images when rendered from random angles





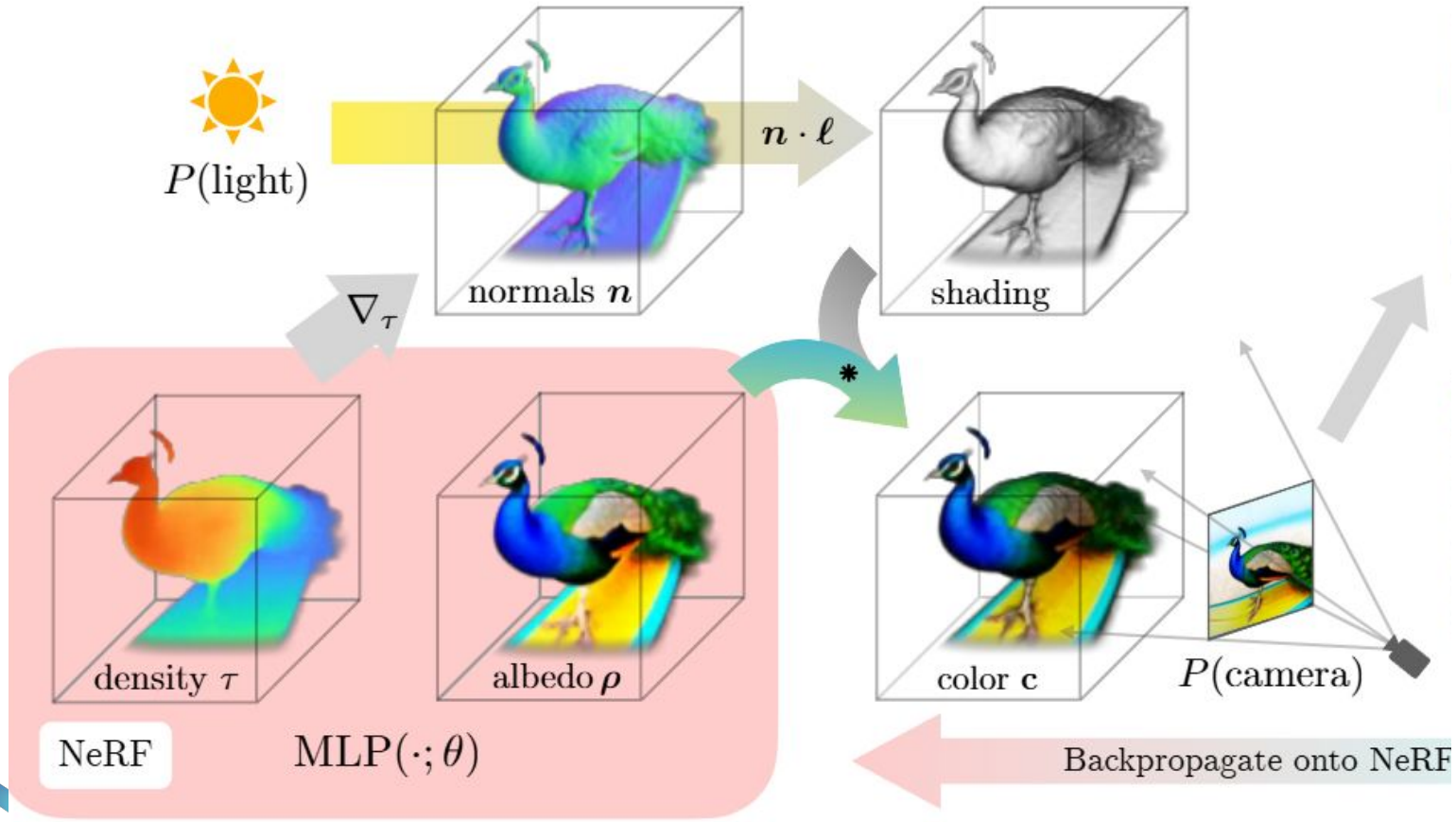
DreamFusion · 2022 →

Text-to-3D: DreamFusion



DreamFusion · 2022 →

Text-to-3D: DreamFusion



NeRF: density + albedo

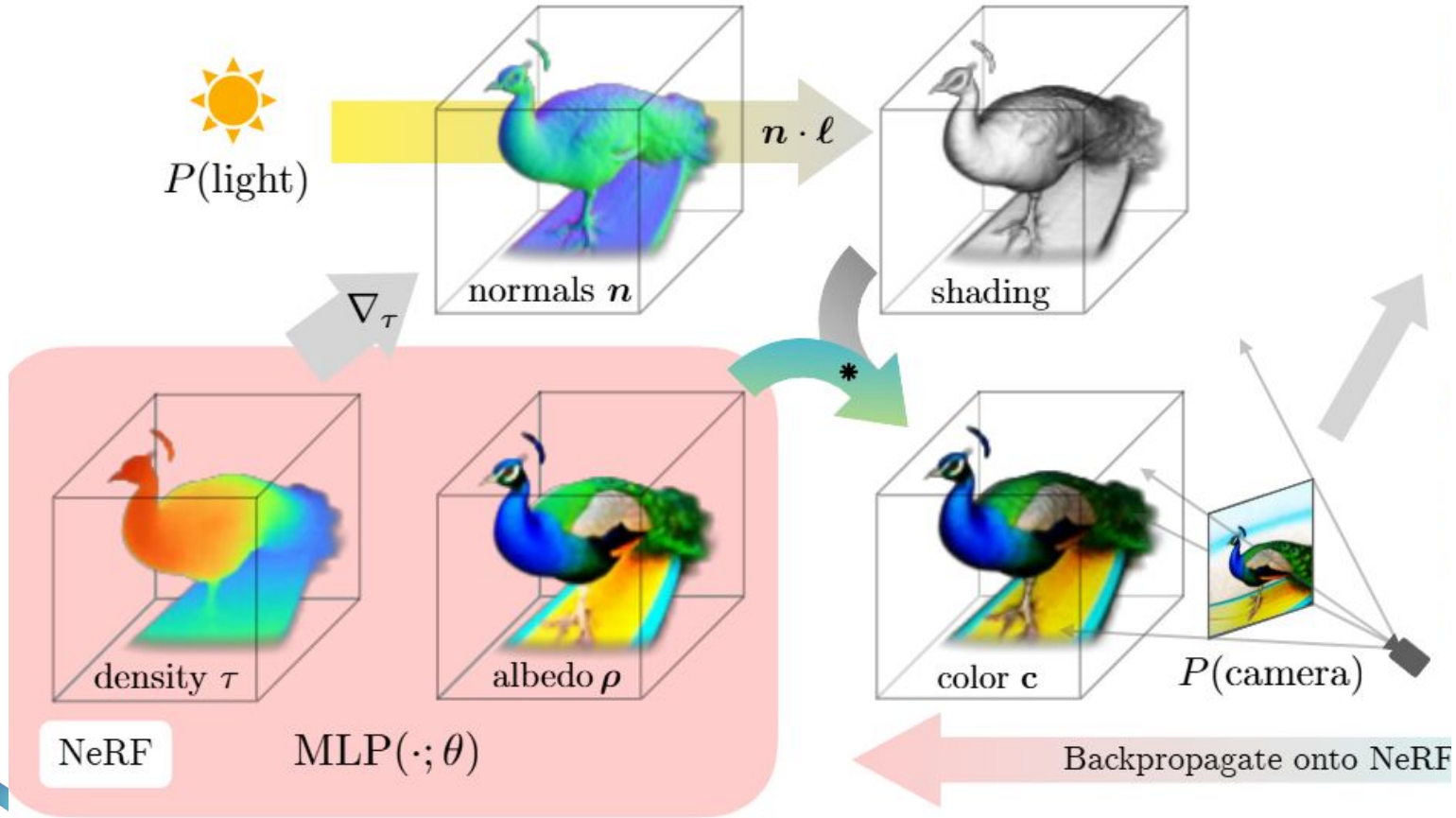
$$(\tau,\ \rho) = \mathrm{MLP}(\mu;\ \theta)$$

DreamFusion · 2022 →

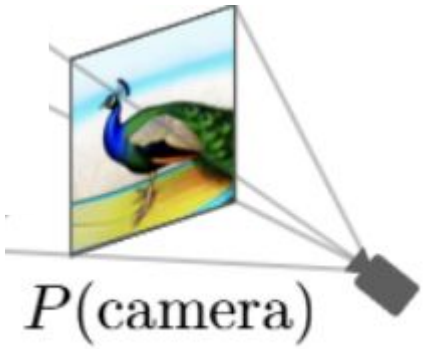
Text-to-3D: DreamFusion

NeRF: density + albedo

$$(\tau,\ \rho) = \mathrm{MLP}(\mu;\ \theta)$$







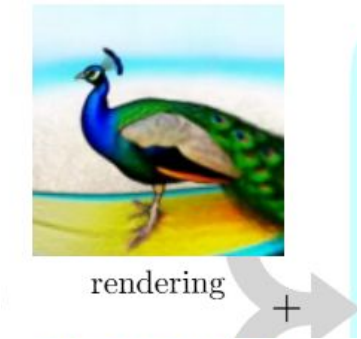
DreamFusion · 2022 →

DLS

Text-to-3D: DreamFusion

Calculate the loss

$$x = g(\theta)$$



1. Render

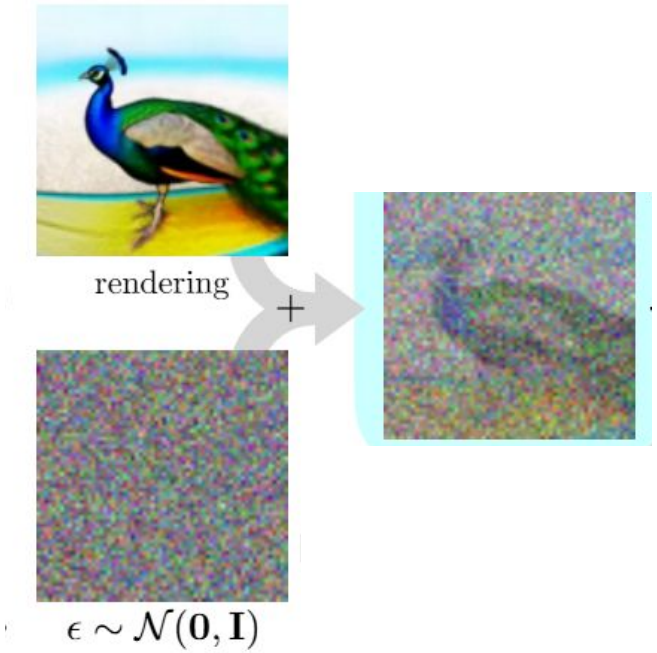
the NeRF, seen from a random camera

DreamFusion · 2022 →

Text-to-3D: DreamFusion

Calculate the loss

$$x = g(\theta)$$



1. Render

the NeRF, seen from a random camera

2. Add noise

$$\mathbf{z}_t = \alpha_t\, x + \sigma_t\, \epsilon$$

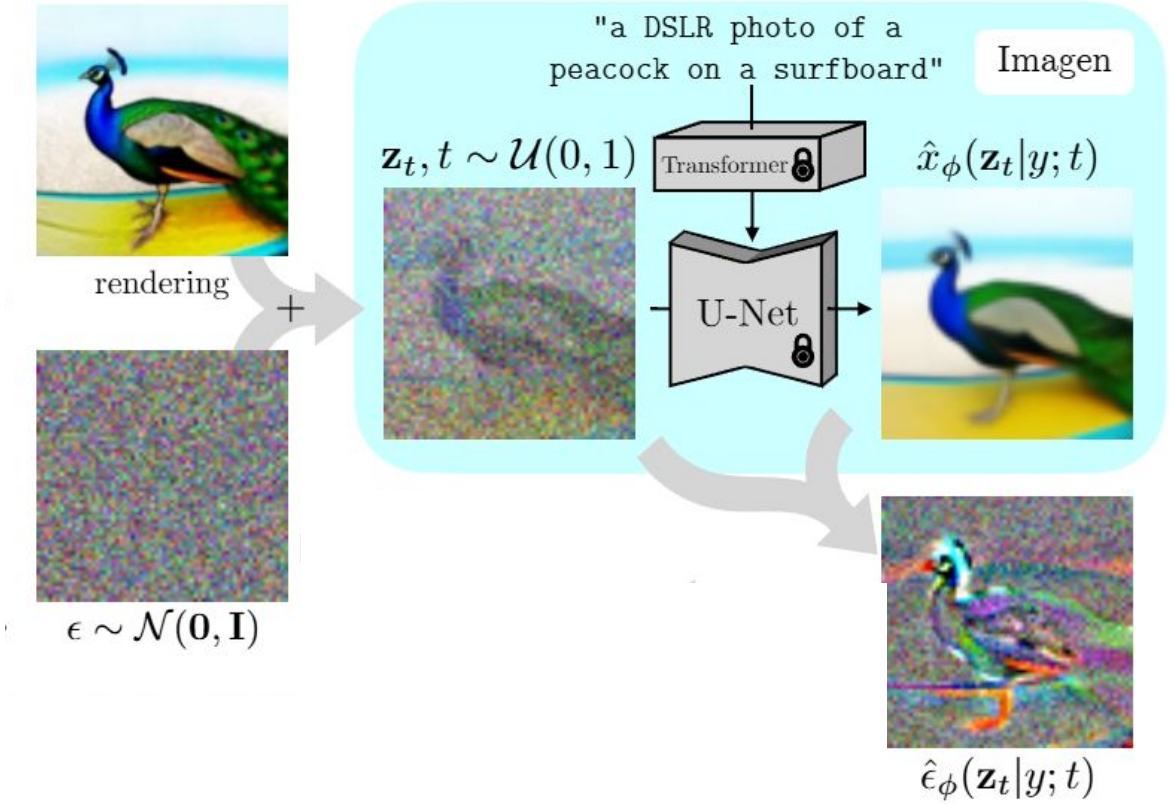
at a random noise level

DreamFusion · 2022 →

Text-to-3D: DreamFusion

Calculate the loss

$$x = g(\theta)$$



1. Render

the NeRF, seen from a random camera

2. Add noise

$$\mathbf{z}_t = \alpha_t\, x + \sigma_t\, \epsilon$$

at a random noise level

3. Frozen Imagen predicts the noise

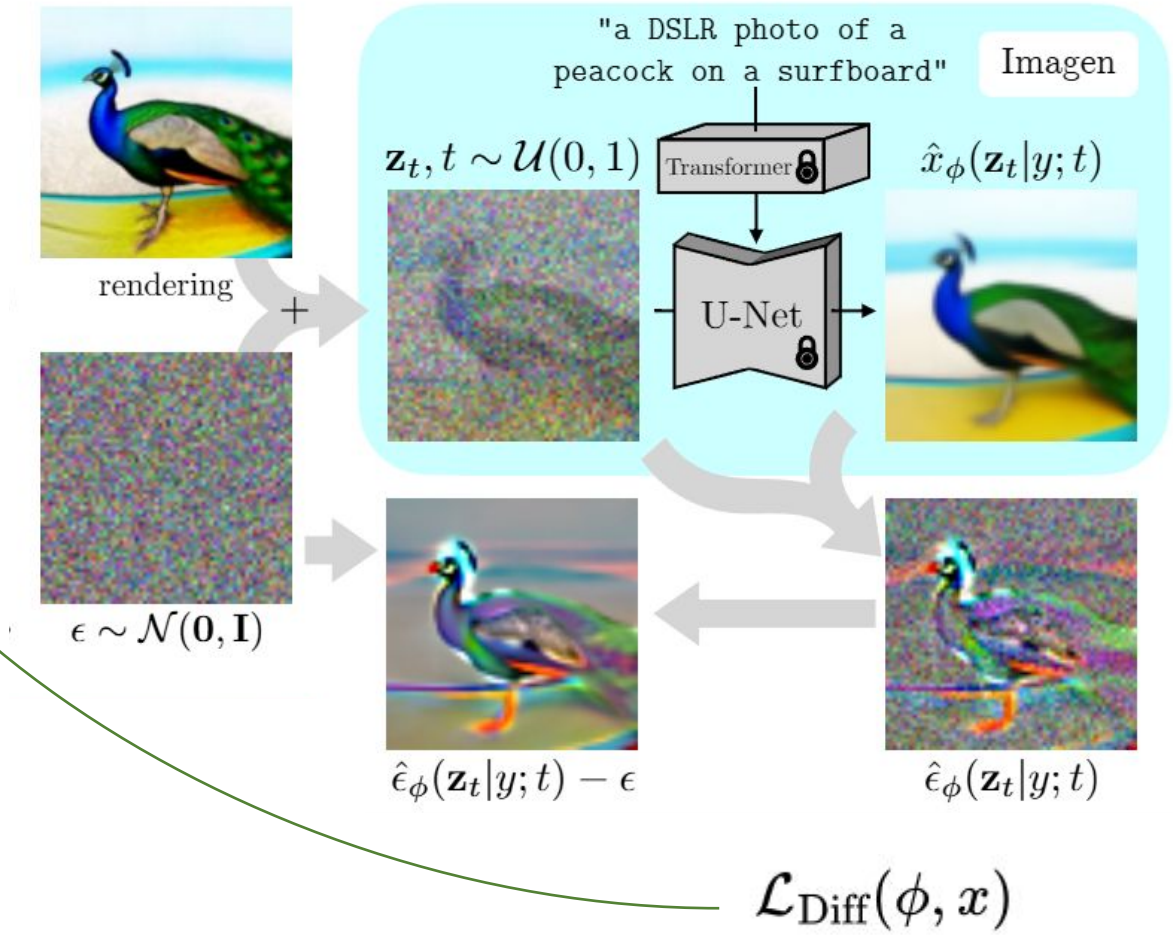
$$\hat{\epsilon}_\phi(\mathbf{z}_t \mid y;\ t)$$

DreamFusion · 2022 →

Text-to-3D: DreamFusion

Calculate the loss

$$x = g(\theta)$$



1. Render

the NeRF, seen from a random camera

2. Add noise

$$\mathbf{z}_t = \alpha_t\, x + \sigma_t\, \epsilon$$

at a random noise level

3. Frozen Imagen predicts the noise

$$\hat{\epsilon}_\phi(\mathbf{z}_t \mid y;\ t)$$

4. Diffusion loss

$$\mathcal{L}_{\text{Diff}} = \mathbb{E}_{t,\epsilon}\left[\, w(t)\, \|\hat{\epsilon}_\phi - \epsilon\|^2\right]$$

DreamFusion · 2022 →

DLS

Text-to-3D: DreamFusion

$$\mathcal{L}_{\text{Diff}}(\phi, \mathbf{x}) = \mathbb{E}_{t\sim\mathcal{U}(0,1),\epsilon\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\left[w(t)\|\epsilon_\phi(\alpha_t\mathbf{x} + \sigma_t\epsilon; t) - \epsilon\|_2^2\right],$$

$$\nabla_\theta\mathcal{L}_{\text{Diff}}(\phi, \mathbf{x} = g(\theta)) = \mathbb{E}_{t,\epsilon}\Bigg[w(t)\underbrace{(\hat{\epsilon}_\phi(\mathbf{z}_t; y, t) - \epsilon)}_{\text{Noise Residual}}\underbrace{\frac{\partial\hat{\epsilon}_\phi(\mathbf{z}_t; y, t)}{\mathbf{z}_t}}_{\text{U-Net Jacobian}}\underbrace{\frac{\partial\mathbf{x}}{\partial\theta}}_{\text{Generator Jacobian}}\Bigg]$$

DreamFusion · 2022 →

DLS

Text-to-3D: DreamFusion

$$\mathcal{L}_{\text{Diff}}(\phi, \mathbf{x}) = \mathbb{E}_{t\sim\mathcal{U}(0,1),\epsilon\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\left[w(t)\|\epsilon_\phi(\alpha_t\mathbf{x} + \sigma_t\epsilon; t) - \epsilon\|_2^2\right],$$

$$\nabla_\theta\mathcal{L}_{\text{Diff}}(\phi, \mathbf{x} = g(\theta)) = \mathbb{E}_{t,\epsilon}\Bigg[w(t)\underbrace{(\hat{\epsilon}_\phi(\mathbf{z}_t; y, t) - \epsilon)}_{\text{Noise Residual}}\underbrace{\frac{\partial\hat{\epsilon}_\phi(\mathbf{z}_t; y, t)}{\mathbf{z}_t}}_{\text{U-Net Jacobian}}\underbrace{\frac{\partial\mathbf{x}}{\partial\theta}}_{\text{Generator Jacobian}}\Bigg]$$

DreamFusion · 2022 →

DLS

Text-to-3D: DreamFusion

$$\mathcal{L}_{\text{Diff}}(\phi, \mathbf{x}) = \mathbb{E}_{t\sim\mathcal{U}(0,1),\epsilon\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\left[w(t)\|\epsilon_\phi(\alpha_t\mathbf{x} + \sigma_t\epsilon; t) - \epsilon\|_2^2\right],$$

$$\nabla_\theta\mathcal{L}_{\text{Diff}}(\phi, \mathbf{x} = g(\theta)) = \mathbb{E}_{t,\epsilon}\left[w(t)\underbrace{(\hat{\epsilon}_\phi(\mathbf{z}_t; y, t) - \epsilon)}_{\text{Noise Residual}}\underbrace{\frac{\partial\hat{\epsilon}_\phi(\mathbf{z}_t; y, t)}{\mathbf{z}_t}}_{\text{U-Net Jacobian}}\underbrace{\frac{\partial\mathbf{x}}{\partial\theta}}_{\text{Generator Jacobian}}\right]$$

DreamFusion · 2022 →

DLS

Text-to-3D: DreamFusion

$$\mathcal{L}_{\text{Diff}}(\phi, \mathbf{x}) = \mathbb{E}_{t\sim\mathcal{U}(0,1),\epsilon\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\left[w(t)\|\epsilon_\phi(\alpha_t\mathbf{x} + \sigma_t\epsilon; t) - \epsilon\|_2^2\right],$$

$$\nabla_\theta\mathcal{L}_{\text{Diff}}(\phi, \mathbf{x} = g(\theta)) = \mathbb{E}_{t,\epsilon}\Big[w(t)\underbrace{(\hat{\epsilon}_\phi(\mathbf{z}_t; y, t) - \epsilon)}_{\text{Noise Residual}}\underbrace{\frac{\partial\hat{\epsilon}_\phi(\mathbf{z}_t; y, t)}{\mathbf{z}_t}}_{\text{U-Net Jacobian}}\underbrace{\frac{\partial\mathbf{x}}{\partial\theta}}_{\text{Generator Jacobian}}\Big]$$

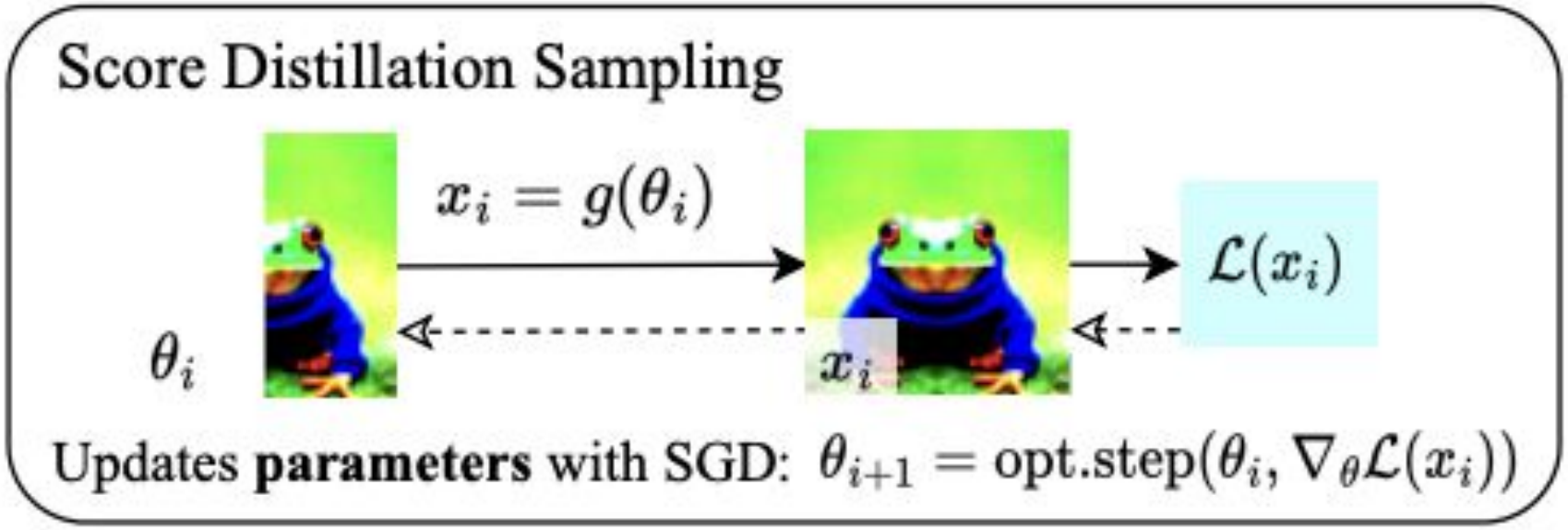
$$\nabla_\theta\mathcal{L}_{\text{SDS}}(\phi, \mathbf{x} = g(\theta)) \triangleq \mathbb{E}_{t,\epsilon}\left[w(t)\,(\hat{\epsilon}_\phi(\mathbf{z}_t; y, t) - \epsilon)\,\frac{\partial\mathbf{x}}{\partial\theta}\right]$$

DreamFusion · 2022 →

DLS

Text-to-3D: DreamFusion

$$\nabla_\theta \mathcal{L}_{\text{SDS}}(\phi, \mathbf{x} = g(\theta)) \triangleq \mathbb{E}_{t,\epsilon}\left[w(t)\left(\hat{\epsilon}_\phi(\mathbf{z}_t; y, t) - \epsilon\right)\frac{\partial \mathbf{x}}{\partial \theta}\right]$$



DreamFusion · 2022 →

SJC · 2022 →

DLS

DreamFusion: details that matter

Prior	Imagen, 64 × 64 px
3D	mip-NeRF 360 + shading: albedo, random light, textureless renders
View prompts	“front view”, “side view”, “back view”
Guidance	$\omega = 100$ classifier-free guidance (vs. 7.5 for images)
Time	~1.5 h per object on TPUv4



A DreamFusion sample

Perp-Neg · 2023 →

SDS: problems

Oversaturation

Guidance scale 100 → saturated colors, over-smoothed textures

Low diversity

Mode seeking: different seeds, almost the same object

Janus

No notion of camera: a face on every side

Slow

Hours per object

low-resolution prior



Perp-Neg · 2023 →

SDS: problems

Oversaturation

Guidance scale 100 → saturated colors, over-smoothed textures

Low diversity

Mode seeking: different seeds, almost the same object

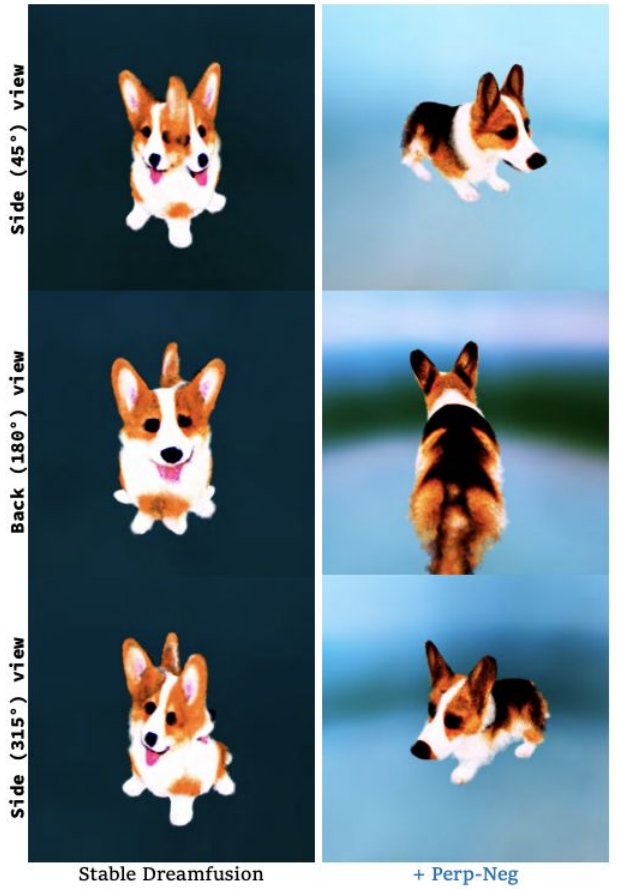
Janus

No notion of camera: a face on every side

Slow

Hours per object

low-resolution prior



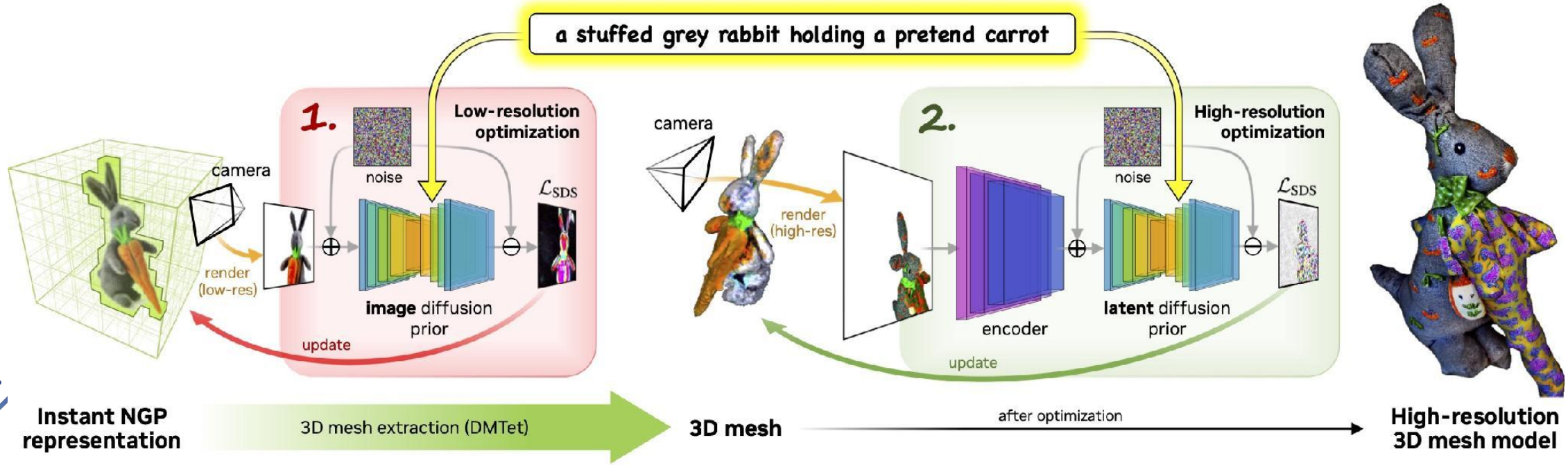
Janus: Stable DreamFusion vs. + Perp-Neg

Magic3D · 2022 →

Coarse-to-fine: Magic3D

Low resolution → 2-stage training

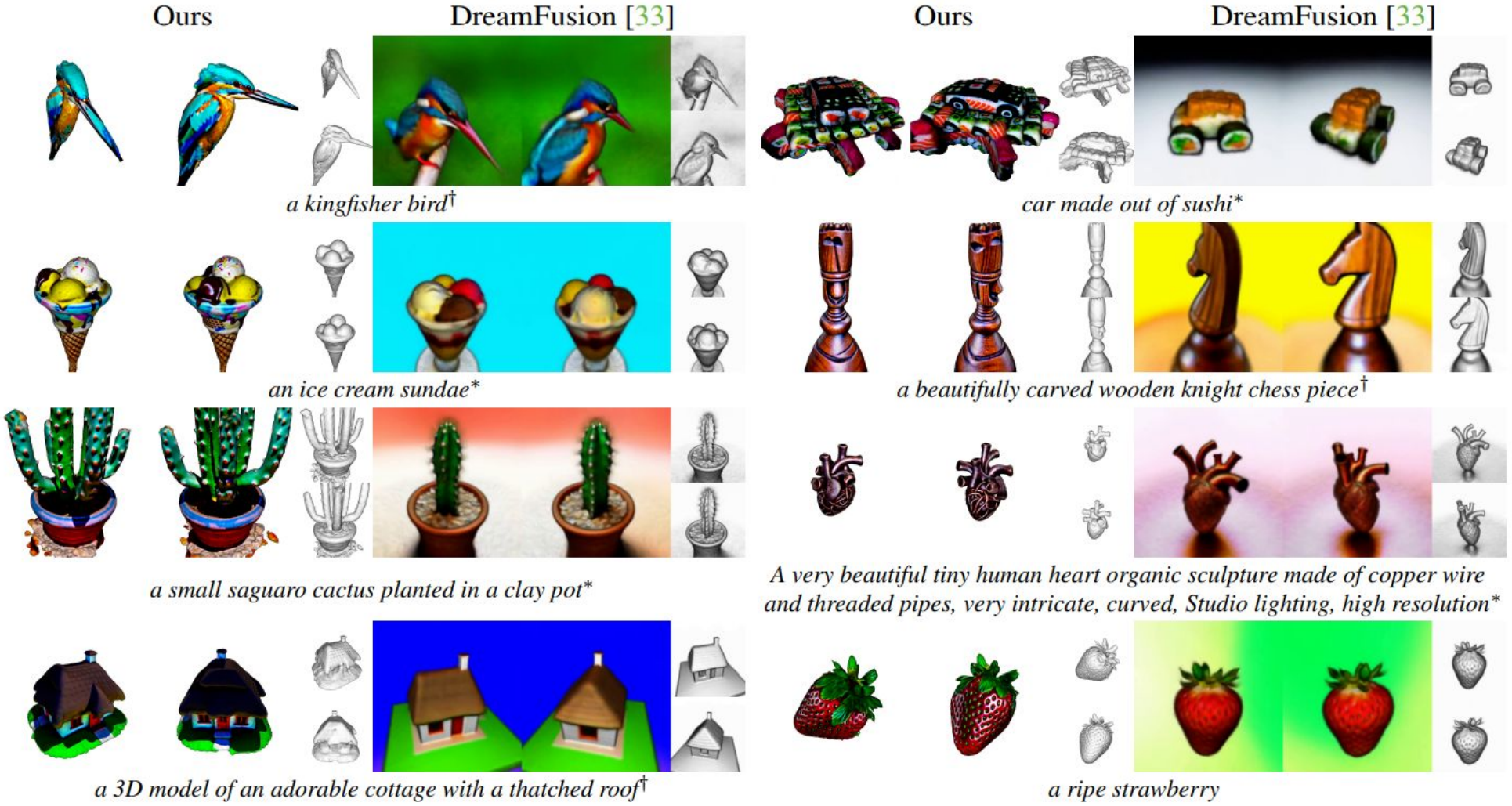
1. Coarse: NeRF (Instant-NGP) with a 64 × 64 prior
2. Fine: extract a mesh, optimize it with DMTet under a 512 × 512 latent prior



Magic3D · 2022 →

DLS

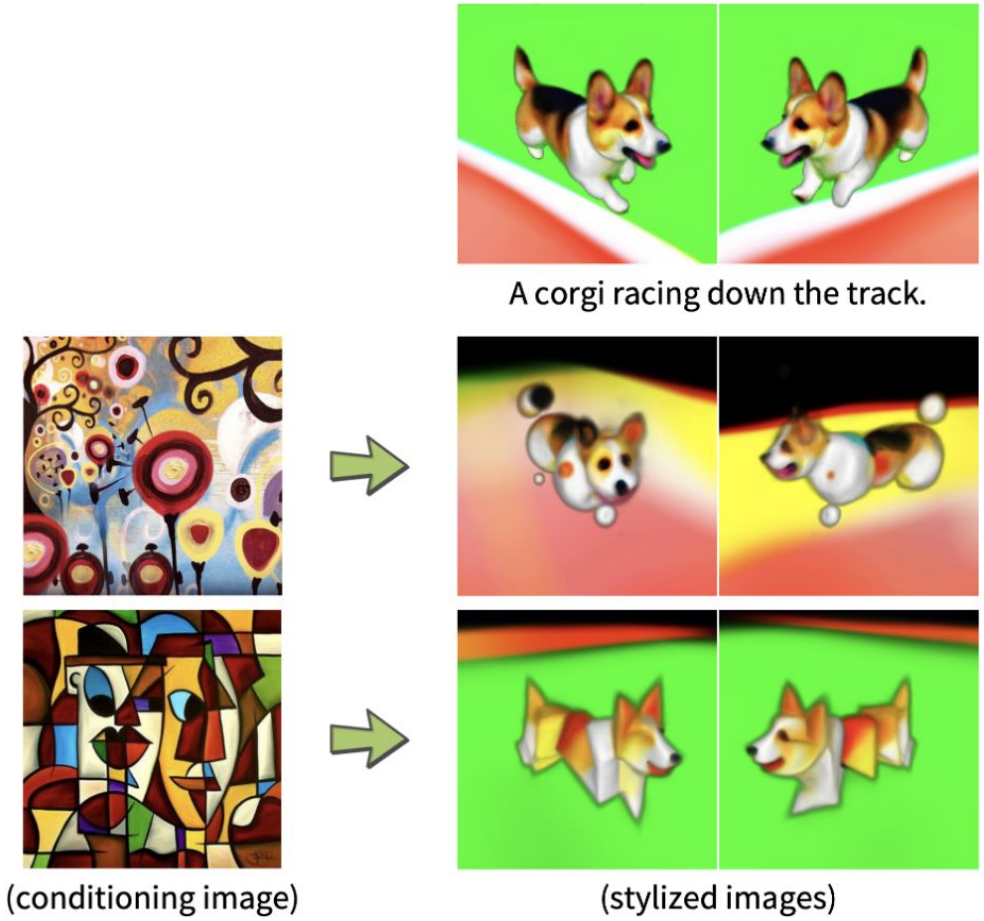
Coarse-to-fine: Magic3D

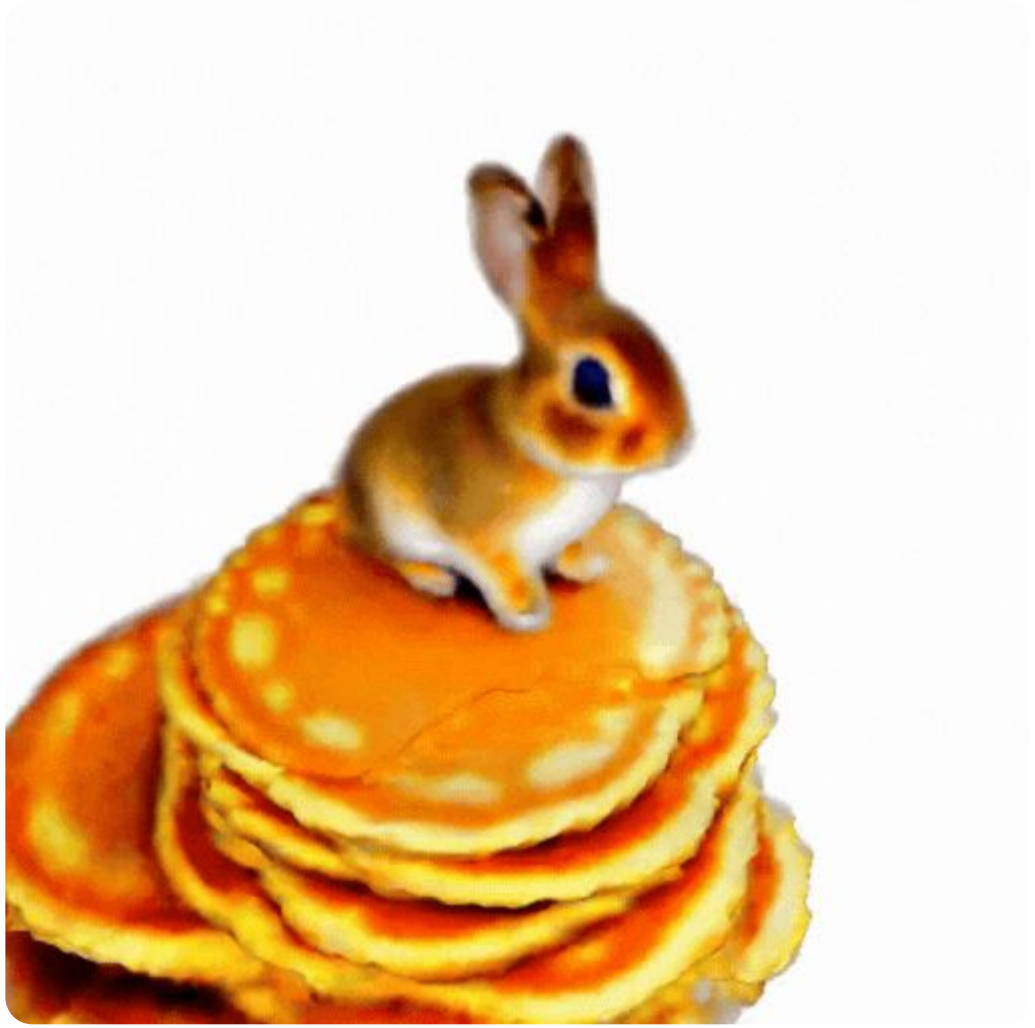


Magic3D · 2022 →

DLS

Coarse-to-fine: Magic3D





Magic3D · 2022 →

DLS

Magic3D: main problems

Still consistency problems

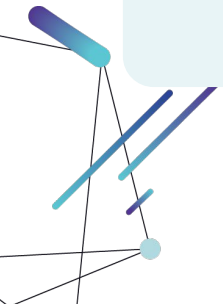
Janus faces and duplicated parts: the 2D prior doesn’t know where the camera is

Optimization for each prompt

Every new prompt starts a new optimization from scratch

Time

Still ~40 min per object, even with coarse-to-fine



Magic3D · 2022 →

Magic3D: main problems

Still consistency problems

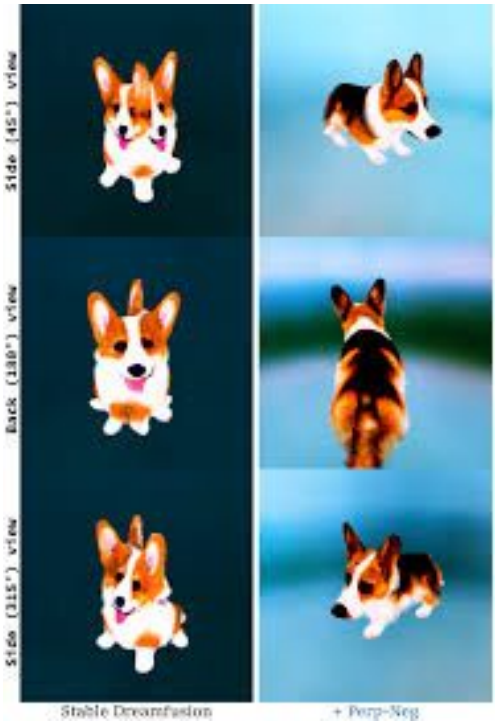
Janus faces and duplicated parts: the 2D prior doesn’t know where the camera is

Optimization for each prompt

Every new prompt starts a new optimization from scratch

Time

Still ~40 min per object, even with coarse-to-fine



Janus: Stable DreamFusion vs. + Perp-Neg

Magic3D · 2022 →

DLS

Magic3D: main problems

Still consistency problems

Janus faces and duplicated parts: the 2D prior doesn't know where the camera is

→ camera knowledge



Optimization for each prompt

Every new prompt starts a new optimization from scratch

Time

Still ~40 min per object, even with coarse-to-fine

Zero-1-to-3 · 2023 →

Zero-1-to-3

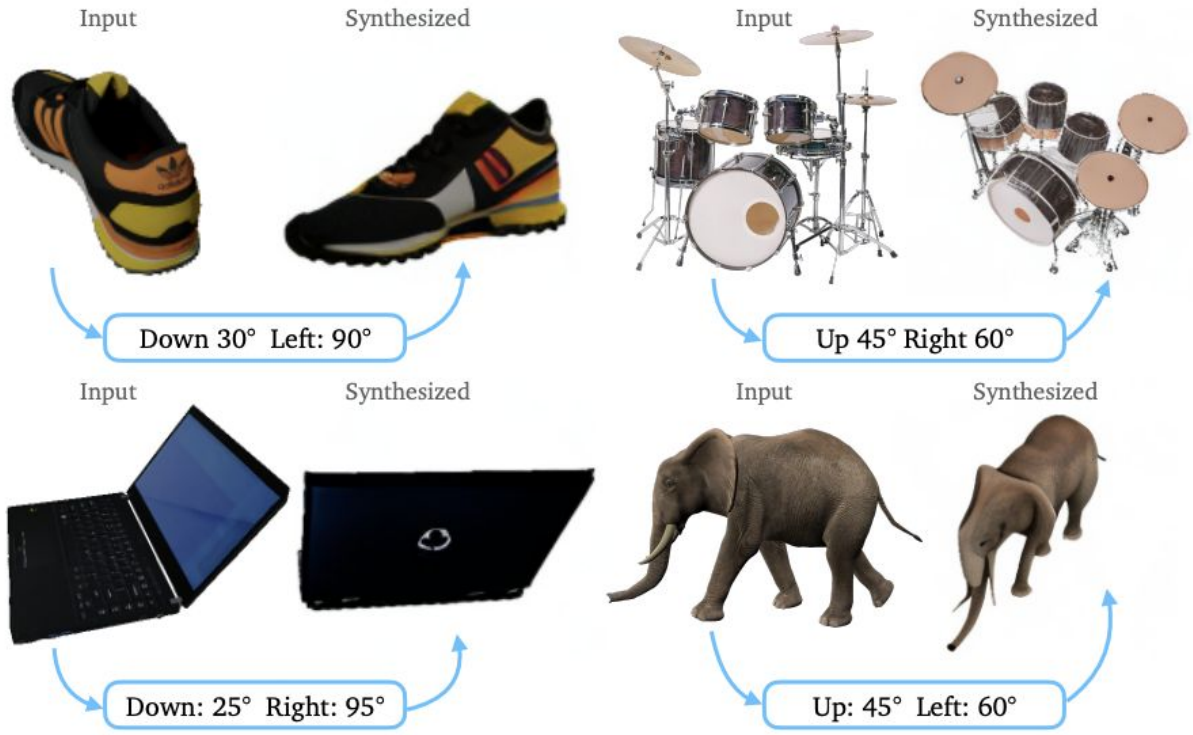
View-conditioned diffusion

Stable Diffusion fine-tuned on Objaverse renders: an input image plus a relative camera rotation and translation → the same object from a new view

$$\hat{x}_{R,T} = f(x, R, T)$$

As an SDS prior

Image-to-3D with a model that knows where the camera is



Novel views synthesized from a single input image

Magic3D · 2022 →

Magic3D: main problems

Still consistency problems

Janus faces and duplicated parts: the 2D prior doesn't know where the camera is

Optimization for each prompt

Every new prompt starts a new optimization from scratch

Time

Still ~40 min per object, even with coarse-to-fine

→ multi-view diffusion



MVDream · 2023 →

MVDream: multi-view prior for SDS

Four views at once

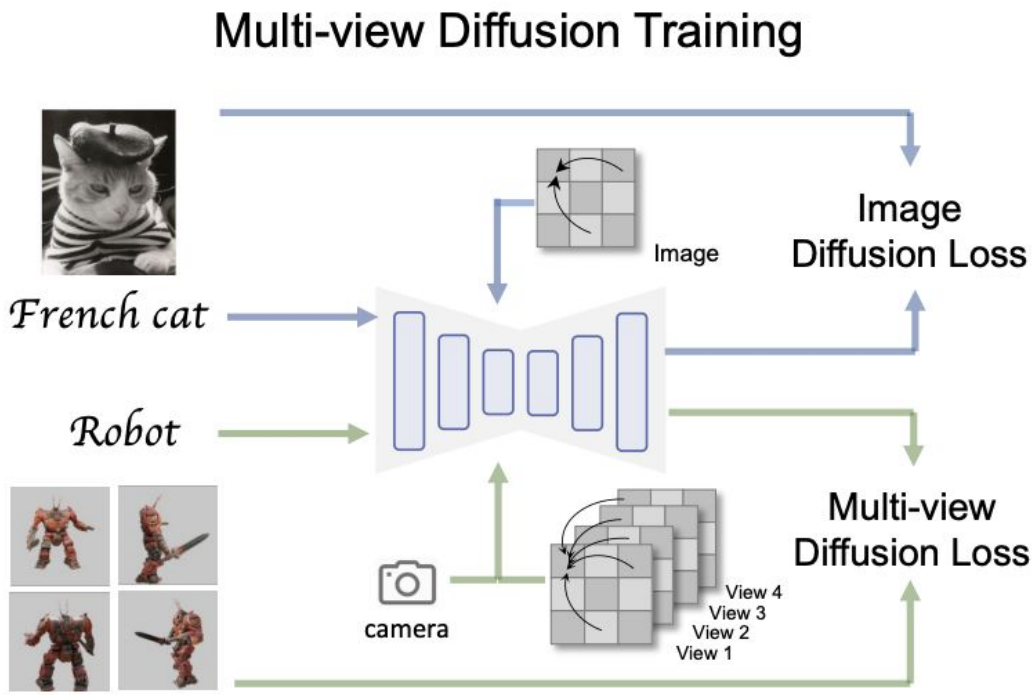
Self-attention spans all views; a camera embedding is added to the timestep embedding

Data

Objaverse renders + LAION images, to keep 2D diversity

As an SDS prior

The Janus problem almost disappears



Multi-view diffusion training with image and multi-view losses

Magic3D · 2022 →

Magic3D: main problems

Still consistency problems

Janus faces and duplicated parts: the 2D prior doesn't know where the camera is

Optimization for each prompt

Every new prompt starts a new optimization from scratch

Time

Still ~40 min per object, even with coarse-to-fine

→ Gaussians?



DreamGaussian · 2023 →

GaussianDreamer · 2023 →

DreamGaussian: SDS on Gaussians

Gaussians instead of NeRF

Rasterization is much faster than volume rendering

Stage 2 · mesh

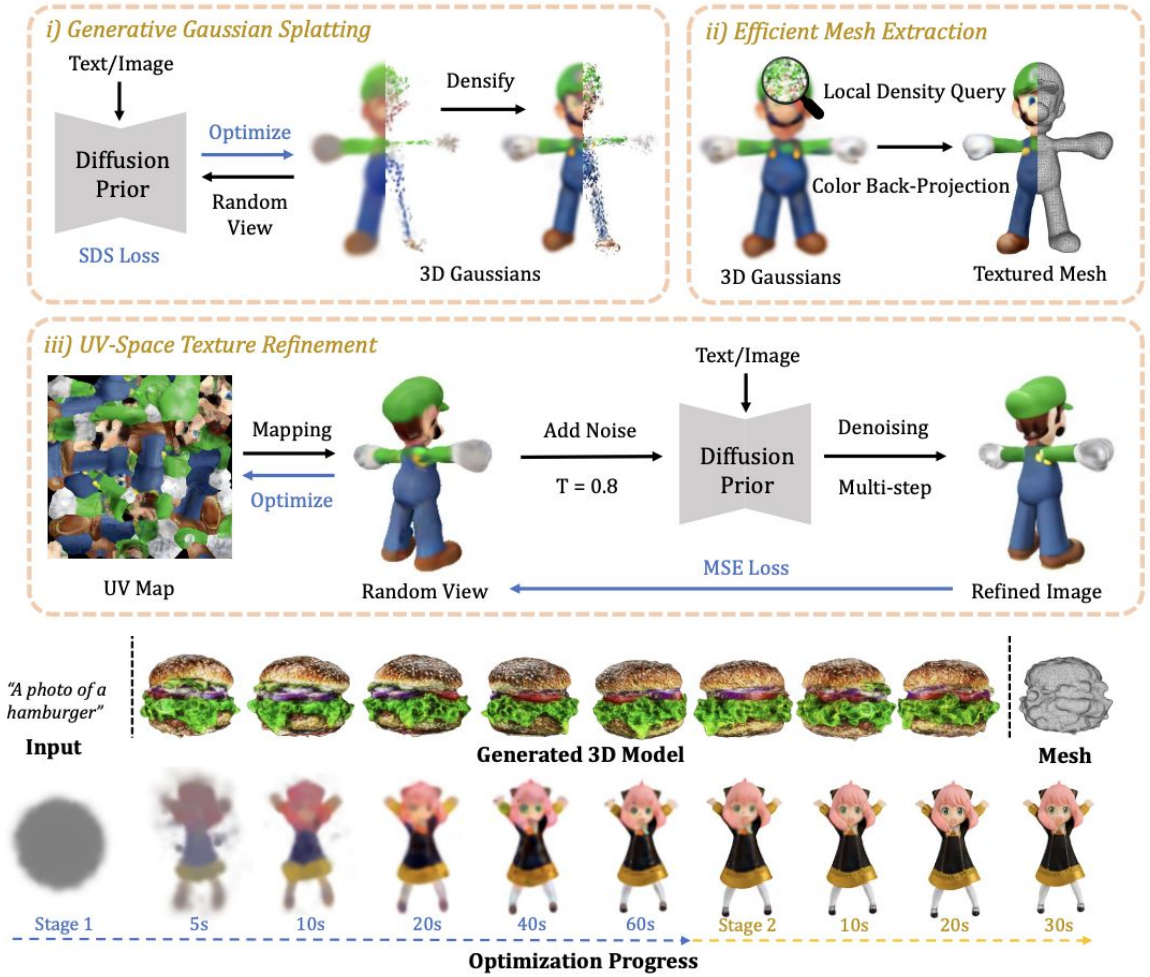
Extract a mesh and refine its UV texture

≈ 2 min image-to-3D

About 10× faster than NeRF-based SDS

GaussianDreamer

Initialize from Shap-E / Point-E; 2D diffusion adds the detail



Magic3D · 2022 →

Magic3D: main problems

Still consistency problems

Janus faces and duplicated parts: the 2D prior doesn’t know where the camera is

Optimization for each prompt

Every new prompt starts a new optimization from scratch

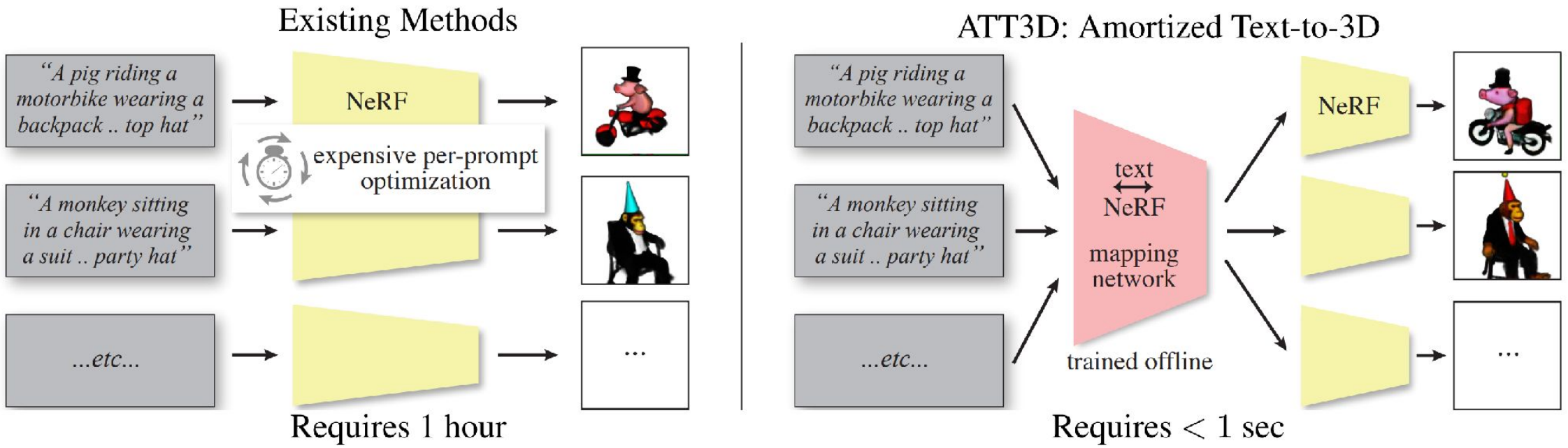
Time

Still ~40 min per object, even with coarse-to-fine



ATT3D · 2023 →

ATT3D: one model, many prompts



Per-prompt optimization vs. one amortized model

How: a text-conditioned mapping network modulates the NeRF grid features; trained with SDS on many prompts at once

ATT3D · 2023 →

ATT3D: one model, many prompts

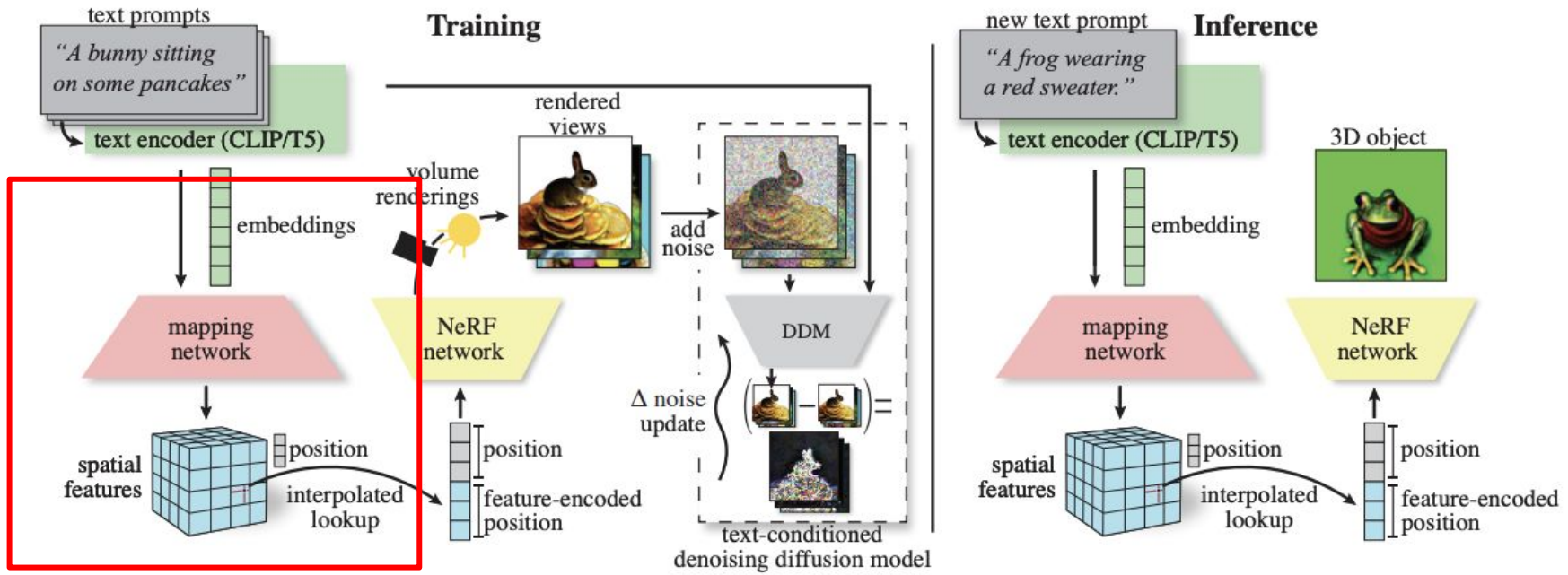
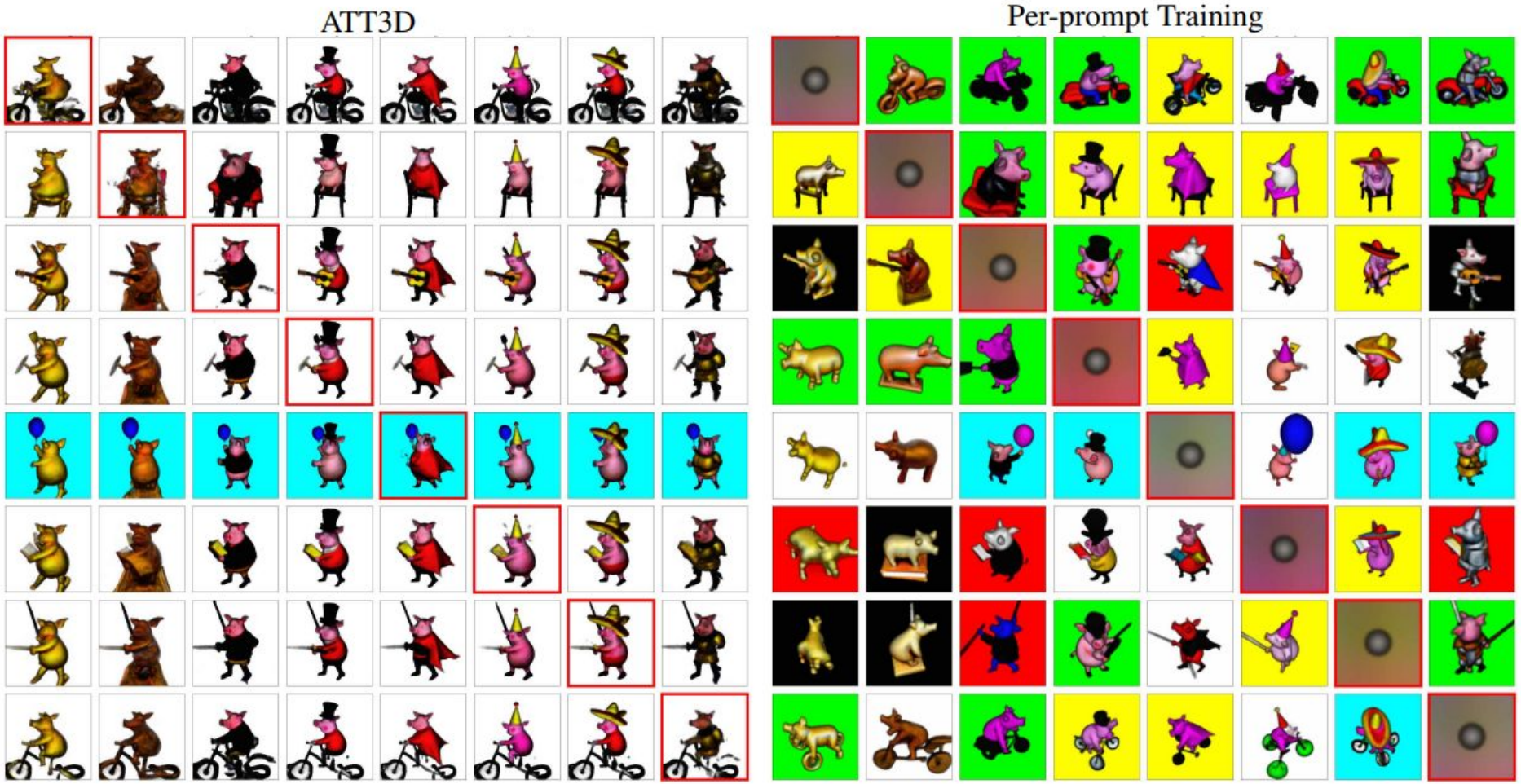


Figure 4: We show a schematic of our text-to-3D pipeline with changes from DreamFusion's pipeline [1] shown in red and pseudocode in Alg. 1. The text encoder (in green) provides its – potentially cached – text embedding c to the text-to-image DDM and now also to the mapping network m (in red). We use a spatial point-encoder $\gamma_{m(c)}$ (in blue) for our position x, whose parameters are modulations from the mapping network $m(c)$. The final NeRF MLP ν outputs a radiance r given the point encoding: $r = \nu(\gamma_{m(c)}(x))$, which we render into views. *Left:* At training time, the rendered views are input to the DDM to provide a training update. The NeRF network ν, mapping network m, and (effectively) the spatial point encoding $\gamma_{m(c)}$ are optimized. *Right*: At inference time, we use the pipeline up to the NeRF for representing the 3D object.

ATT3D · 2023 →

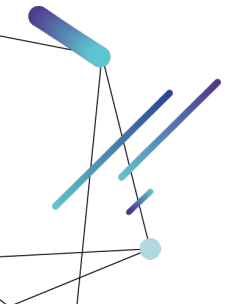
ATT3D: one model, many prompts



DLS

Problems: who solves what

Method	Prior	3D	Solves
DreamFusion	Imagen 64×64	NeRF	the base idea
Magic3D	eDiff-I + SD	NeRF → mesh	resolution, speed
Zero-1-to-3	SD + camera	NeRF → mesh	image-to-3D, Janus
ATT3D	64×64 text-to-image model	NeRF + mapping network	per-prompt optimization
MVDream	multi-view SD	NeRF	Janus
DreamGaussian	Zero-1-to-3 or SD	3DGS → mesh	speed

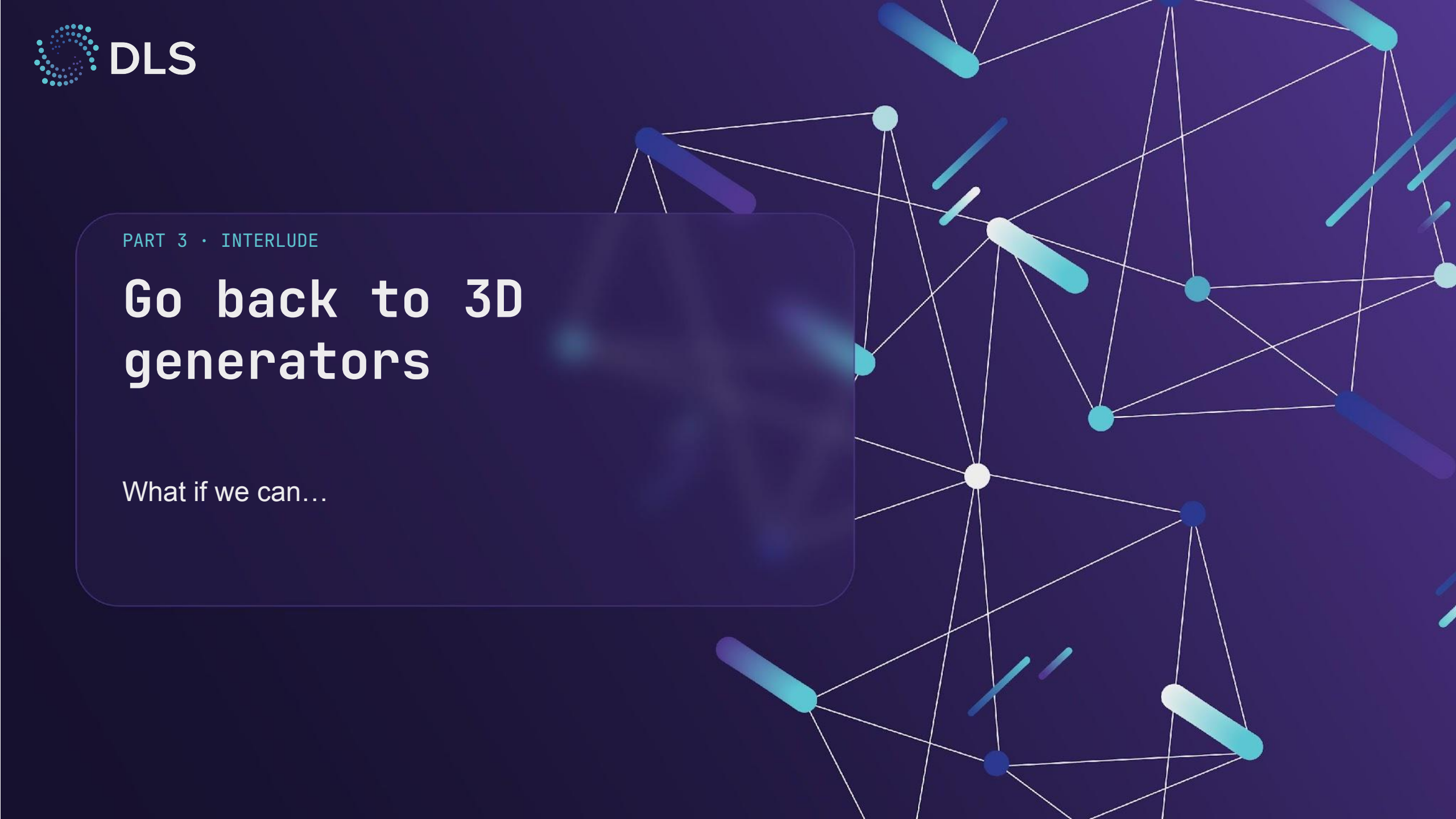


DLS

PART 3 · INTERLUDE

Go back to 3D generators

What if we can…

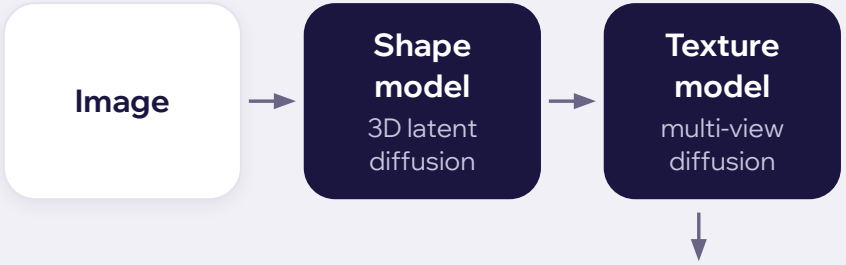


Hunyuan3D 2.0 · 2025 →

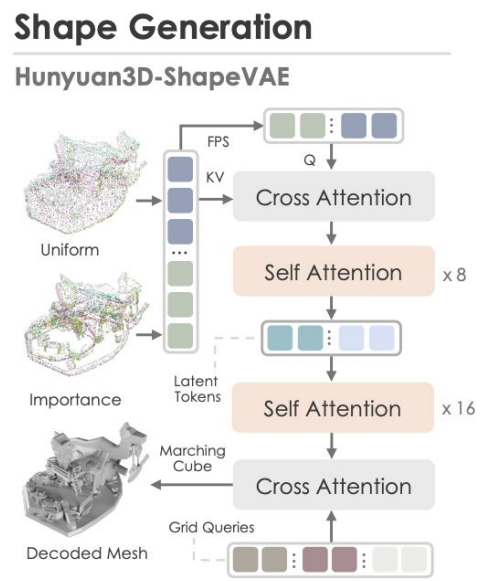
DLS

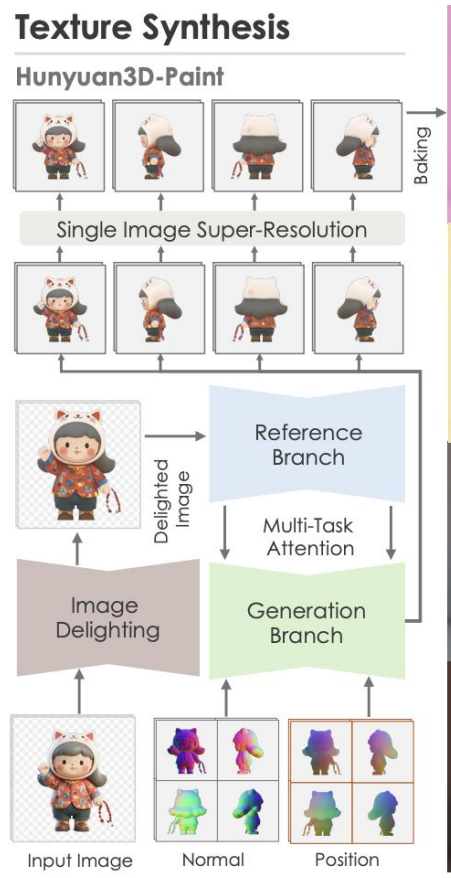
Hunyuan3D 2.0: motivation and idea

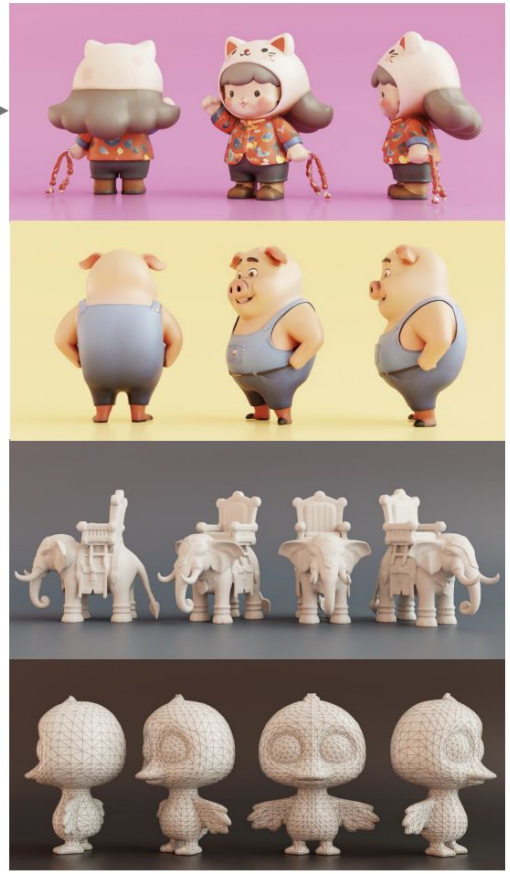
Decouple shape and texture



Geometry learned natively in 3D
Appearance borrowed from 2D diffusion







PART 1 RECIPE encode 3D into a latent and diffuse there, as in Shap-E — now with enough data

PART 2 LESSON where 3D data is scarce — appearance — borrow 2D diffusion priors

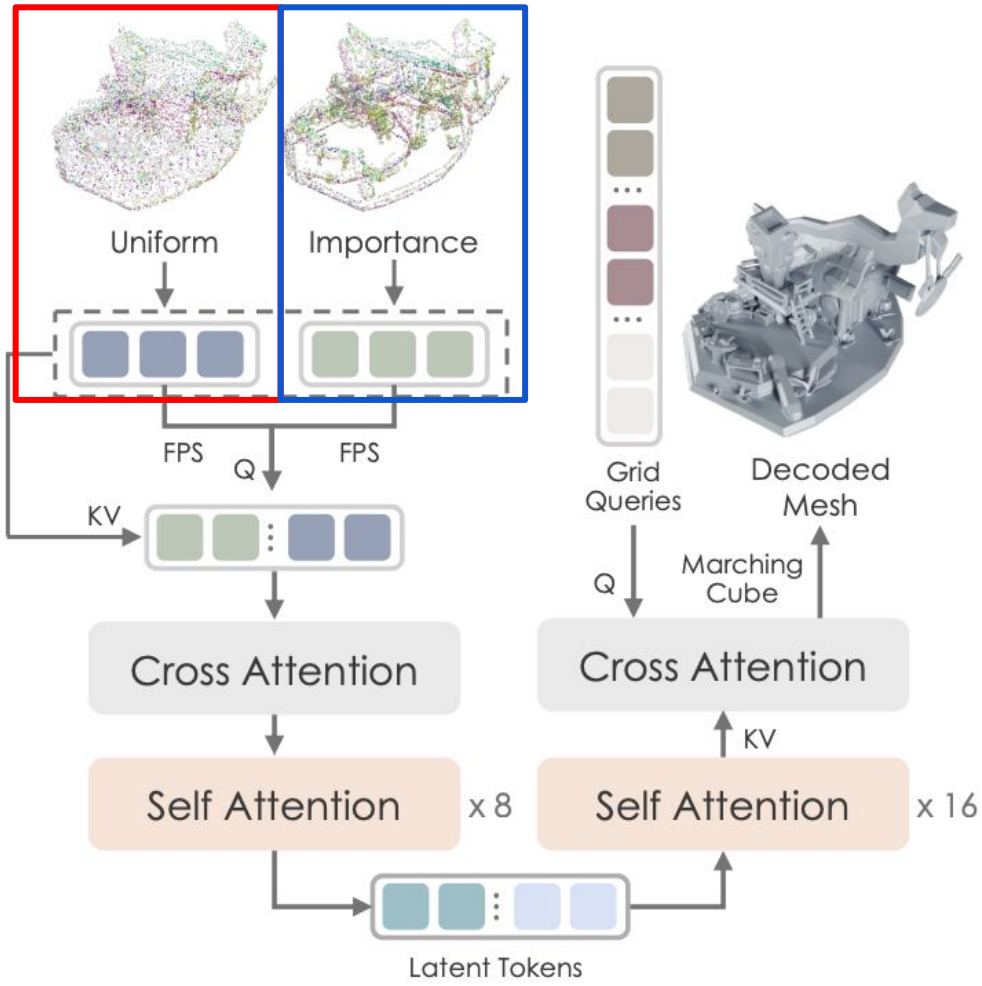
Hunyuan3D 2.0 · 2025 →

3DShape2VecSet · 2023 →

Hunyuan3D 2.0: ShapeVAE

1 Sample the surface

points + normals: uniform, plus extra samples on edges and corners



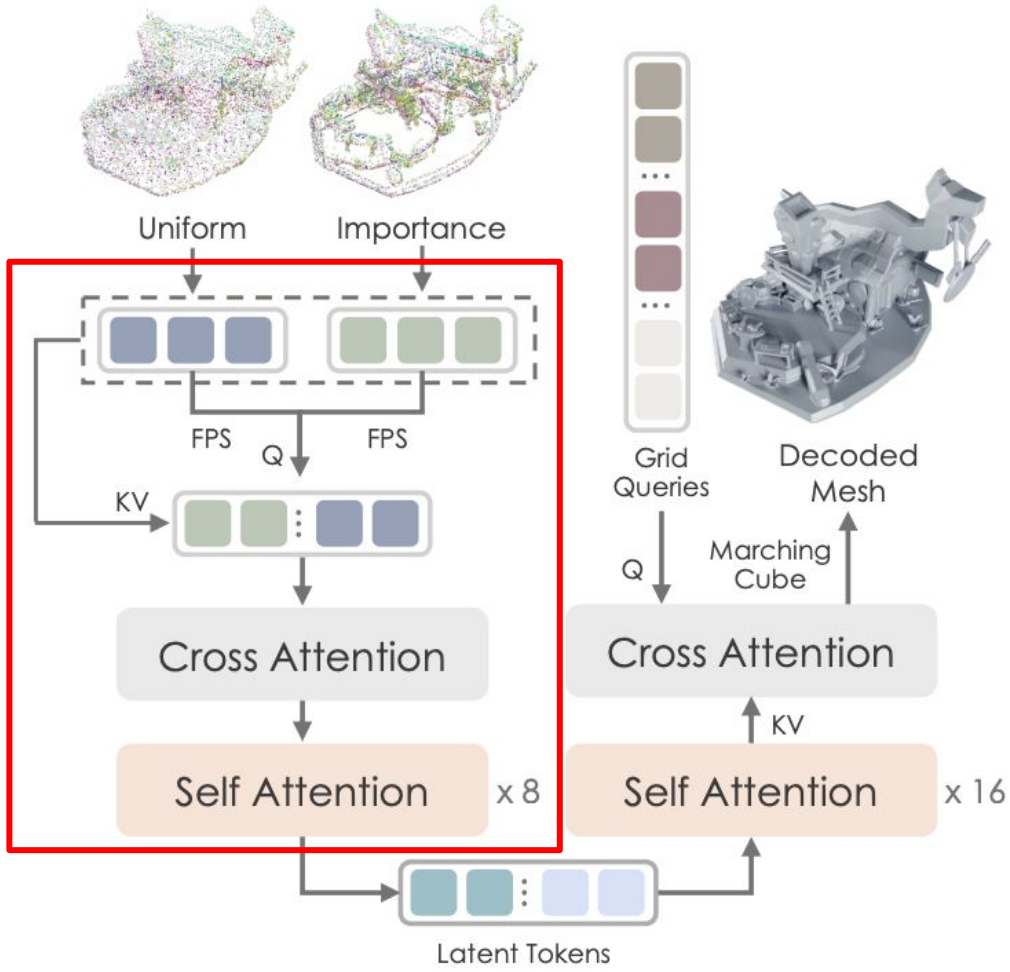
Hunyuan3D 2.0 · 2025 →

3DShape2VecSet · 2023 →

Hunyuan3D 2.0: ShapeVAE

1 Sample the surface
points + normals: uniform, plus extra samples on edges and corners

2 Points → tokens
FPS picks query points; cross-attention pulls the cloud in, then self-attention



Hunyuan3D 2.0 · 2025 →

3DShape2VecSet · 2023 →

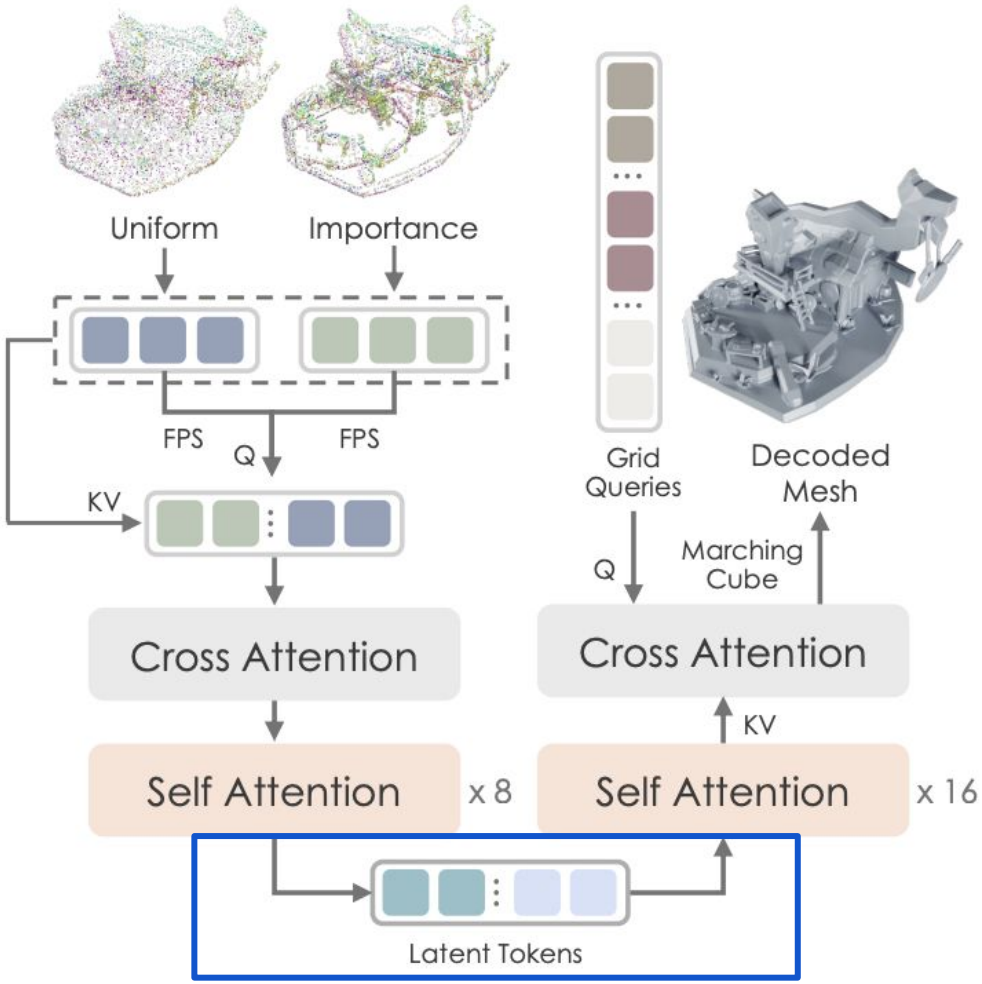
DLS

Hunyuan3D 2.0: ShapeVAE

1 **Sample the surface**
points + normals: uniform, plus extra samples on edges and corners

2 **Points → tokens**
FPS picks query points; cross-attention pulls the cloud in, then self-attention

3 **Variational latent**
mean and variance per token; up to 3072 tokens, length varies in training



Hunyuan3D 2.0 · 2025 →

3DShape2VecSet · 2023 →

DLS

Hunyuan3D 2.0: ShapeVAE

1 **Sample the surface**
points + normals: uniform, plus extra samples on edges and corners

2 **Points → tokens**
FPS picks query points; cross-attention pulls the cloud in, then self-attention

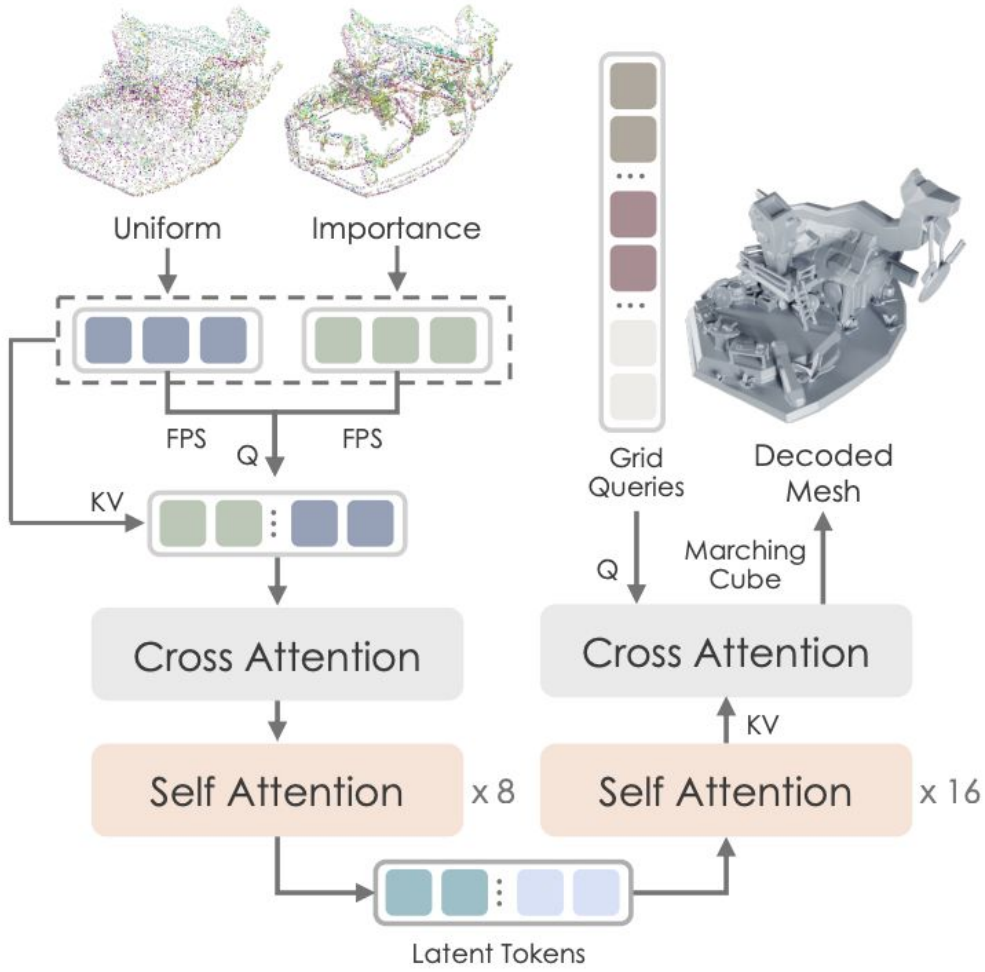
3 **Variational latent**
mean and variance per token; up to 3072 tokens, length varies in training

4 **Tokens → SDF → mesh**
a point perceiver queries a 3D grid for SDF values; Marching Cubes

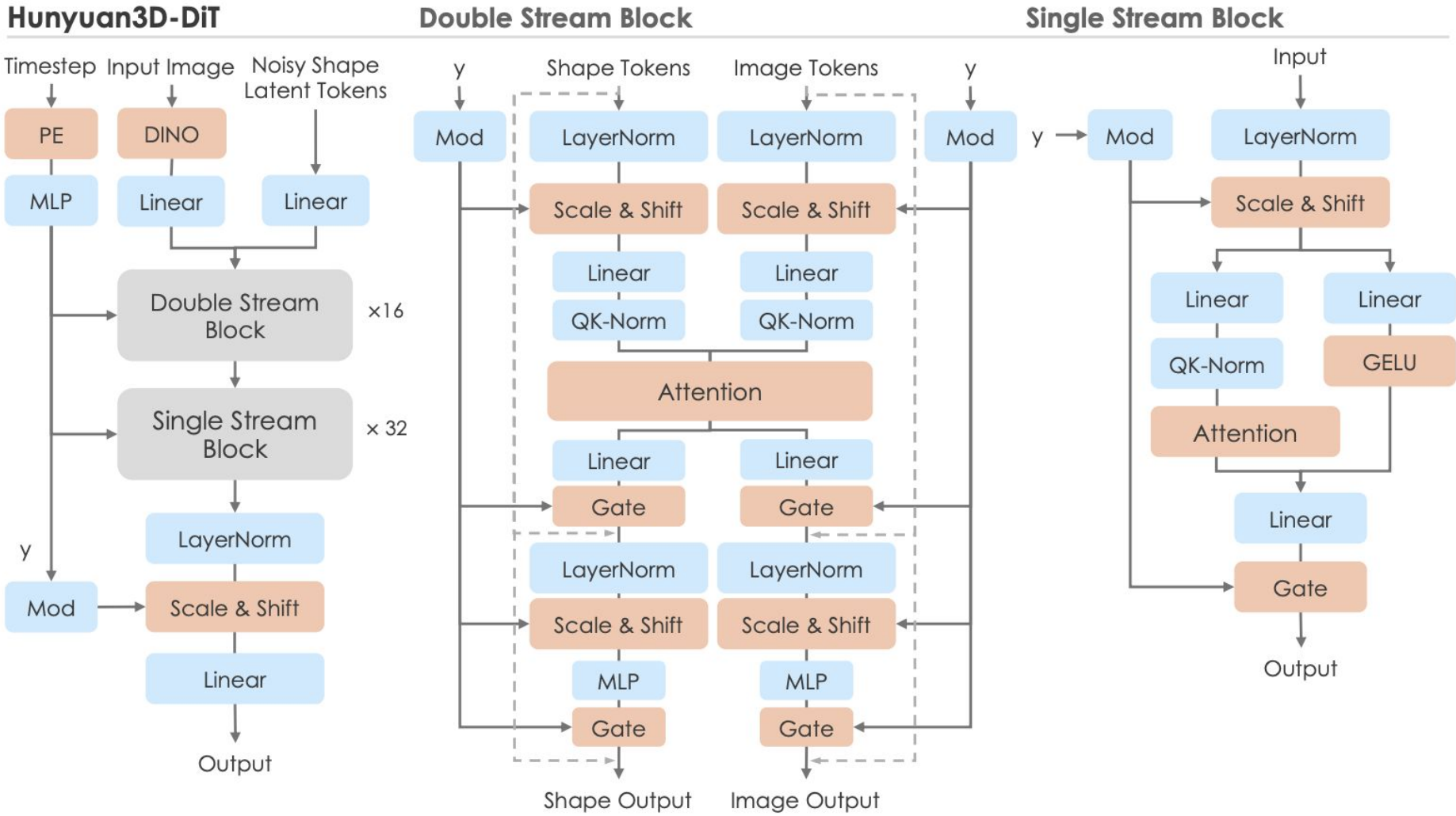
Reconstruction loss

$$\mathcal{L}_r = \mathbb{E}_{\mathbf{x}} \left\| \mathcal{D}_s(\mathbf{x} \mid Z_s) - \mathrm{SDF}(\mathbf{x}) \right\|^2 + \gamma \mathcal{L}_{\mathrm{KL}}$$

supervised at random points in space and near the surface

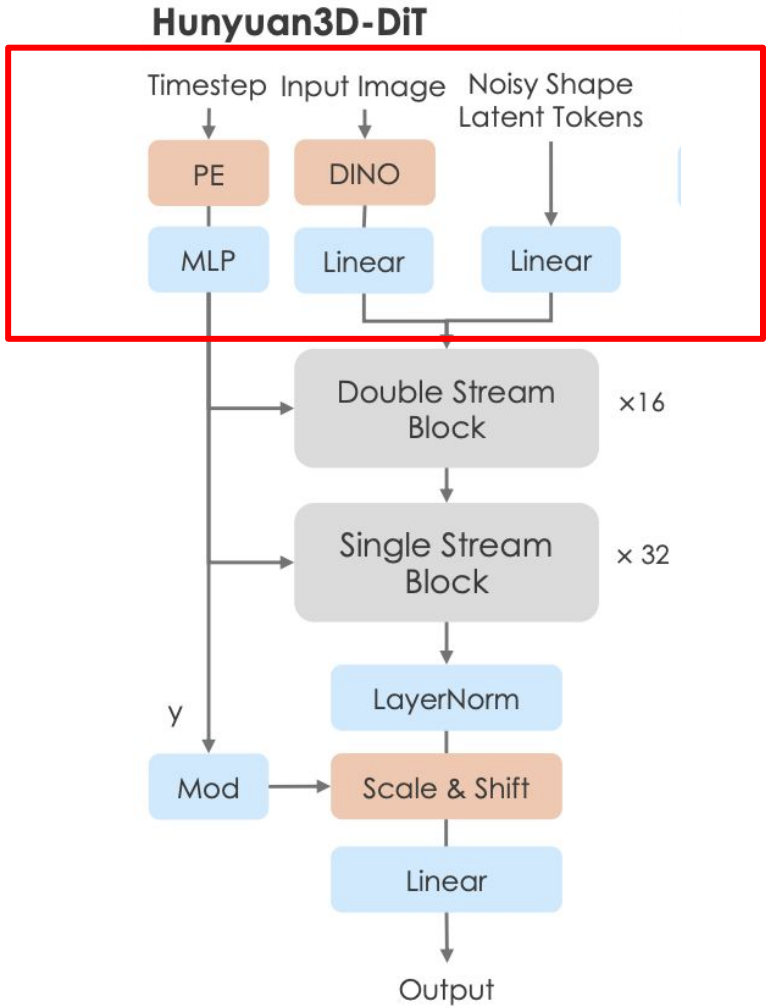


Hunyuan3D 2.0: DiT



Hunyuan3D 2.0: DiT

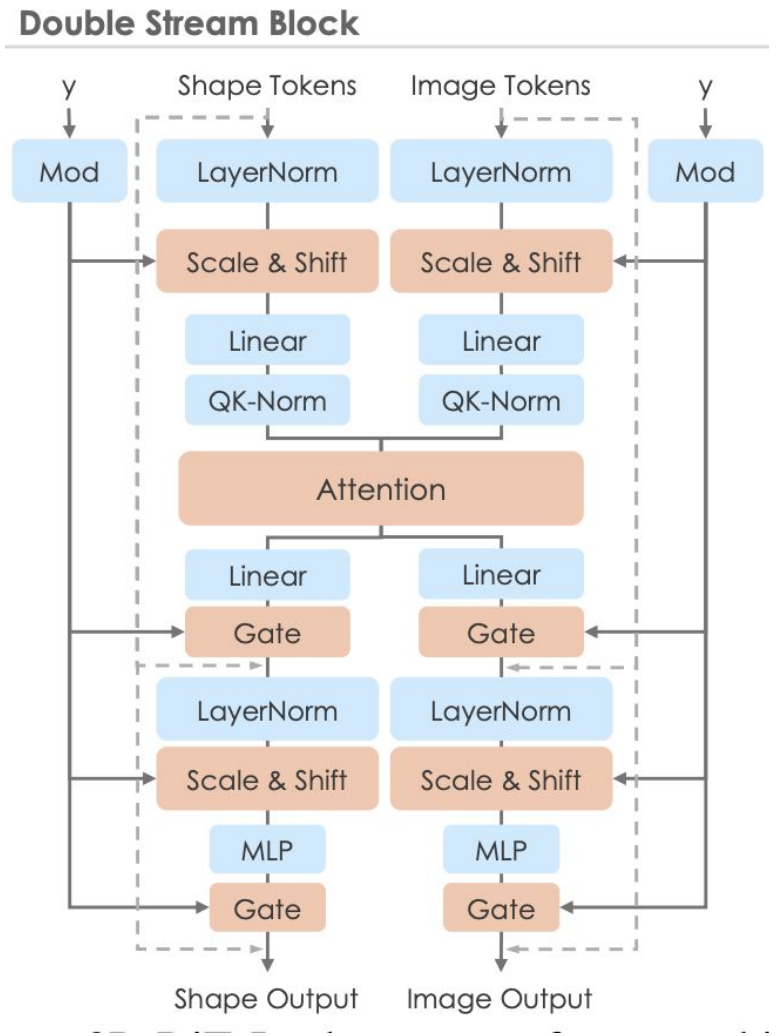
1 **Image condition**
DINOv2 Giant at 518 × 518 on a centered object over white



Hunyuan3D 2.0: DiT

1 **Image condition**
DINOv2 Giant at 518 × 518 on a centered object over white

2 **Double-stream blocks**
latent and image tokens keep their own QKV and MLP



Hunyuan3D 2.0: DiT

1 **Image condition**
DINOv2 Giant at 518 × 518 on a centered object over white

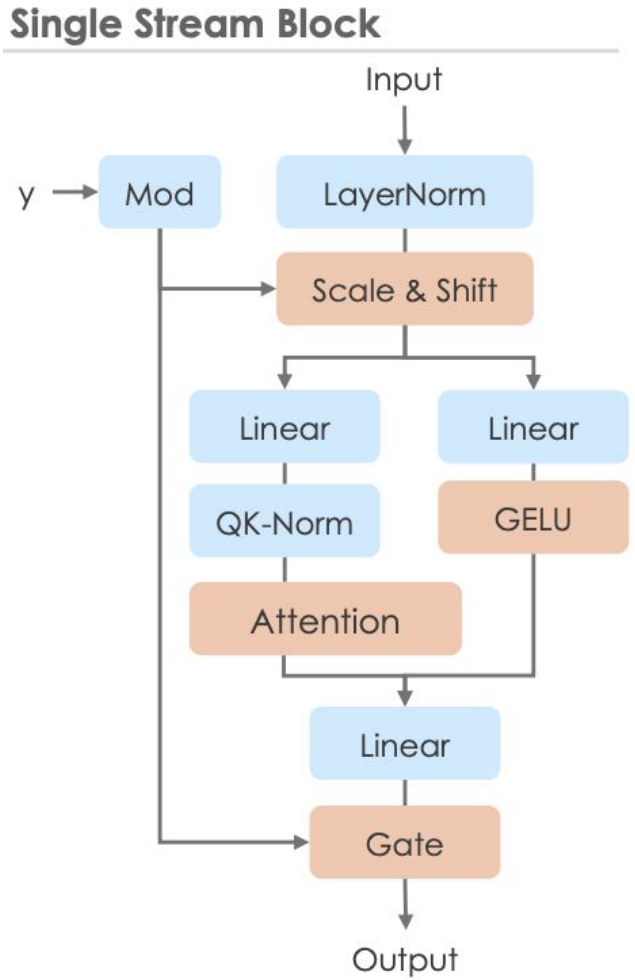
2 **Double-stream blocks**
latent and image tokens keep their own QKV and MLP

3 **Single-stream blocks**
tokens concatenated. Spatial and channel attention in parallel

4 **No positional embedding**
a token's content, not its index, decides where it lands in 3D

Flow matching

$$x_t = (1-t)\,x_0 + t\,x_1, \qquad \mathcal{L} = \mathbb{E}\left\|u_\theta(x_t, c, t) - (x_1 - x_0)\right\|^2$$



Hunyuan3D 2.0: DiT

1 **Image condition**
DINOv2 Giant at 518 × 518 on a centered object over white

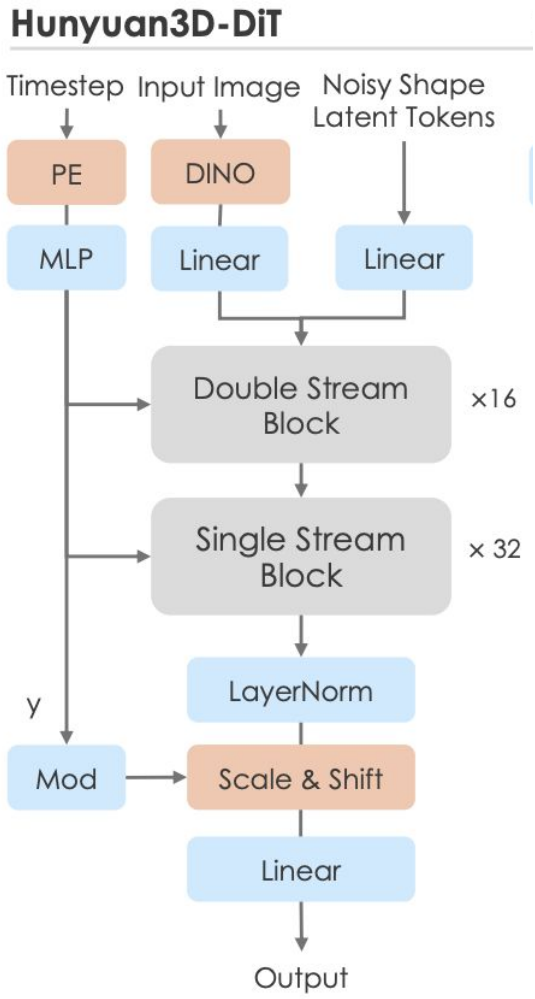
2 **Double-stream blocks**
latent and image tokens keep their own QKV and MLP

3 **Single-stream blocks**
tokens concatenated. Spatial and channel attention in parallel

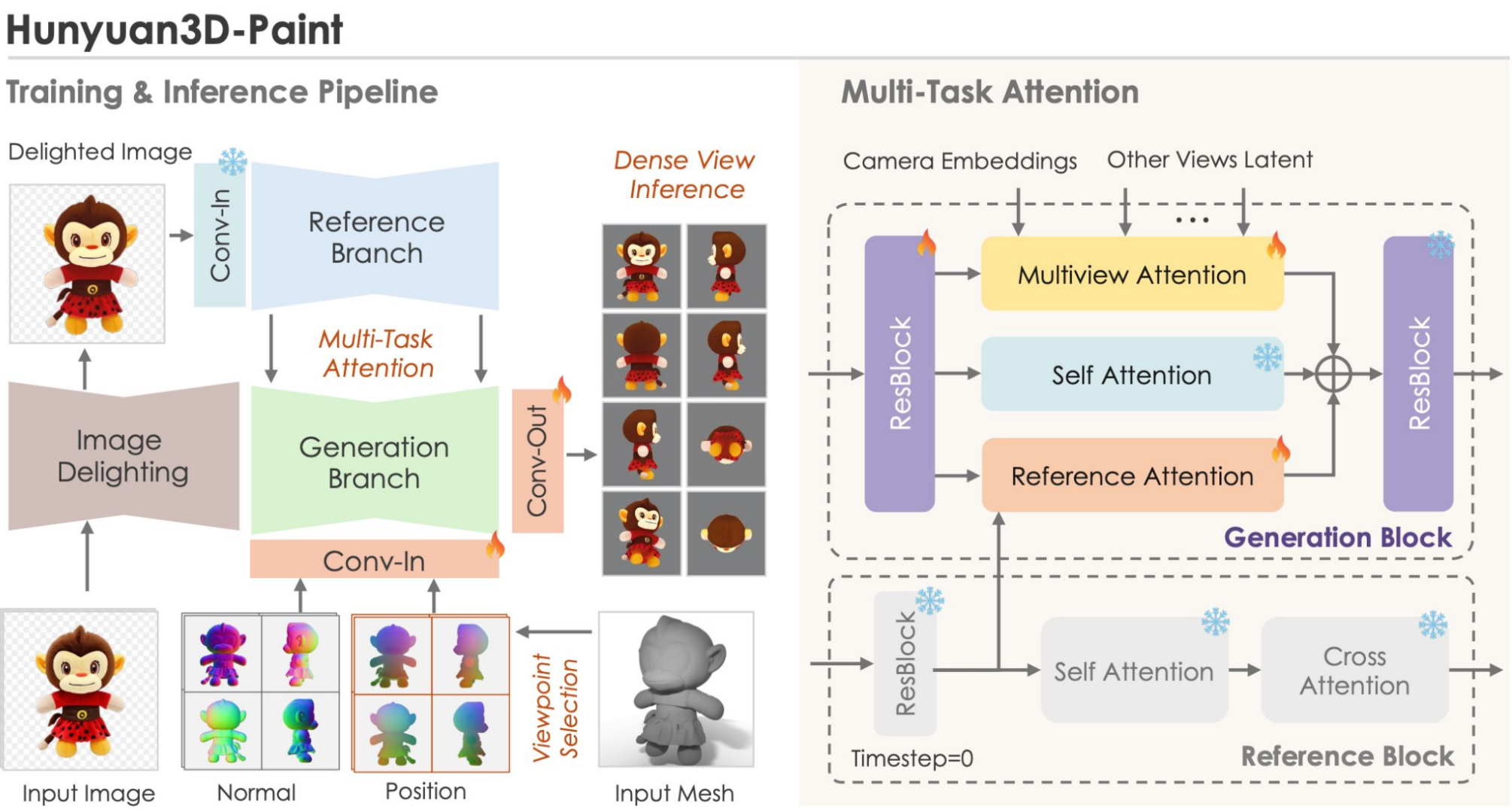
4 **No positional embedding**
a token's content, not its index, decides where it lands in 3D

Flow matching

$$x_t = (1-t)\,x_0 + t\,x_1, \qquad \mathcal{L} = \mathbb{E}\left\|u_\theta(x_t, c, t) - (x_1 - x_0)\right\|^2$$



Hunyuan3D 2.0: Paint



TRELLIS · 2025 →

GitHub →

DLS

TRELLIS: structured 3D latents

Active voxels

Cells of a 64 × 64 × 64 grid that the surface passes through: about 20K per object

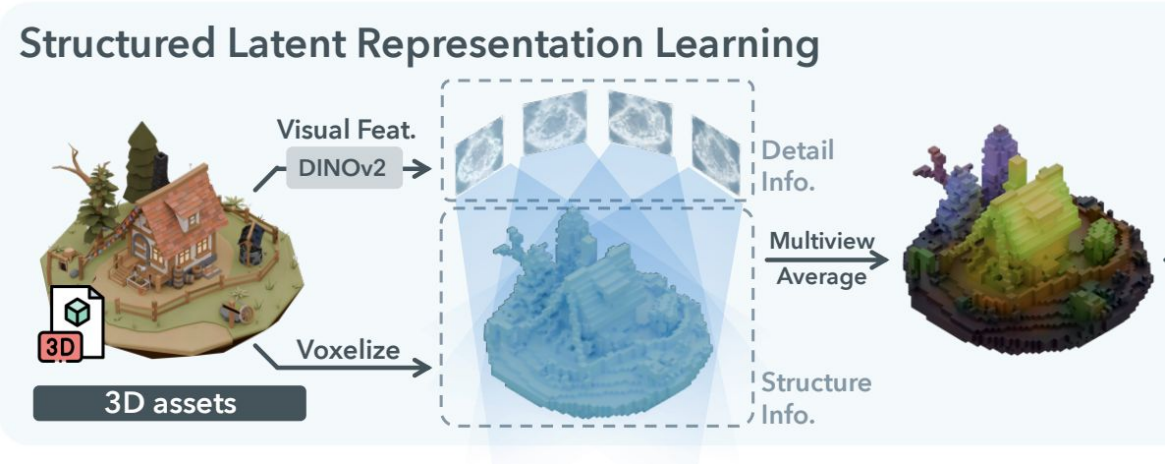
A latent on every voxel

8 numbers each, holding local geometry and appearance

Structured latent (SLAT)

$$\mathbf{z} = \{(\mathbf{z}_i,\ \mathbf{p}_i)\}_{i=1}^{L}, \qquad \mathbf{z}_i \in \mathbb{R}^8, \quad \mathbf{p}_i \in \{0, \dots, 63\}^3$$

position says where, the vector says what



TRELLIS · 2025 →

GitHub →

DLS

TRELLIS: structured 3D latents

Active voxels

Cells of a 64 × 64 × 64 grid that the surface passes through: about 20K per object

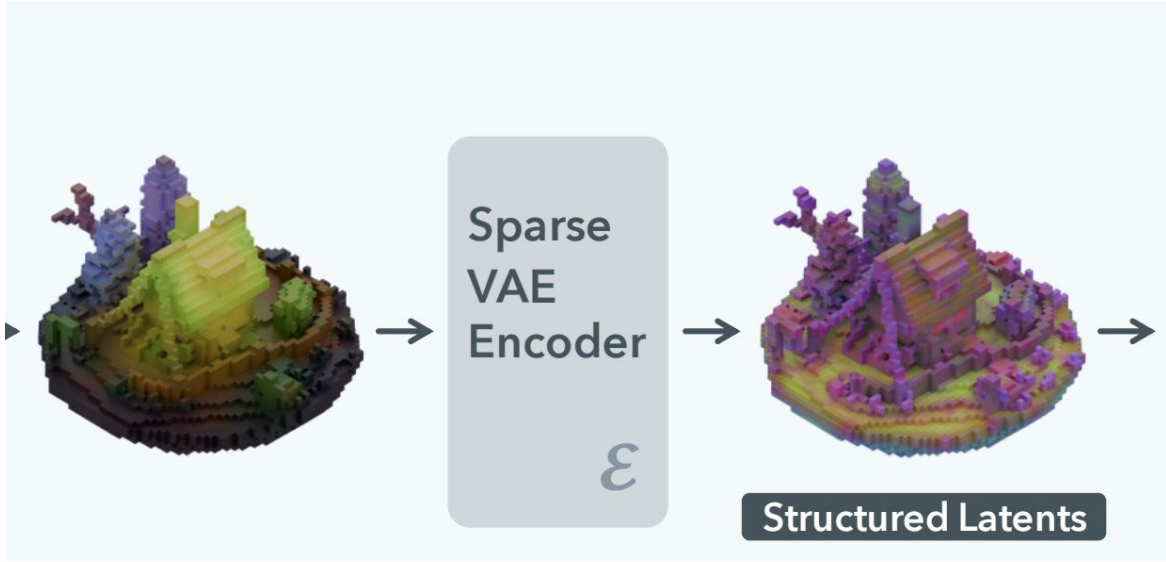
A latent on every voxel

8 numbers each, holding local geometry and appearance

Structured latent (SLAT)

$$\mathbf{z} = \{(\mathbf{z}_i,\ \mathbf{p}_i)\}_{i=1}^{L}, \qquad \mathbf{z}_i \in \mathbb{R}^8, \quad \mathbf{p}_i \in \{0, \dots, 63\}^3$$

position says where, the vector says what



TRELLIS · 2025 →

GitHub →

DLS

TRELLIS: structured 3D latents

Active voxels

Cells of a 64 × 64 × 64 grid that the surface passes through: about 20K per object

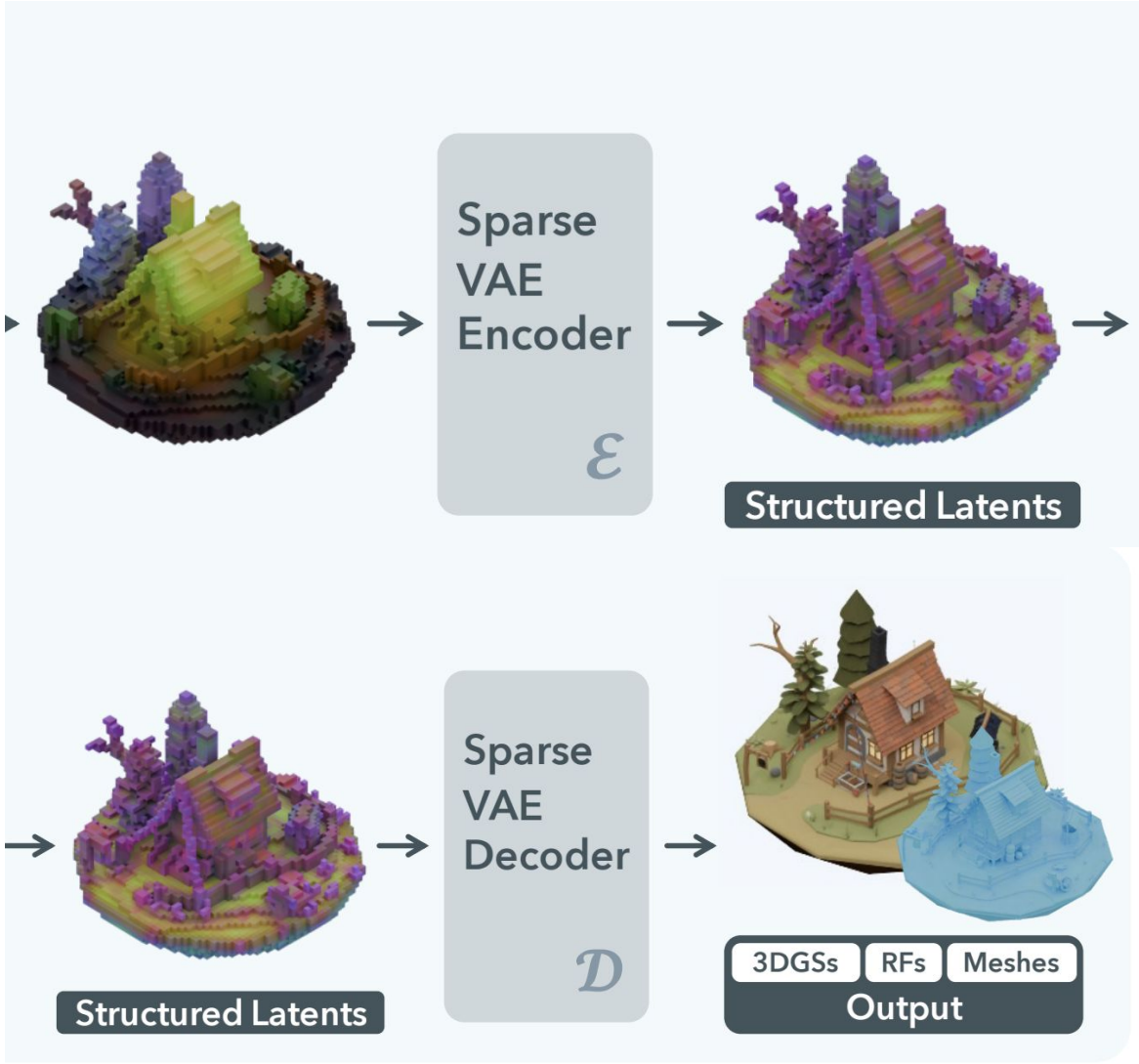
A latent on every voxel

8 numbers each, holding local geometry and appearance

Structured latent (SLAT)

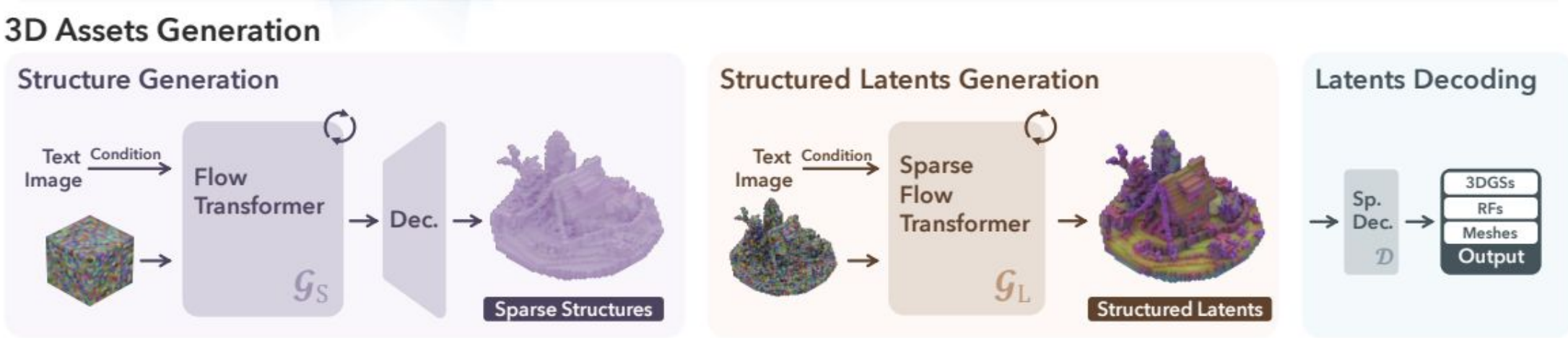
$$\mathbf{z} = \{(\mathbf{z}_i,\ \mathbf{p}_i)\}_{i=1}^{L}, \qquad \mathbf{z}_i \in \mathbb{R}^8, \quad \mathbf{p}_i \in \{0, \dots, 63\}^3$$

position says where, the vector says what



TRELLIS · 2025 → GitHub →

TRELLIS: generation in two stages



STAGE 1 · WHERE

Sparse structure

The 64^3 occupancy grid is compressed by a small 3D-conv VAE to $16^3 \times 8$. A flow transformer generates this grid; decoding it gives the active voxels

STAGE 2 · WHAT

Structured latents

A sparse flow transformer generates 8 numbers on every active voxel. To save tokens it packs 2 × 2 × 2 voxels (64 → 32) and unpacks at the end, with skip connections

DECODE

Any format

The VAE decoders: Gaussians, radiance field or mesh

Hunyuan3D 2.0 · 2025 →

TRELLIS · 2025 →

DLS

Hunyuan3D 2.0 vs. TRELLIS

	Hunyuan3D 2.0	TRELLIS
Representation	Vector set: up to 3072 tokens	SLAT: latents on active voxels of a 64 × 64 × 64 grid
Token position	Implicit, in the content; no positional embedding	Explicit voxel coordinates
What the VAE encodes	Geometry; input: surface points and normals	Geometry + appearance, from DINOv2 features
Color	Separate model: Hunyuan3D-Paint	In the same latent
Generation stages	Shape DiT → Paint for texture	Structure → latents
Output	SDF → mesh	Gaussians, radiance field or mesh

DLS

PART 4 · INTERLUDE

Video and Multi-view diffusion for 3D



V3D · 2024 →

Stable Video Diffusion · 2023 →

DLS

V3D: a video model as a view generator

More views than image models

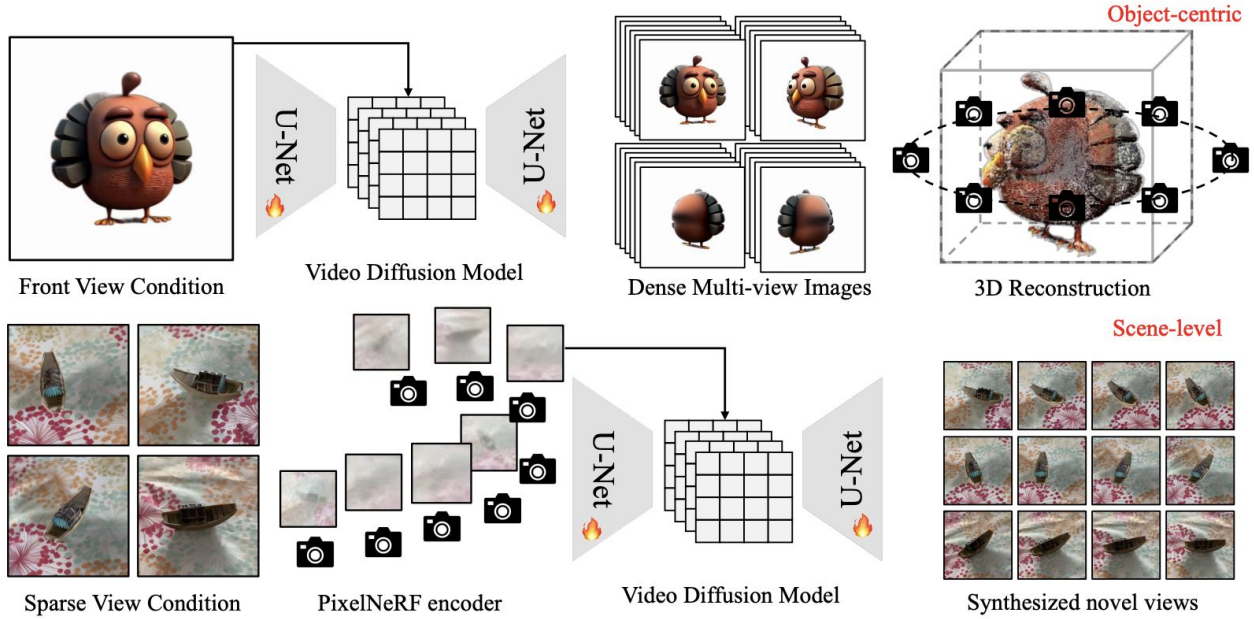
Image-based multi-view diffusion gives fewer than 6 views; V3D generates an 18-frame orbit at 512 × 512

Recipe

Fine-tune Stable Video Diffusion on 360° orbit renders of about 290K Objaverse objects: one image in, an orbit out

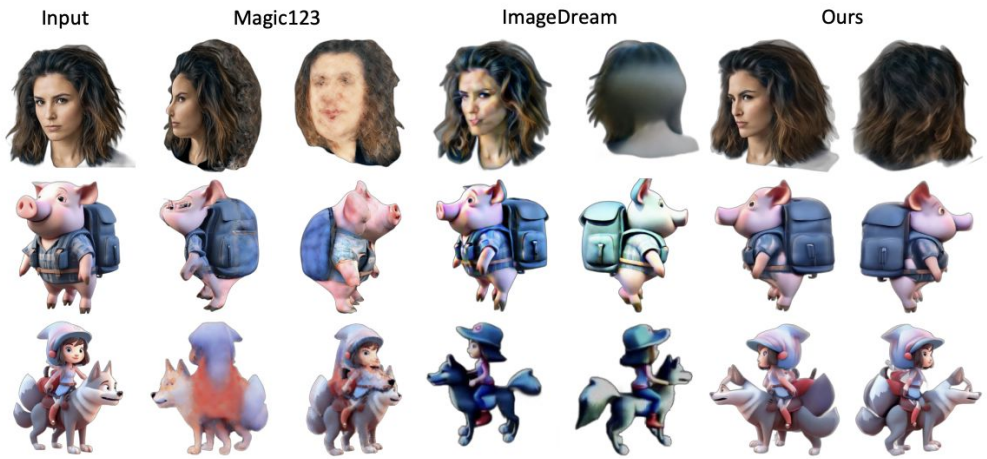
Don’t learn 3D from scratch

Take a pretrained video model and tune it for geometric consistency



V3D: reconstruction and results







Generated Multi-views

CAT3D · 2024 →

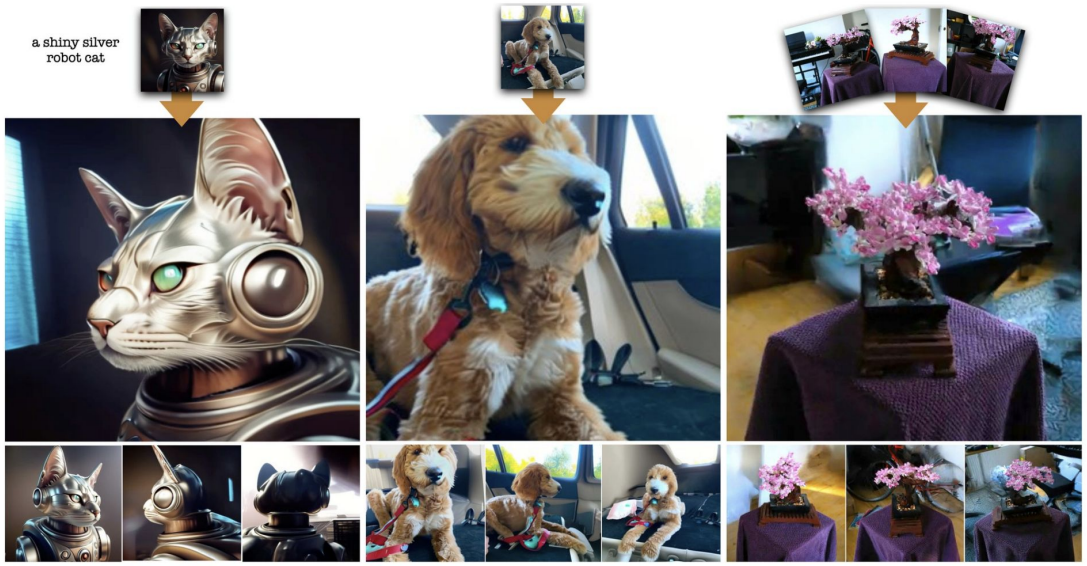
CAT3D: simulate a capture

Input

Any number of photos, from one, plus the cameras where new views are needed

Model

Latent diffusion from a text-to-image model. Trained on 8 views: 1 or 3 given, the rest generated



CAT3D · 2024 →

CAT3D: simulate a capture

Input

Any number of photos, from one, plus the cameras where new views are needed

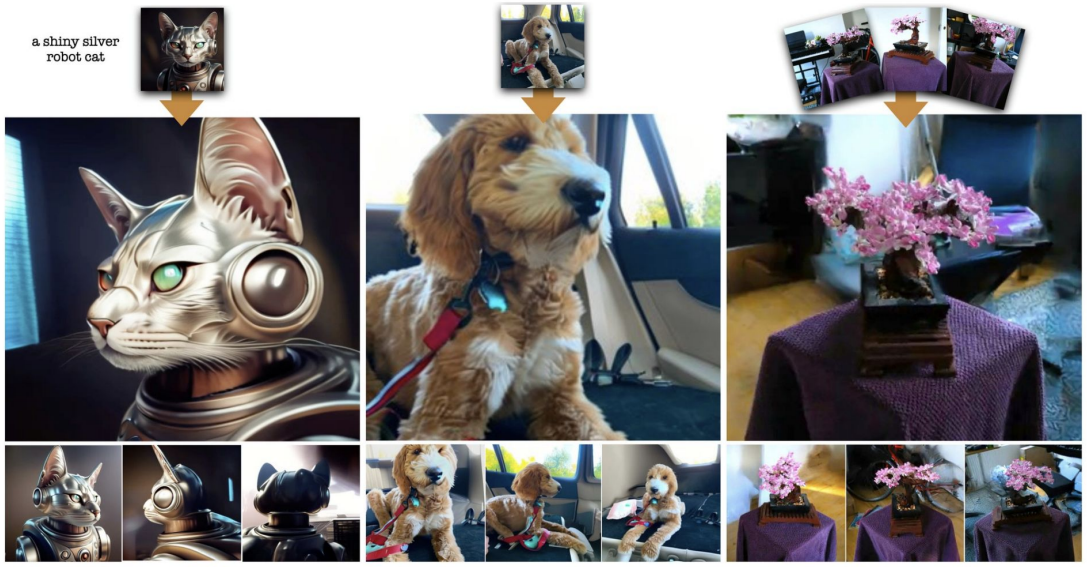
Model

Latent diffusion from a text-to-image model. Trained on 8 views: 1 or 3 given, the rest generated

Cameras as a raymap

$$\text{raymap}(u, v) = \left(\mathbf{o}_{uv},\ \mathbf{d}_{uv}\right)$$

For every pixel: ray origin and direction, concatenated to the image latents



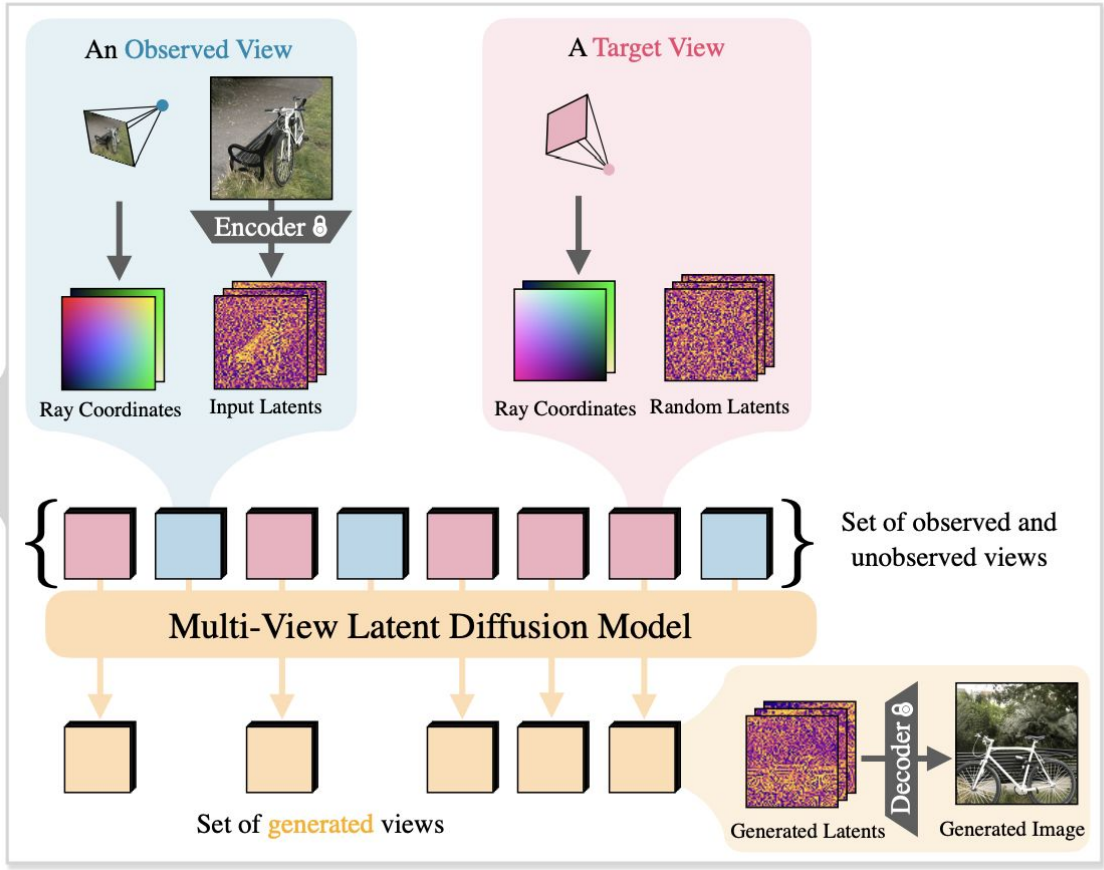
CAT3D: simulate a capture

Pipeline

Input + cameras → anchor views → the rest in groups → Zip-NeRF → 3D scene

Two-step sampling

From one photo: **first 7 anchor views that cover the scene**



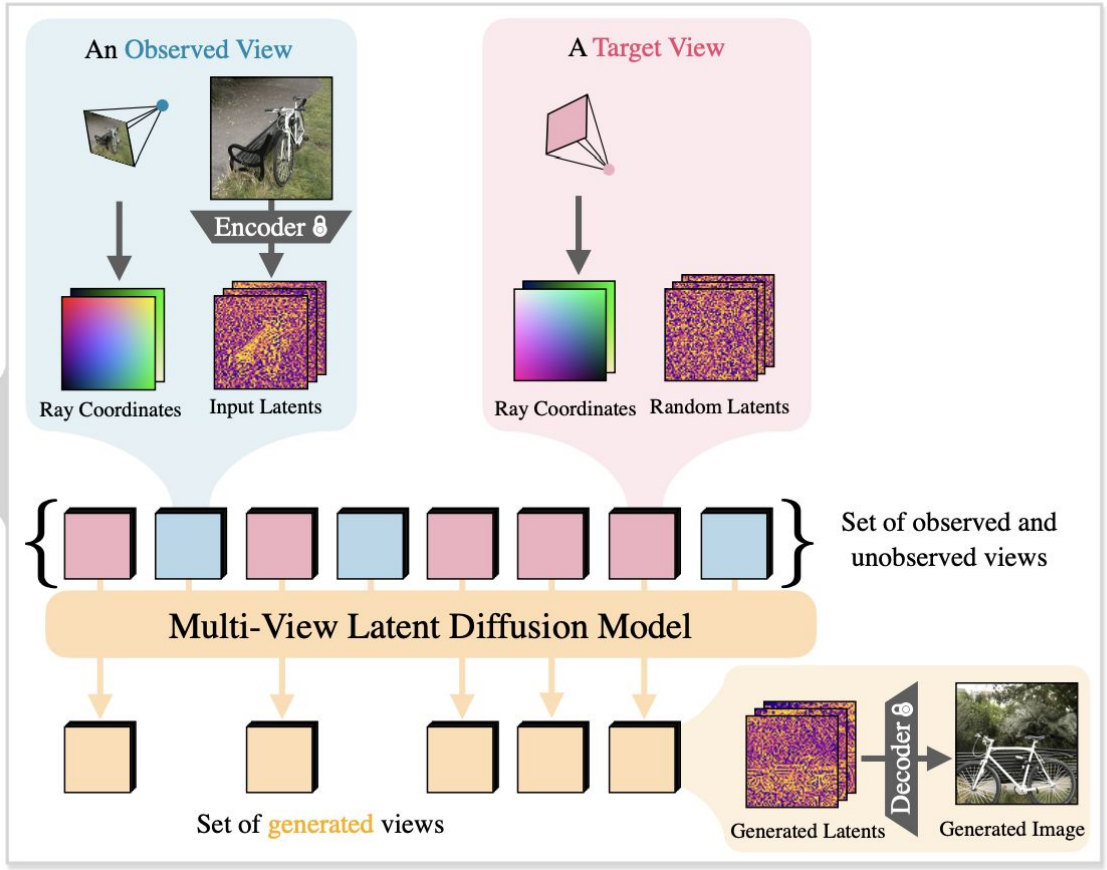
CAT3D: simulate a capture

Pipeline

Input + cameras → anchor views → the rest in groups → Zip-NeRF → 3D scene

Two-step sampling

From one photo: first 7 anchor views that cover the scene, then **the rest in parallel, conditioned on the input and the anchors — 80 views.** From a few photos: 480–960 views



CAT3D · 2024 →

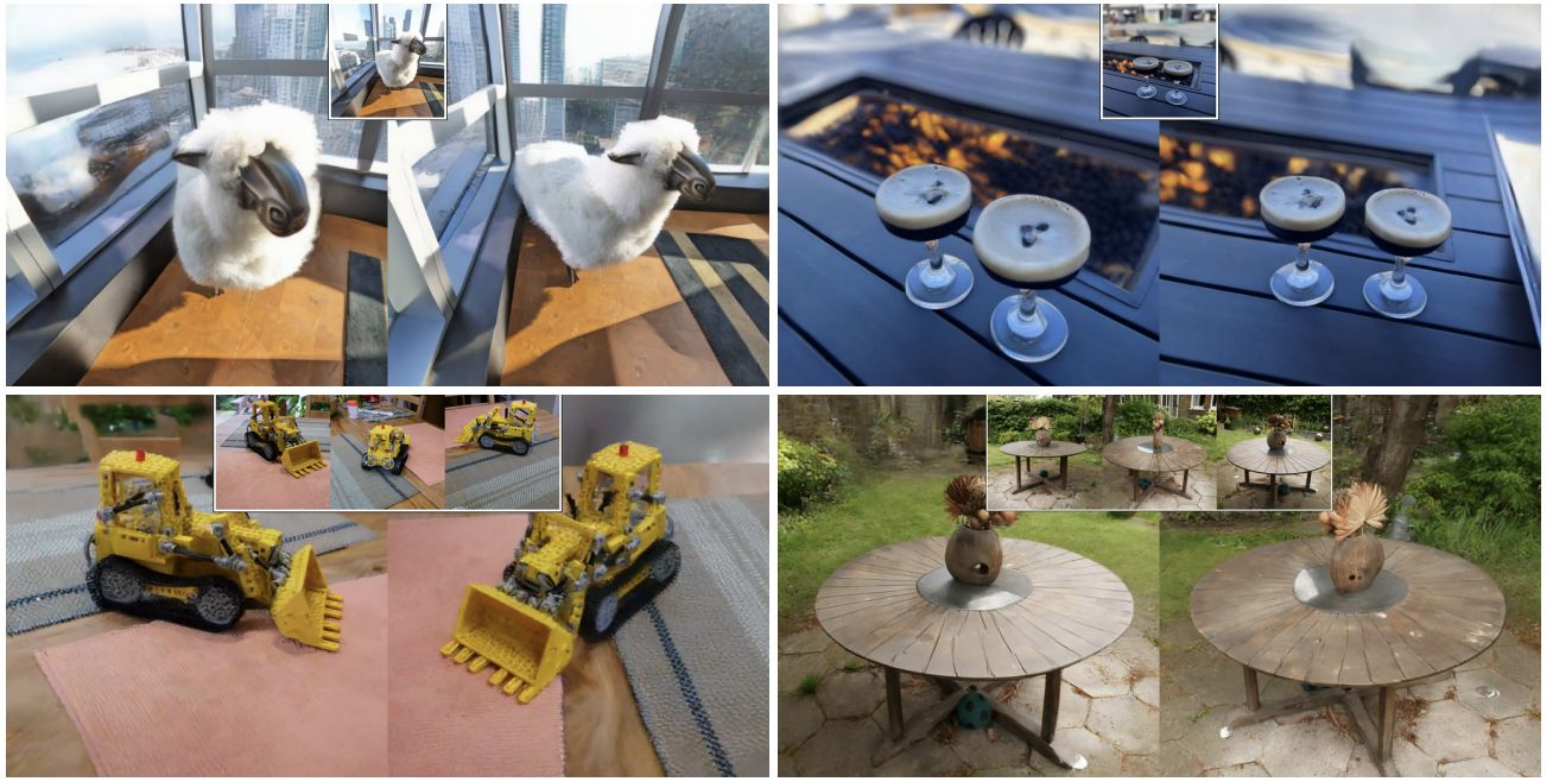
Zip-NeRF · 2023 →

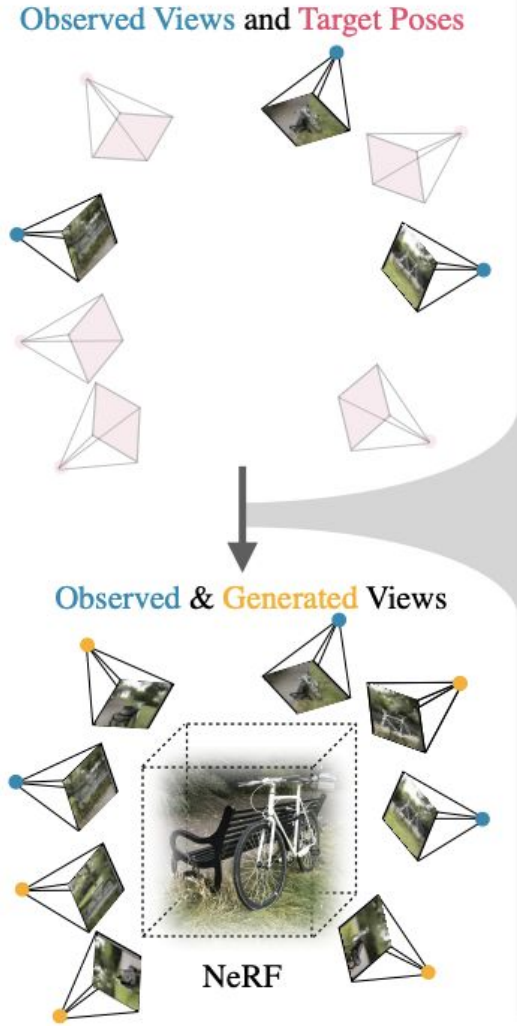
CAT3D: simulate a capture

Robust reconstruction

Zip-NeRF + LPIPS, as in V3D.







DLS

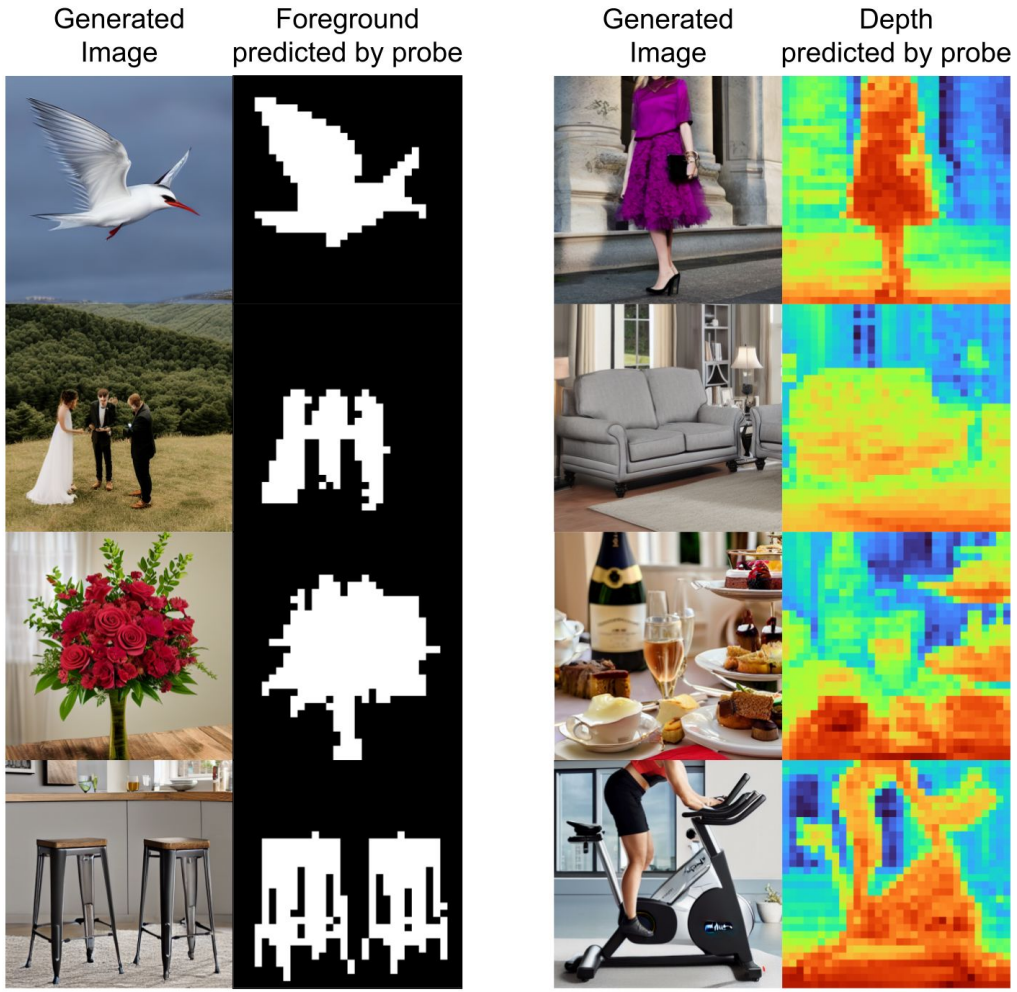
PART 5· INTERLUDE

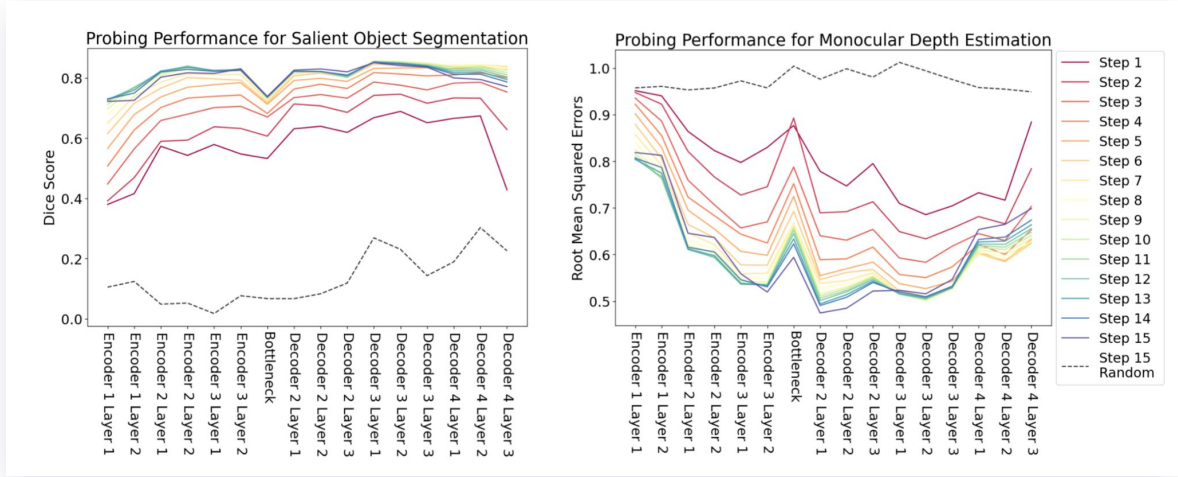
3D inside 2D diffusion

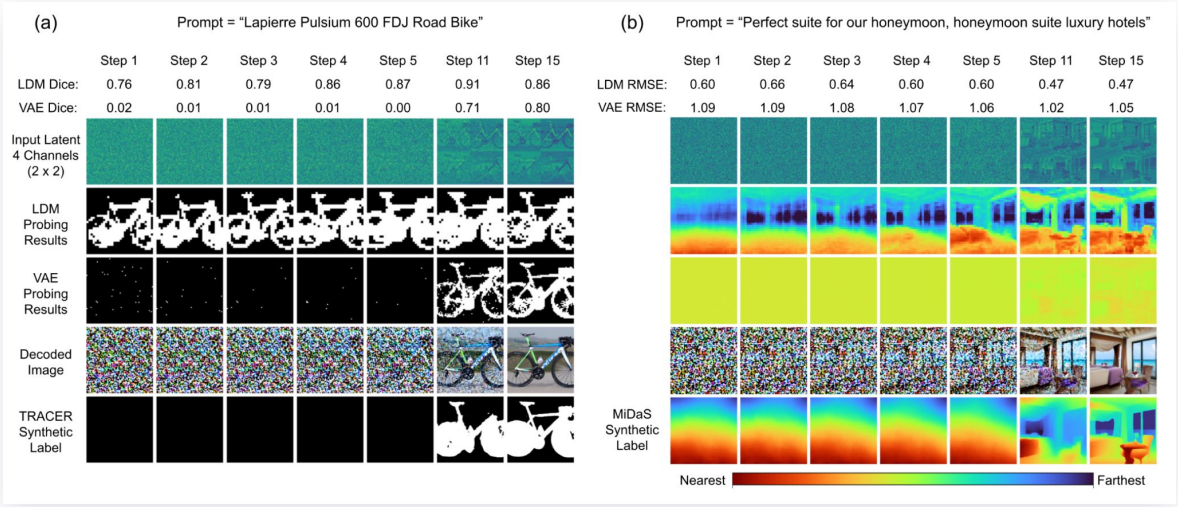
The model has only seen images. Where does its geometry come from — and where does it fall short?

paper →

Beyond Surface Statistics







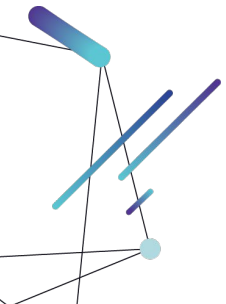
Marigold · 2023 →

DLS

Marigold: image generator → depth estimator

Question

Stable Diffusion has seen billions of images and know some geometry inside. Can we use it as starting point for some geometry prediction, for example depth?



Marigold · 2023 →

DLS

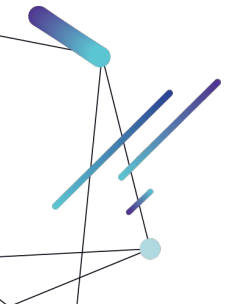
Marigold: image generator → depth estimator

Question

Stable Diffusion has seen billions of images and know some geometry inside. Can we use it as starting point for some geometry prediction, for example depth?

Answer: yes, and cheaply

74K synthetic samples, not a single real depth map; about 2.5 days on one RTX 4090



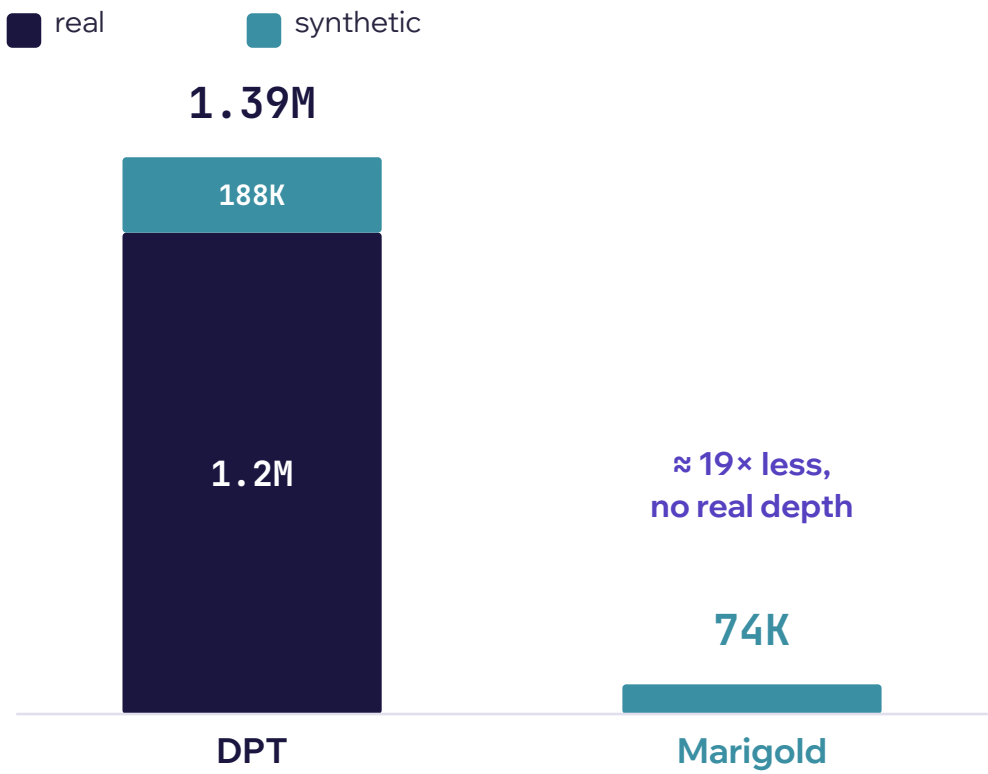
Marigold · 2023 →

DLS

Marigold: image generator → depth estimator

	TRAINING DATA	NYUv2 AbsRel ↓
DPT	1.2M real + 188K synthetic	9.8
Marigold	74K, synthetic only	5.5

TRAINING SAMPLES WITH DEPTH



Marigold also inherits Stable Diffusion's image pretraining (no depth)

Marigold · 2023 → Stable Diffusion · 2021 →

Marigold: image generator → depth estimator

Pipeline

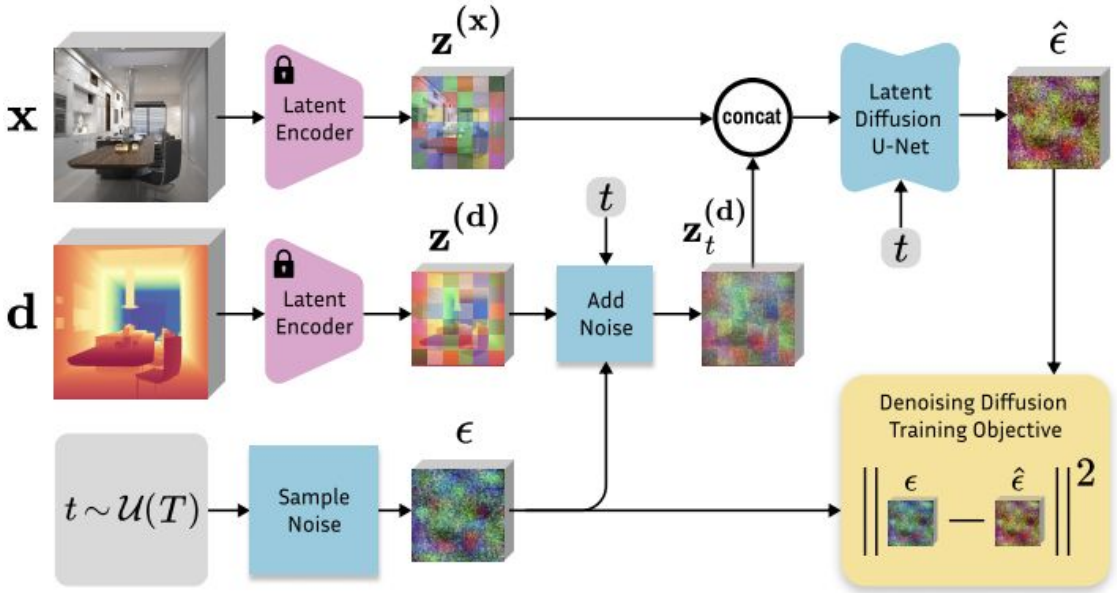
Image and noisy depth → VAE → two latents → U-Net (SD v2) → depth latent → VAE decoder → depth

Image condition by concatenation

Image and depth latents are stacked at the U-Net input; the input layer is doubled, weights copied and halved; no text

Affine-invariant

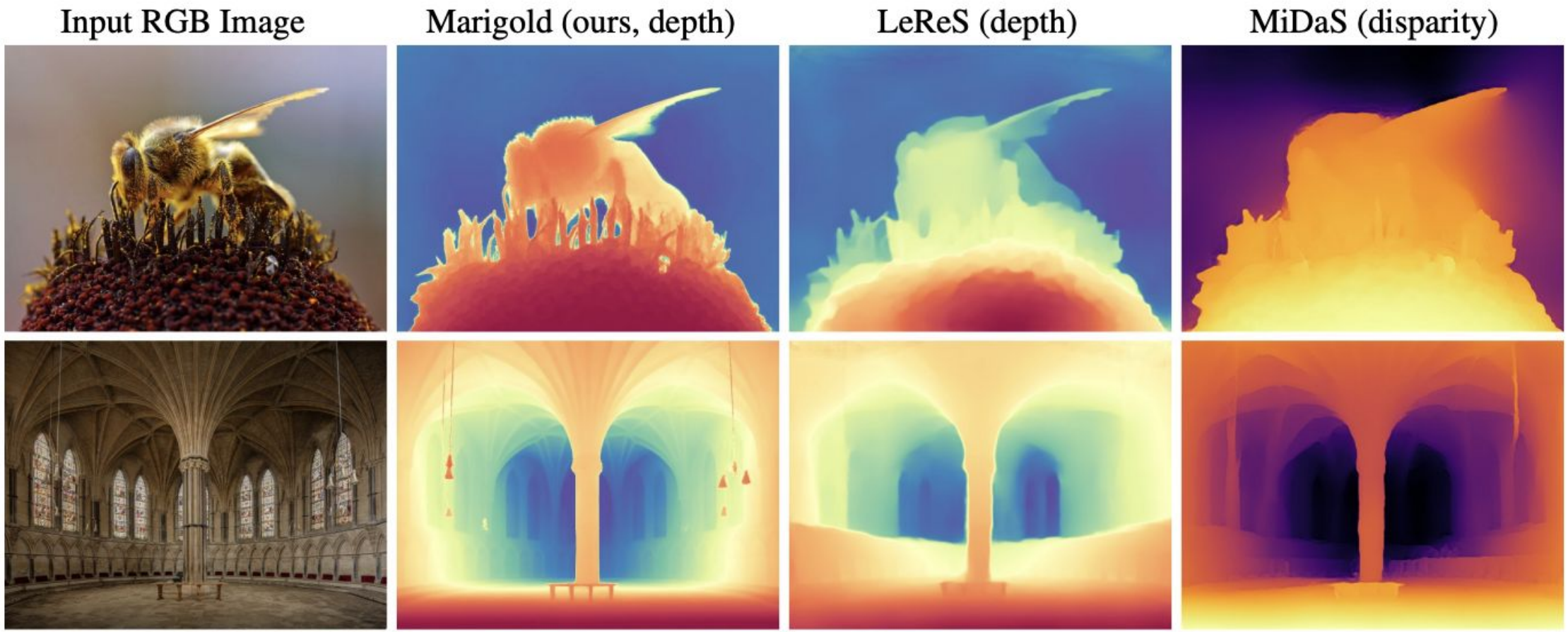
Relative depth: correct up to scale and shift



Marigold · 2023 →

DLS

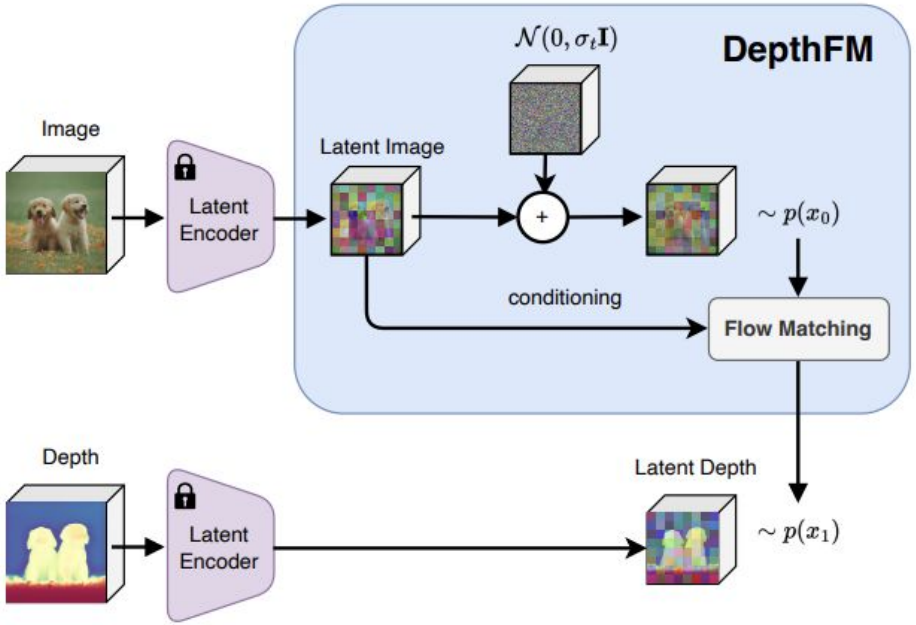
Marigold: image generator → depth estimator



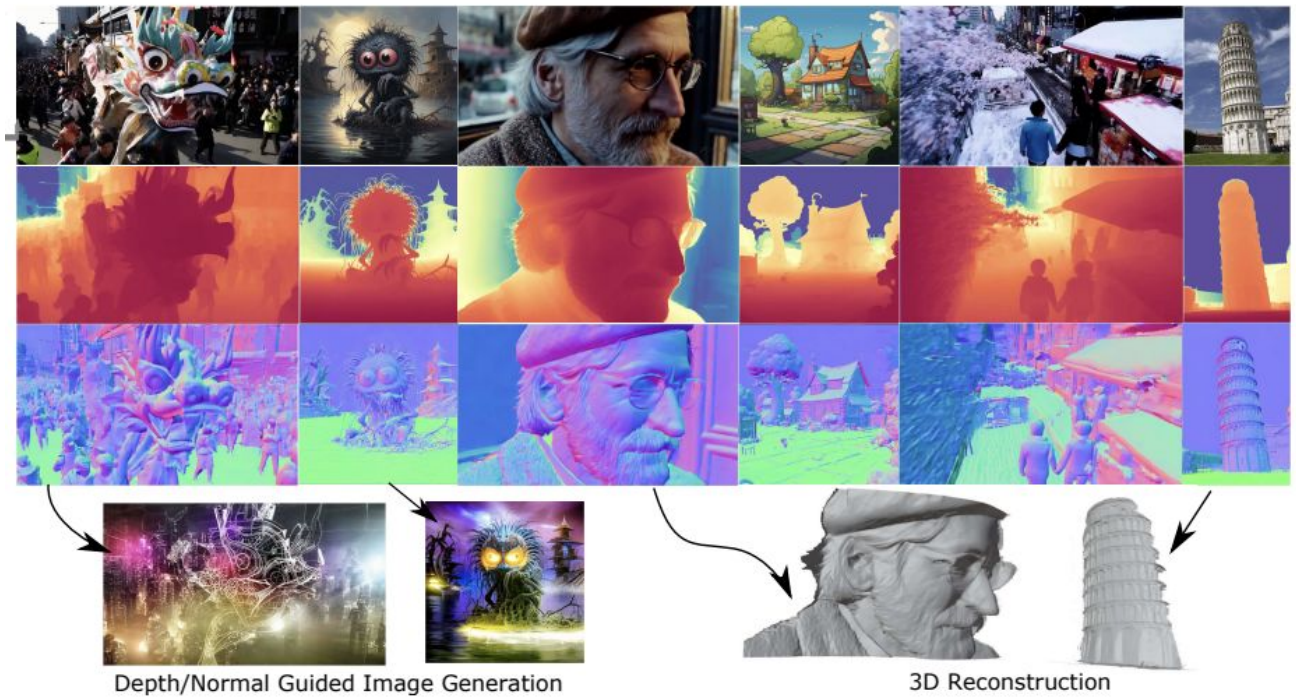
[DepthFM · 2024 →](#) [GeoWizard · 2024 →](#)

DLS

DepthFM and GeoWizard







Depth Anything · 2024 →

Depth Anything V2 · 2024 →

DLS

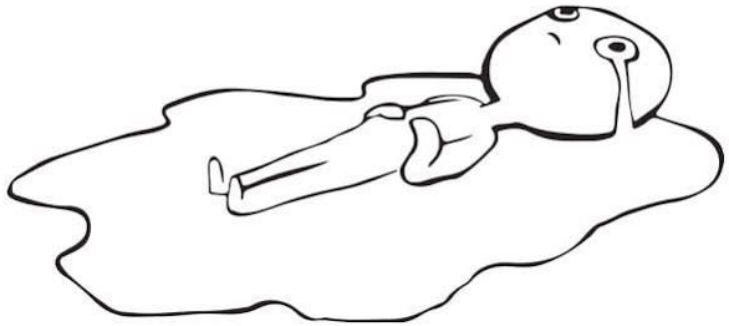
Then Depth Anything: data strikes back

	TRAINING DATA	NYUv2 AbsRel ↓	DA-2K acc. ↑
DPT	1.2M real + 188K synthetic	9.8	–
Marigold	74K, synthetic only	5.5	86.8%
Depth Anything	1.5M labeled + 62M pseudo-labeled real	4.3	88.5%
Depth Anything V2	595K synthetic + 62M pseudo-labeled real	4.5	97.1%

IMAGES USED FOR TRAINING · LINEAR SCALE



Data beats the idea



Geo4D · 2025 → DynamiCrafter · 2023 → DUSt3R · 2023 →

DLS

Geo4D: video generator → 4D geometry

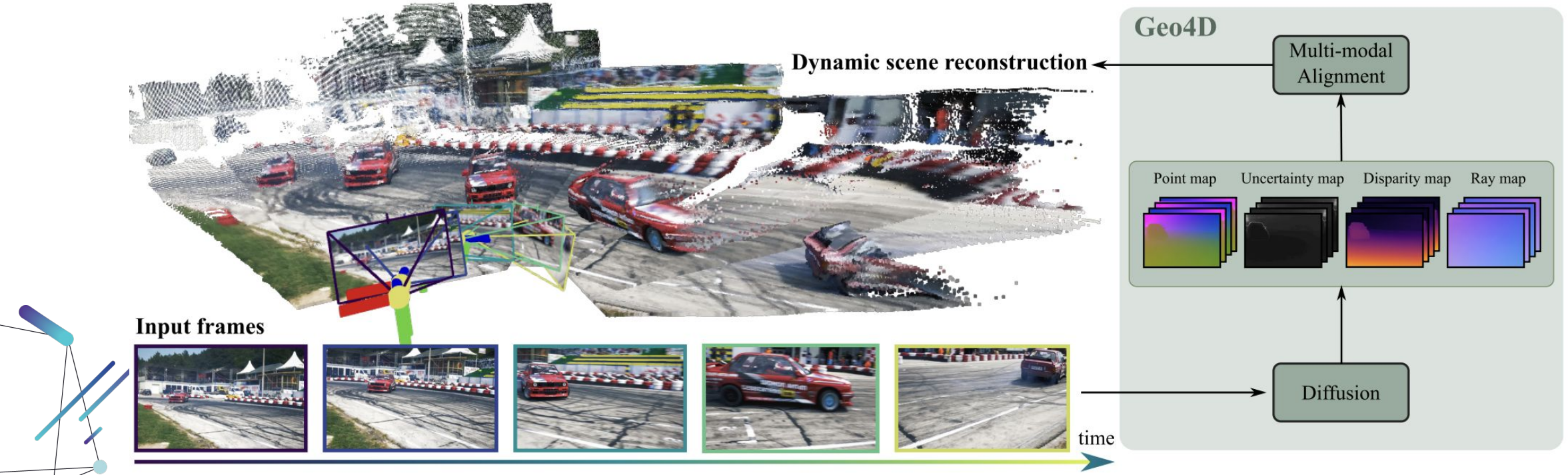
Synthetic → real
Thanks to the strong prior, synthetic training data is enough: zero-shot on real videos

Per frame: three modalities

Point maps 3D point per pixel, shared frame

Disparity inverse depth

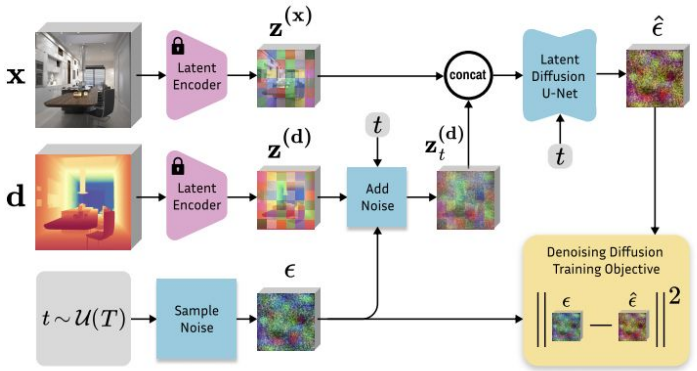
Ray maps camera rays → camera pose



Marigold · 2023 → Geo4D · 2025 →

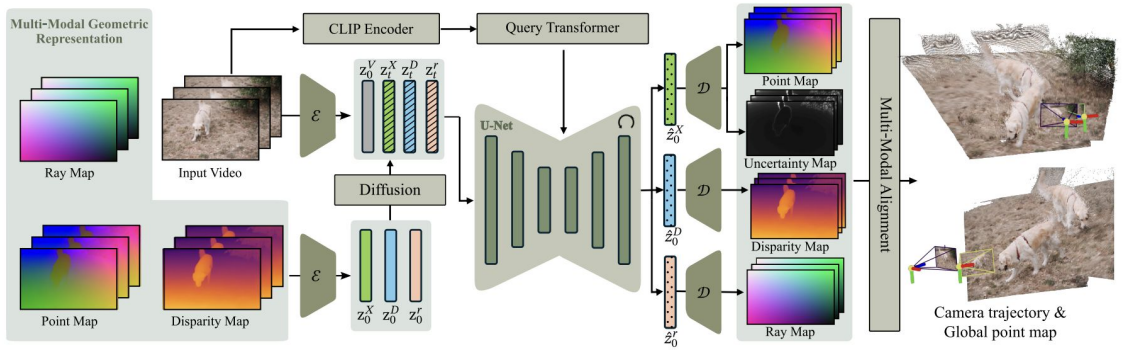
Marigold vs. Geo4D: the same recipe for video

MARIGOLD · IMAGE → DEPTH



BASE	Stable Diffusion v2, images
INPUT	image latent, concatenated by channels
VAE	depth through the image VAE
OUTPUT	affine-invariant depth

GEO4D · VIDEO → 4D



BASE	**DynamiCrafter, video diffusion**
INPUT	**video latents by channels + CLIP per frame**
VAE	**same VAE; point-map decoder fine-tuned**
OUTPUT	**point maps + disparity + ray maps → 4D**

DLS

PART 6 · INTERLUDE

Forcing geometry in diffusion

Can we improve the geometry inside?



Geometry Forcing · 2025 → VGGT · 2025 → REPA · 2024 →

DLS

Video diffusion: not perfect geometry

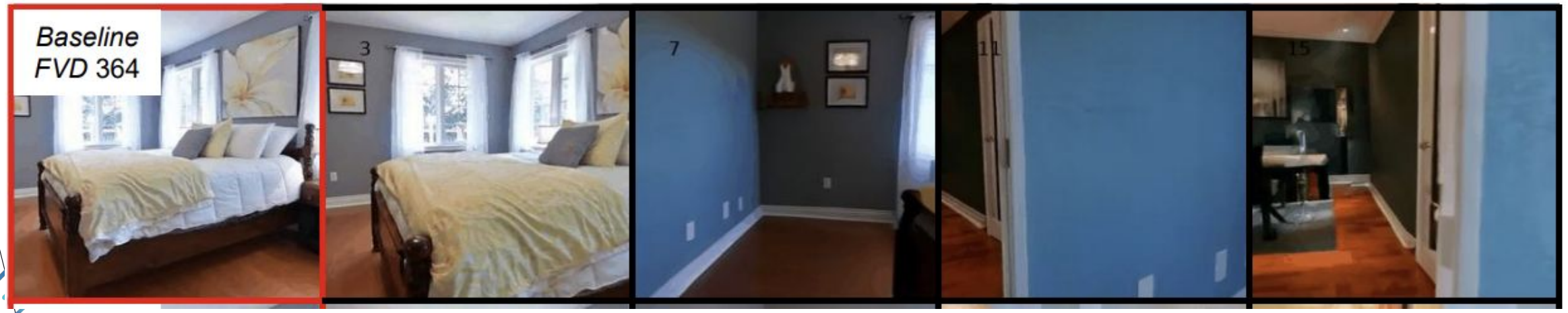
Question

How do we make a video model's geometric prior explicit and useful for generation?

Observation

Trained on raw video only, video diffusion sometimes learns not really good geometric structure in its features





VGGT · 2025 → | DUSt3R · 2023 → | GitHub →

DLS

VGGT: images → 3D in one pass

What it is

A feed-forward transformer, about 1.2B parameters: one, a few or hundreds of photos → 3D in one pass.

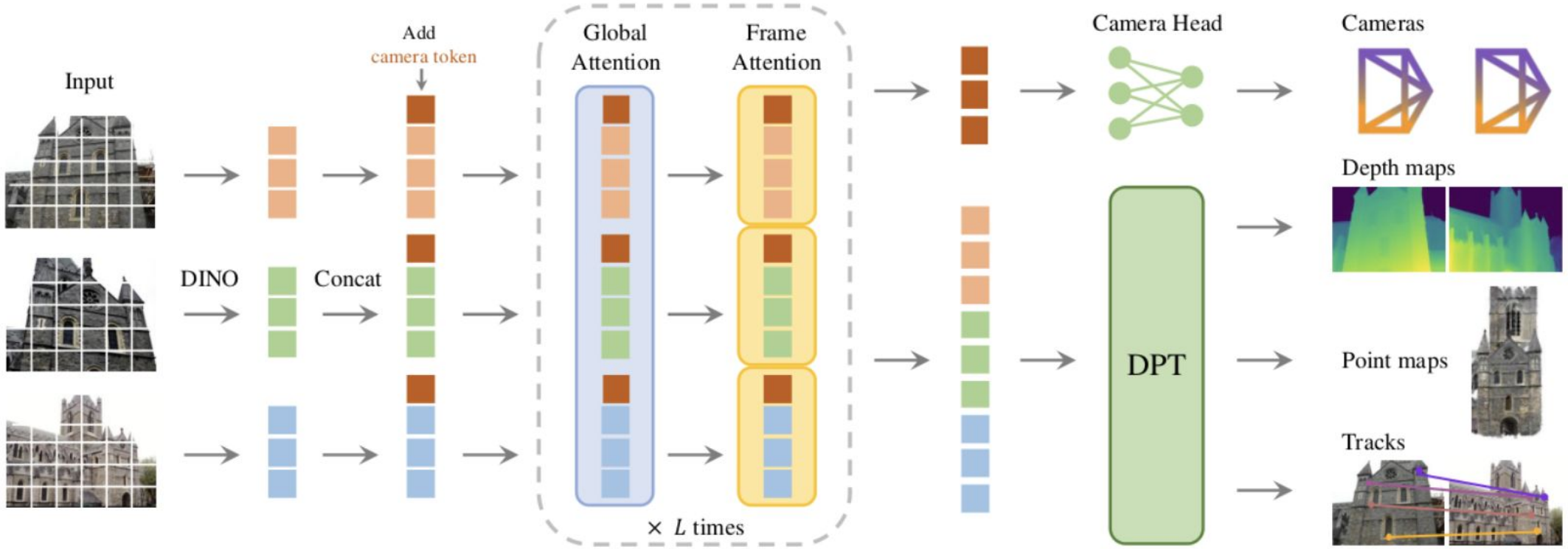
Outputs, for every image

Cameras intrinsics + extrinsics

Point maps in the first camera's frame

Depth a depth map

Tracks 3D point tracks



Geometry Forcing: make geometry explicit

Method, REPA-style

Align intermediate features with VGGT, REPA-like training

Angular Alignment

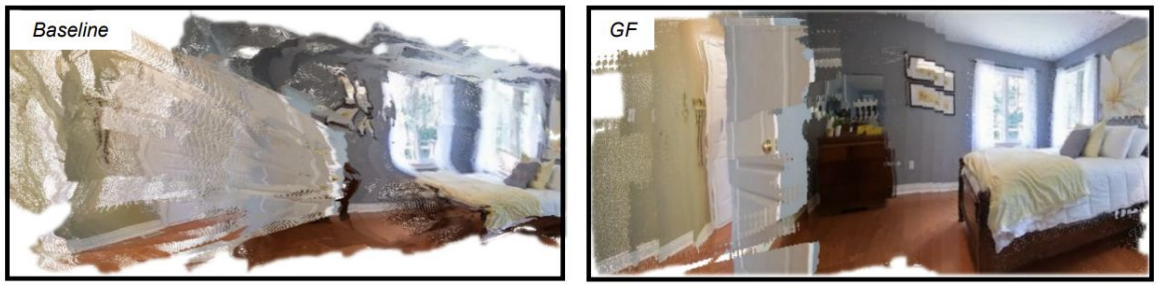
$$\mathcal{L}_{\text{ang}} = 1 - \cos\big(P(\mathbf{h}),\, \mathbf{y}\big)$$

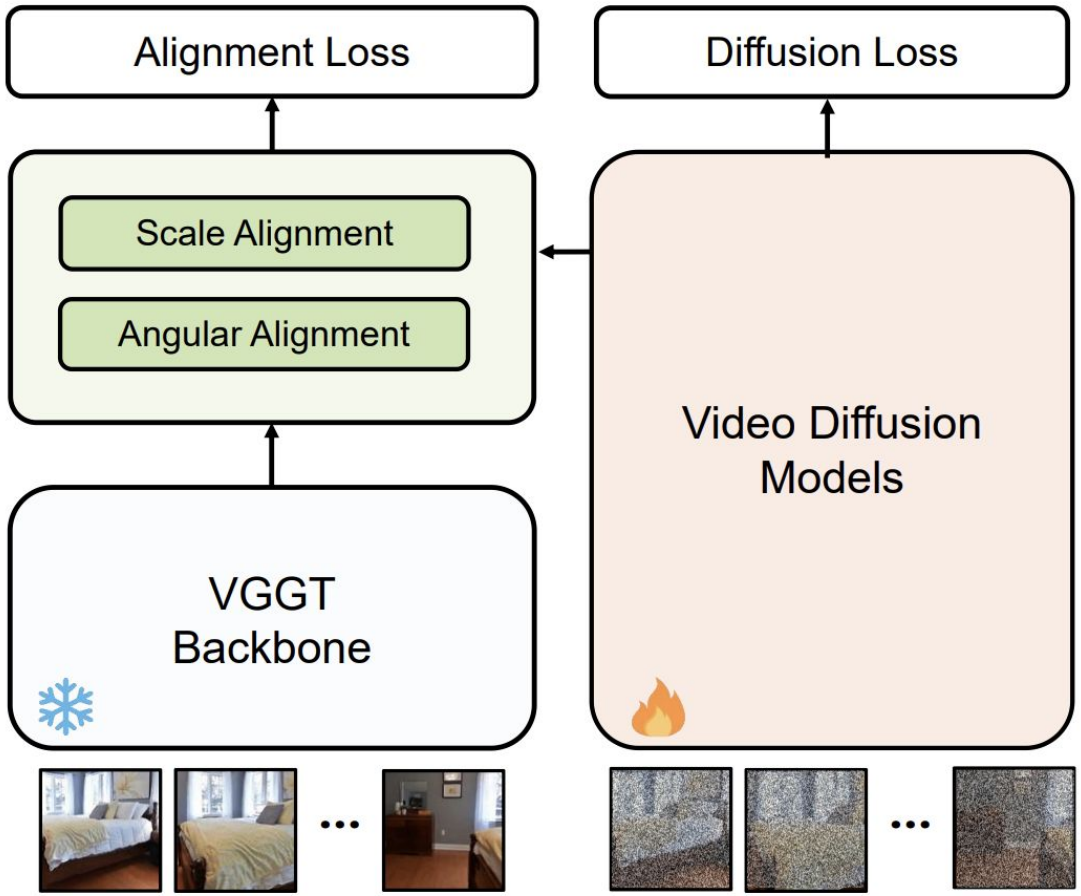
same direction: cosine similarity

Scale Alignment

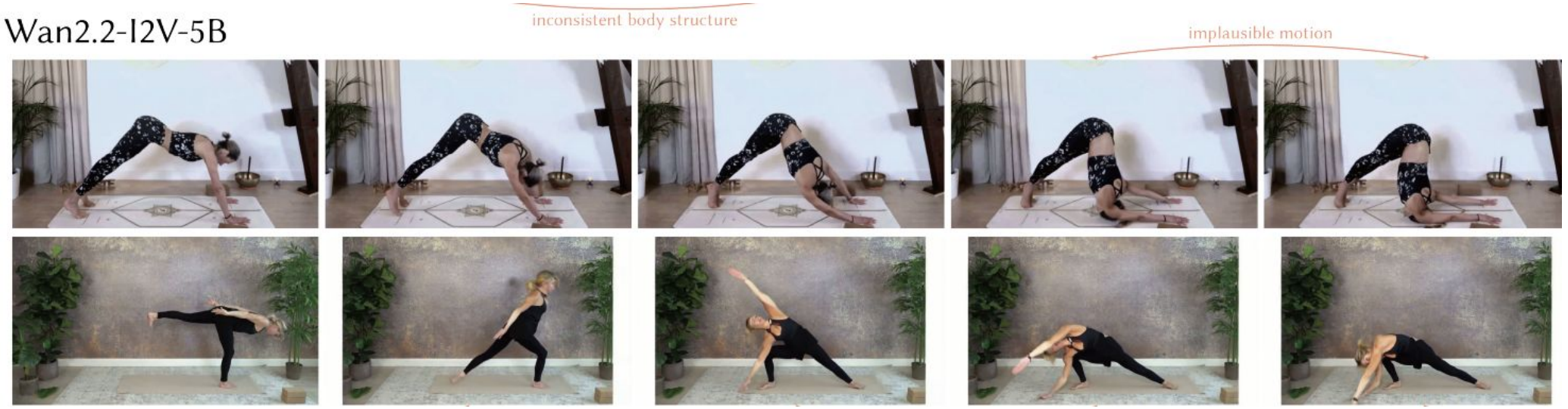
$$\mathcal{L}_{\text{sc}} = \left\| H\left(\frac{P(\mathbf{h})}{\|P(\mathbf{h})\|}\right) - \mathbf{y} \right\|^2$$

keeps the scale information





Co-generation for improvements video generation=



Co-generation for improvements video generation

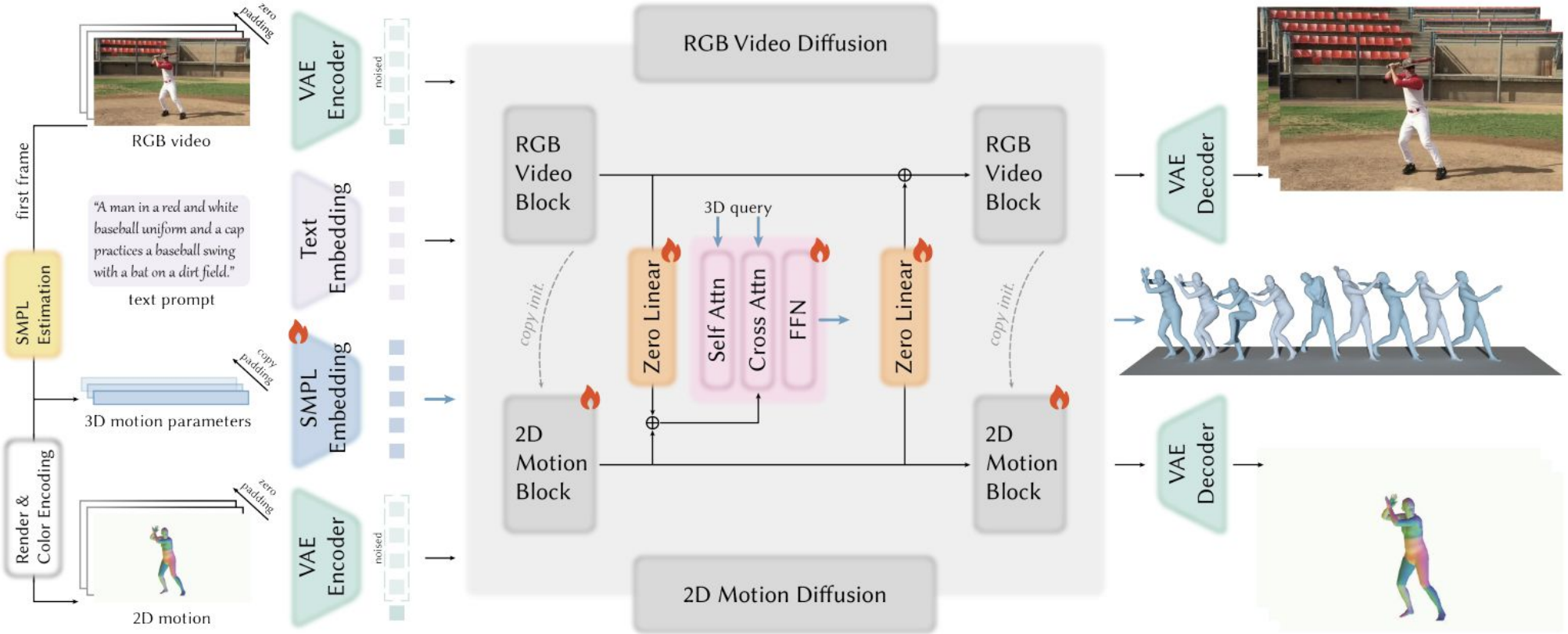


CoMoVi · 2026 → Wan · 2025 →

CoMoVi: 3D human motion + video, together

Idea

3D motion gives the video a structural prior, the video gives motion realism. Generate both at once instead of a cascade



CoMoVi · 2026 → Wan · 2025 →

CoMoVi: 3D human motion + video, together

Wan2.2-I2V-5B

inconsistent body structure

implausible motion



Ours

inconsistent subject



1st row prompt: "A woman in black and white patterned yoga attire performs the downward-facing dog yoga pose on a white yoga mat."
2nd row prompt: "A woman in black activewear holds her arms behind her, then twists her torso and reaching her arms up towards the ceiling."

CoMoVi · 2026 →

Wan · 2025 →

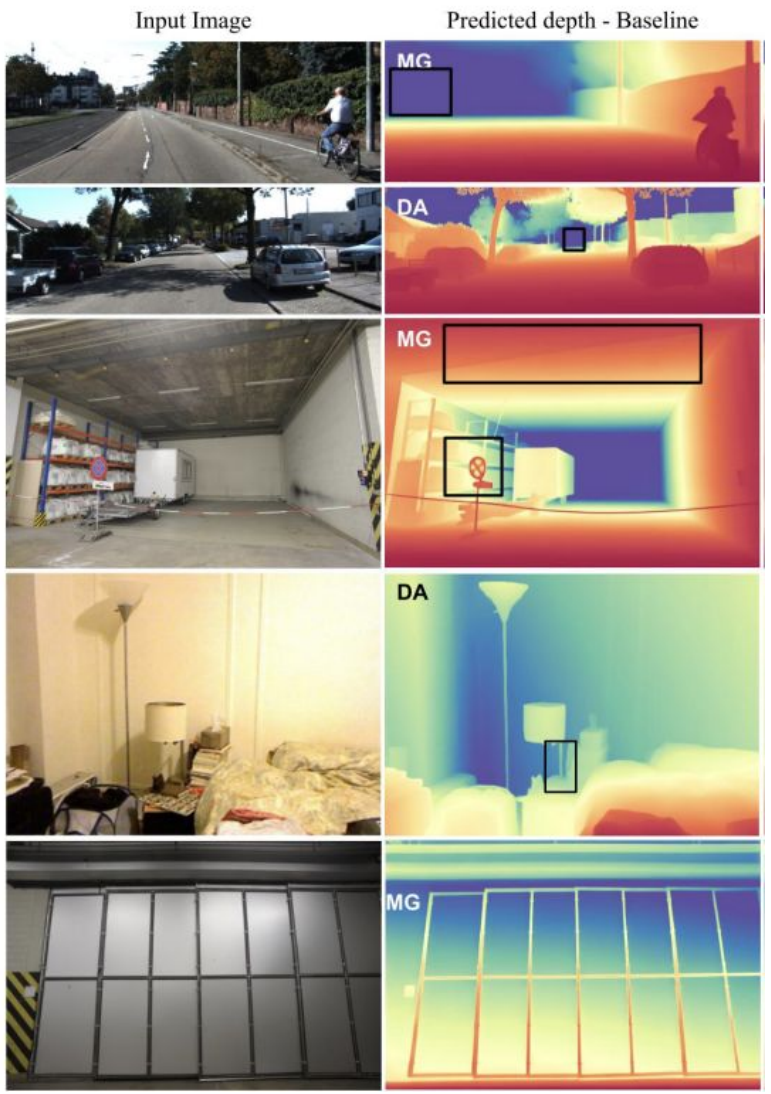
CoMoVi: 3D human motion + video, together

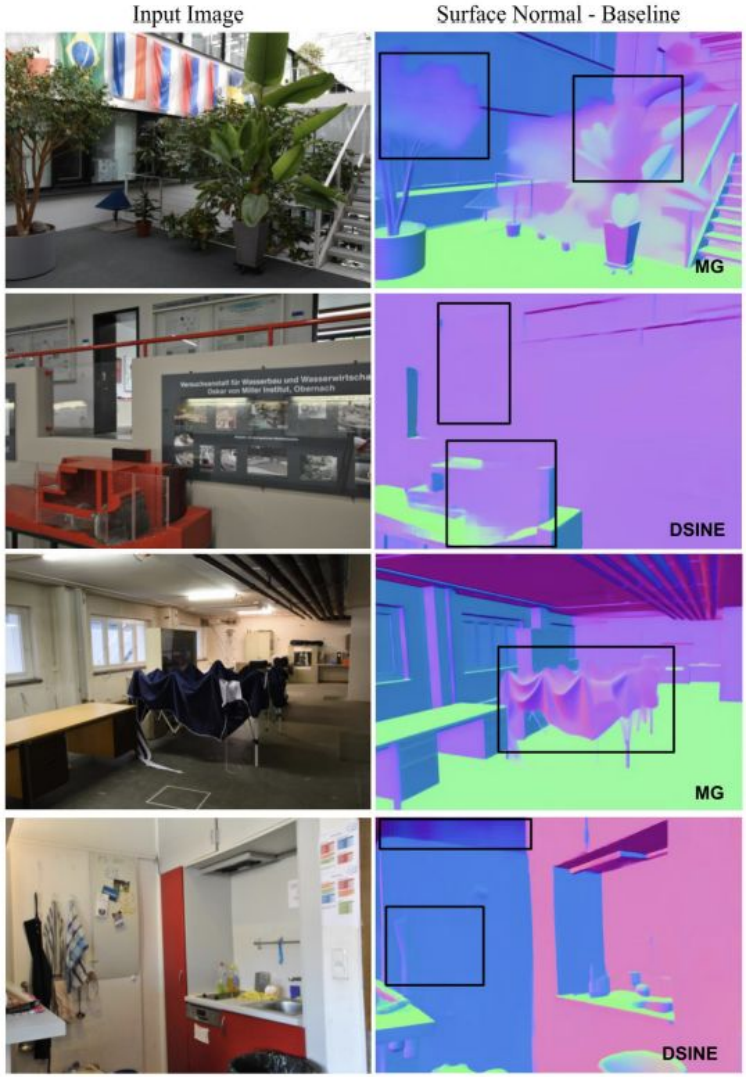


[Orchid · 2025 →](#) [Depth Anything V2 · 2024 →](#)

DLS

Cogeneration for better estimation





Orchid · 2025 →

Depth Anything V2 · 2024 →

Orchid: one latent for color, depth, normals

Idea

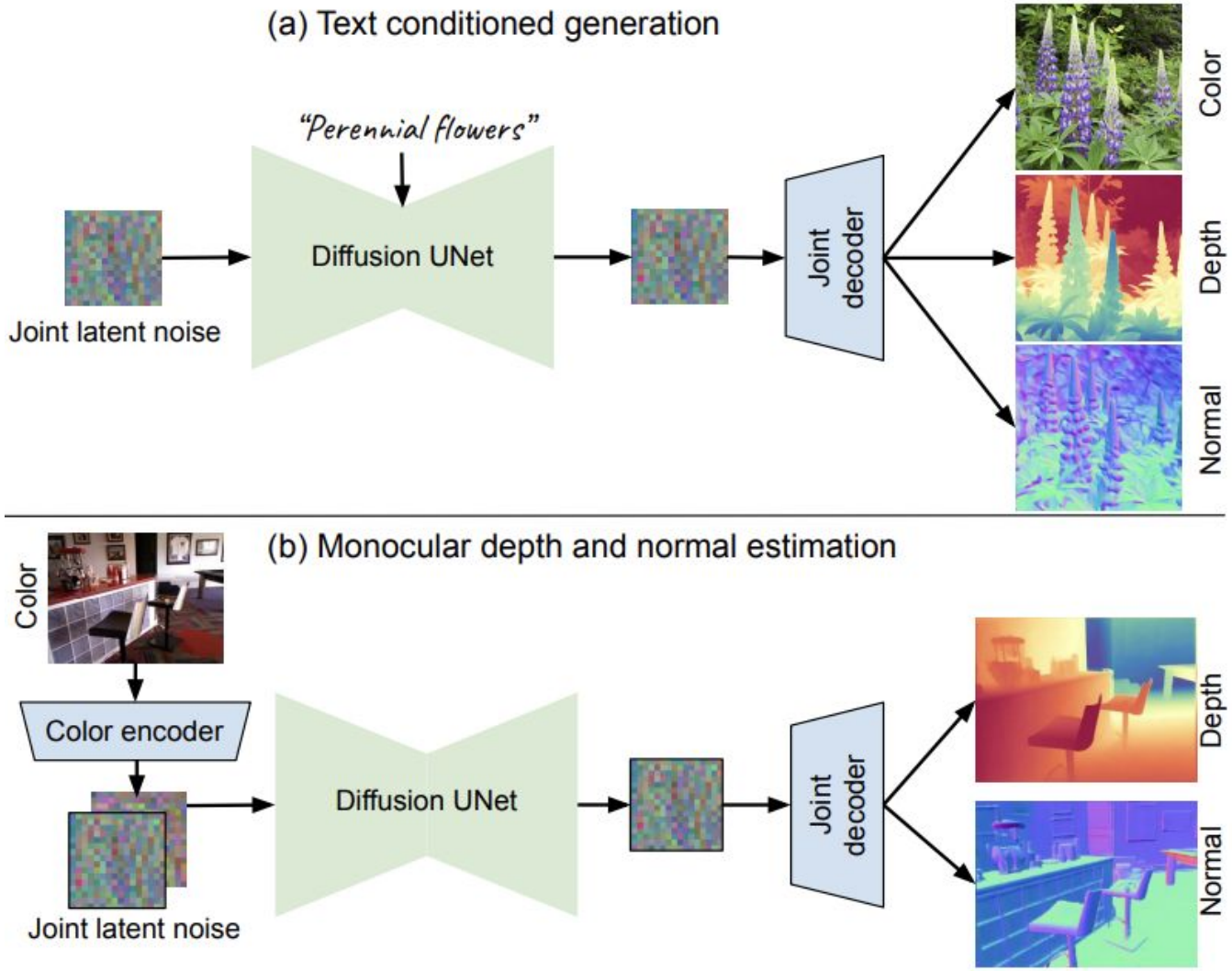
One latent diffusion for RGB, depth and normals, instead of separate color and geometry models

One model, several tasks

text → RGB + depth + normals

image → depth + normals

inpainting → all three at once → 3D scene completion

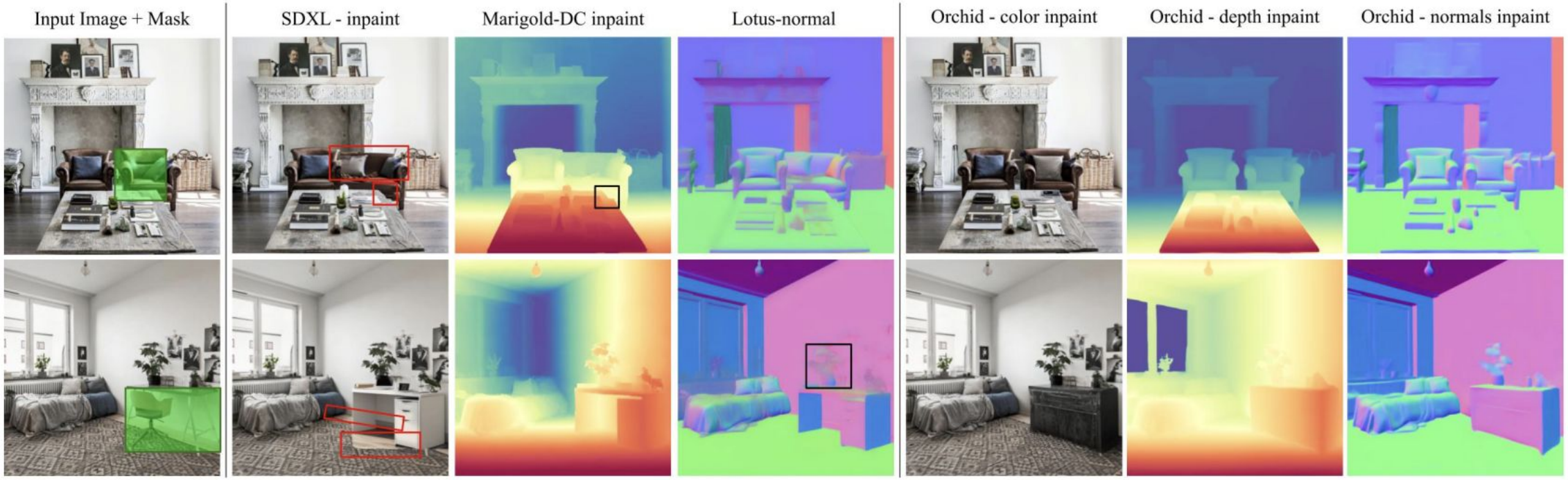


Orchid · 2025 →

Depth Anything V2 · 2024 →

DLS

Orchid: one latent for color, depth, normals

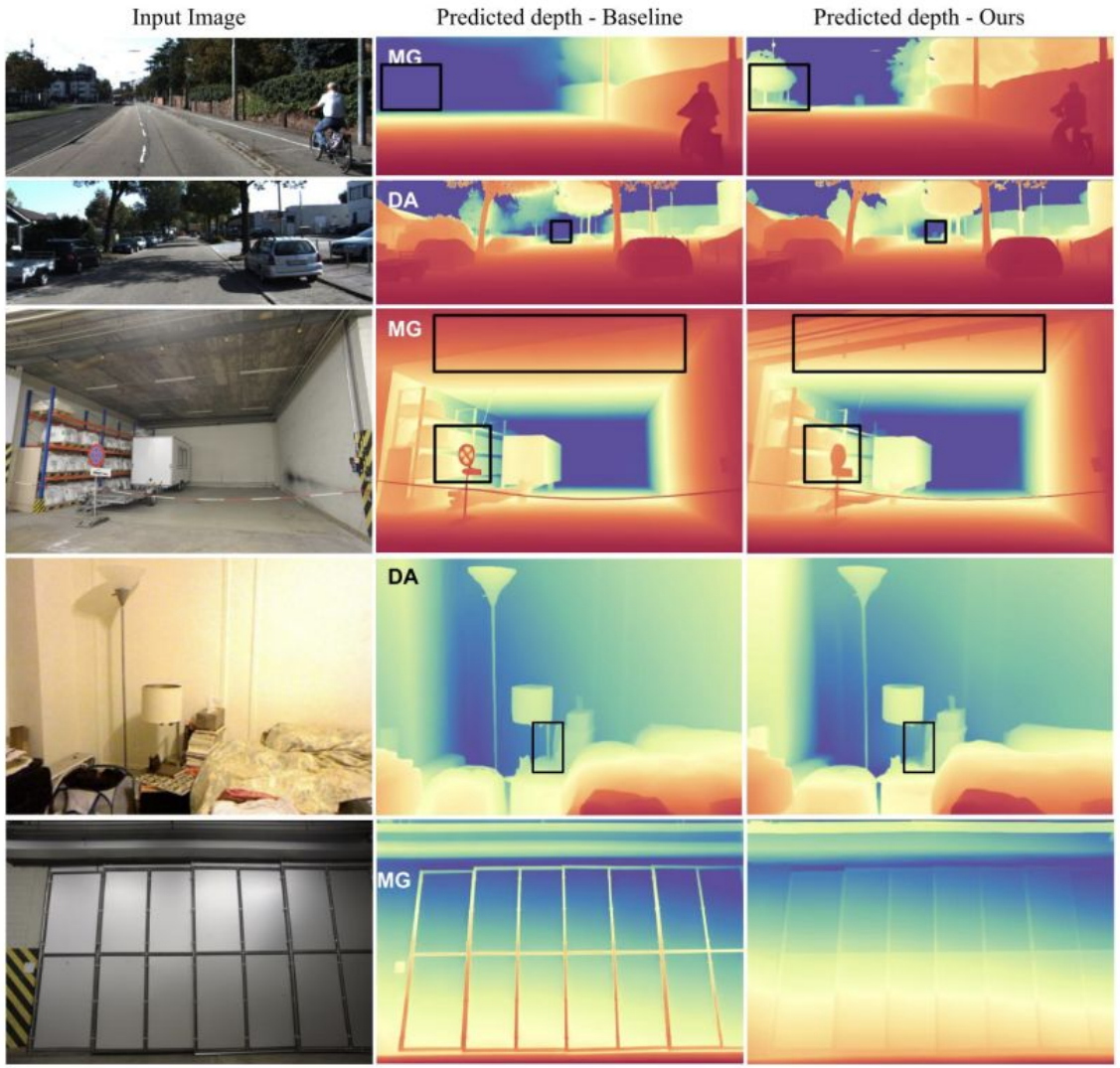


Orchid · 2025 →

Depth Anything V2 · 2024 →

DLS

Orchid: one latent for color, depth, normals



Model	NYUv2		KITTI		ETH3D		ScanNet	
	AbsRel ↓	δ_1 ↑	AbsRel ↓	δ_1 ↑	AbsRel ↓	δ_1 ↑	AbsRel ↓	δ_1 ↑
MiDaS [41]	11.7	87.5	23.6	63.0	18.4	75.2	12.1	84.6
DepthAnything v2 [64]	**4.5**	**97.9**	**7.4**	**94.6**	6.8	95.3	**6.0**	**96.3**
Marigold-depth [24]	6.1	95.8	9.8	91.8	6.8	95.6	6.9	94.6
Lotus-G-depth [18]	5.4	96.6	11.3	87.7	**6.2**	96.1	**6.0**	96.0
Geowizard [13]	5.6	96.3	14.4	82.0	6.6	95.8	6.4	95.0
Orchid (Ours)	5.7	96.9	7.7	94.4	7.3	**96.9**	6.3	95.8

Table 2. **Zero-shot monocular depth estimation**: Comparison of our zero shot affine-invariant depth estimation accuracy to that of other methods. Being a joint depth-normal predictor, ours is comparable to the SOTA depth-only prediction model [64] while being better than the other joint depth-normal method [13]. The first, second and third ranking methods are highlighted.

Model	NYUv2		ScanNet		iBims-1		Sintel	
	Mean ↓	11.25° ↑	Mean ↓	11.25° ↑	Mean ↓	11.25° ↑	Mean ↓	11.25° ↑
Omnidata v2 [9]	17.2	55.5	16.2	60.2	18.2	63.9	40.5	14.7
DSINE [2]	16.4	59.6	16.2	61.0	17.1	67.4	34.9	21.5
Marigold-normal [24]	20.9	50.5	21.3	45.6	18.5	64.7	-	-
Lotus-G-normal [18]	16.9	59.1	15.3	**64.0**	17.5	66.1	35.2	19.9
Geowizard [13]	18.9	50.7	17.4	53.8	19.3	63.0	40.3	12.3
Orchid (Ours)	**15.2**	**60.6**	**14.2**	63.8	**16.3**	**68.1**	**31.7**	**22.6**

Table 3. **Zero-shot monocular normals estimation**: Orchid's color-conditioned surface normal predictions are significantly better than other SOTA methods, including specialized surface normal prediction models. First, second and third ranking methods are highlighted.

Aether · 2025 → CogVideoX · 2024 →

DLS

Aether: generate geometry together with video



Aether · 2025 → CogVideoX · 2024 →

DLS

Aether: generate geometry together with video

Everything in video format

RGB ordinary video, through the video VAE

Depth scale-invariant normalized disparity, as a 3-channel video through the VAE

Camera raymaps, downsampled and stacked to the latent shape

Random masking in training

Some modalities are given as conditions, the rest are generated

ONE MODEL · TASK	GIVEN	GENERATES
4D reconstruction	video	depth + camera
Prediction	first frame (+ camera path)	future video + geometry
Planning	first frame + goal	the path to the goal

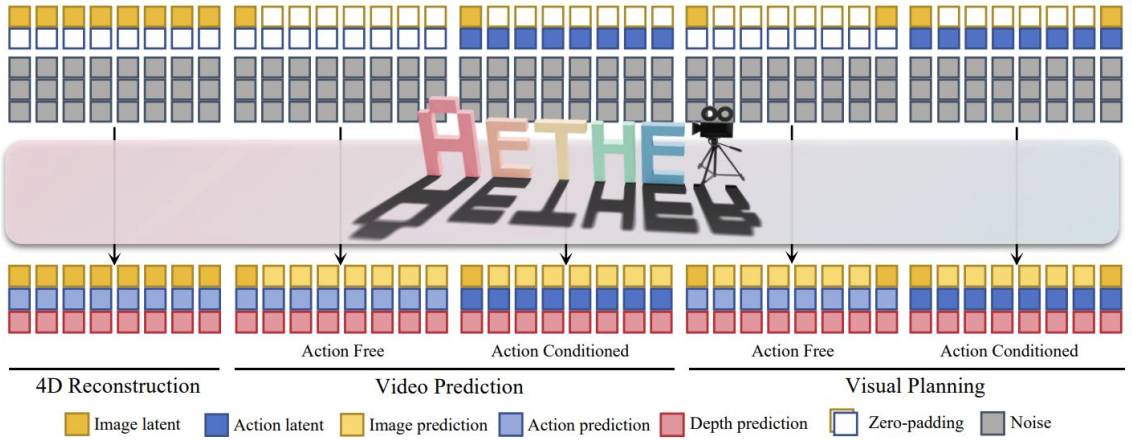
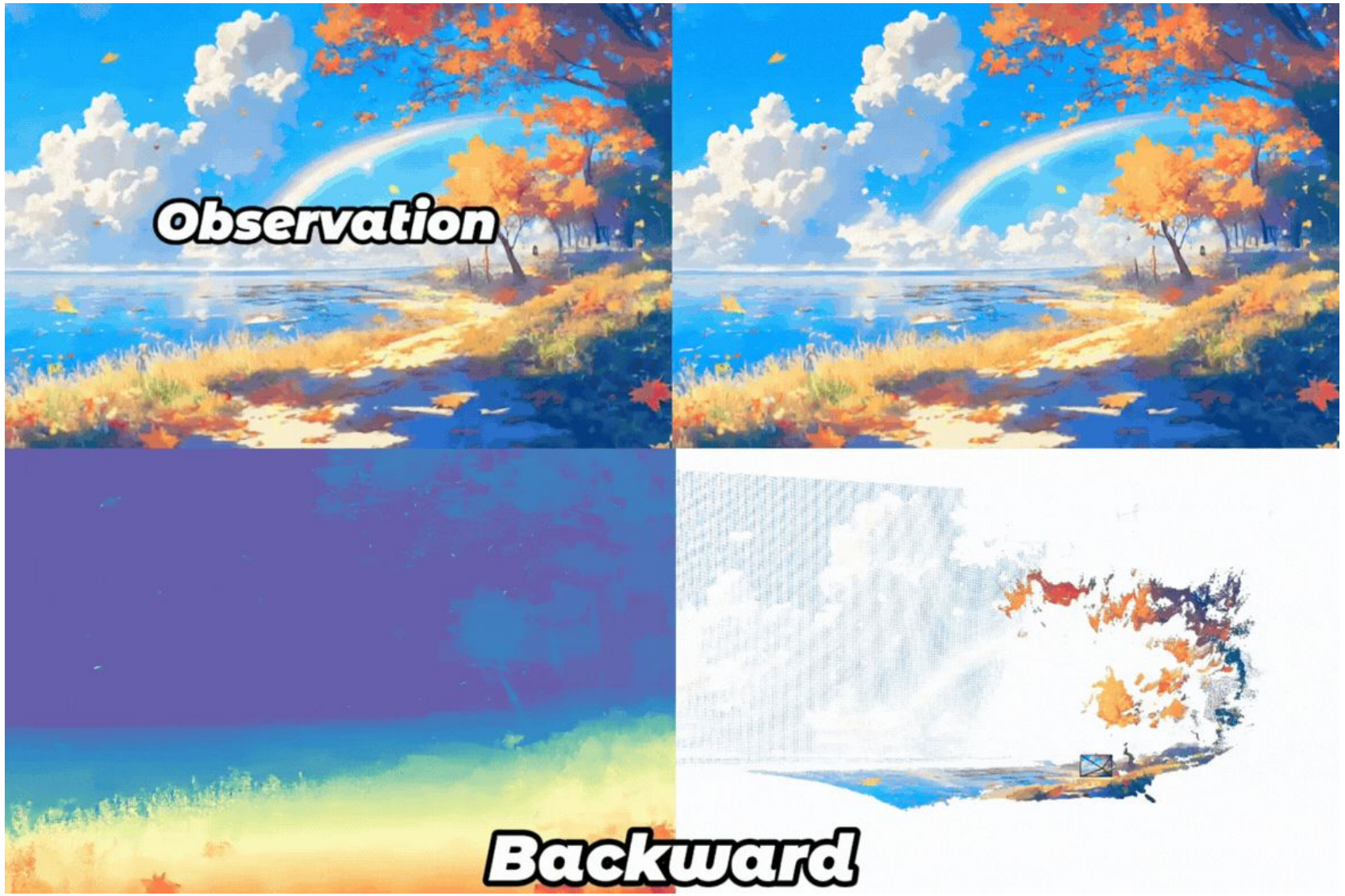


Figure 4. The overall pipeline of AETHER. With different condition combinations, AETHER can serve different tasks.

Aether · 2025 →

CogVideoX · 2024 →

Aether: generate geometry together with video



DLS

Video + X: generated together

VIDEO + DEPTH

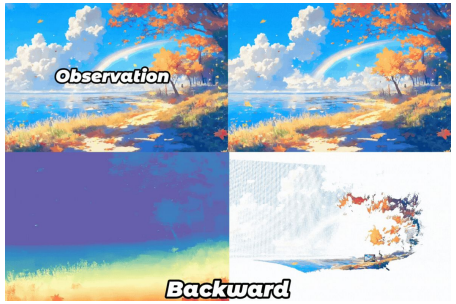
DONE



Aether · WorldWeaver · GeoVideo

VIDEO + CAMERA

DONE



Aether · WVD

VIDEO + MOTION

DONE



VideoJAM · CoMoVi

IMAGE + DEPTH + NORMALS

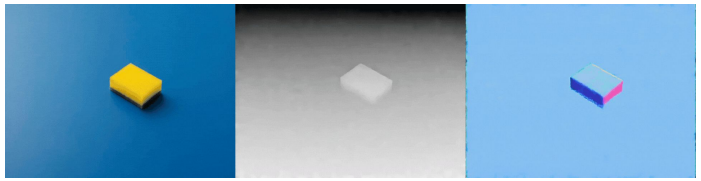
DONE



Orchid · JointDiT

VIDEO + NORMALS

DONE



TesserAct

VIDEO + ?

MAYBE EVEN MORE…

+ ?

DLS

PART 7 · INTERLUDE

Can we generate 3D world?



DLS

Goal: a 3D world you can walk through

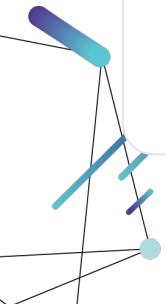
What does it take?

01
Looks real
like a real video

02
Follows where we go
we choose the camera

03
Stays the same when we return
the world remembers

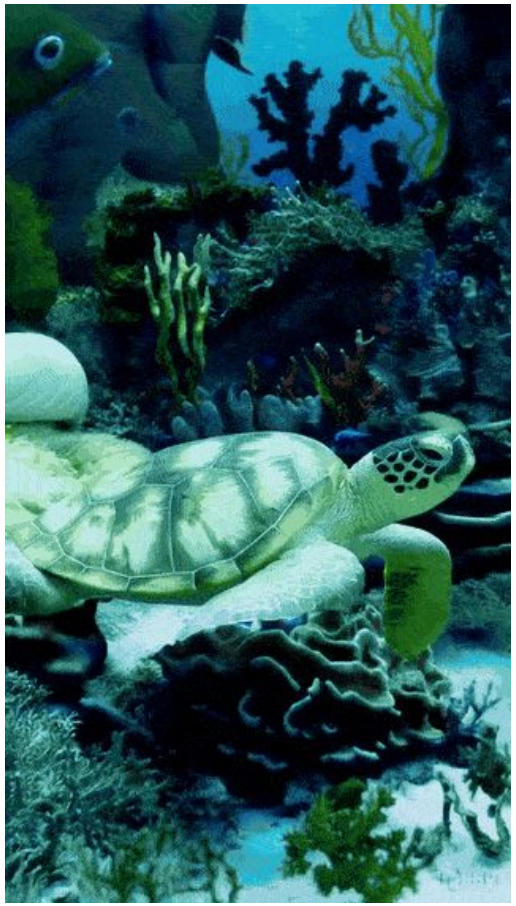
04
Real 3D in the end
something we can open and walk in



Sora · 2024 →

Wan · 2025 →

We already have video generators







They have seen huge amounts of video

and already draw a realistic world

CHECKLIST

- [x] Looks real
- [ ] Follows the camera
- [ ] Same when we return
- [ ] Real 3D

DLS

Problem: we can’t control them



A red vintage car parked alone on a cracked white salt flat at noon, clear blue sky, photorealistic. The camera performs a **smooth 180-degree orbit counterclockwise** around the car at a constant distance, keeping the car centered, and ends looking at the car's rear bumper.

CHECKLIST

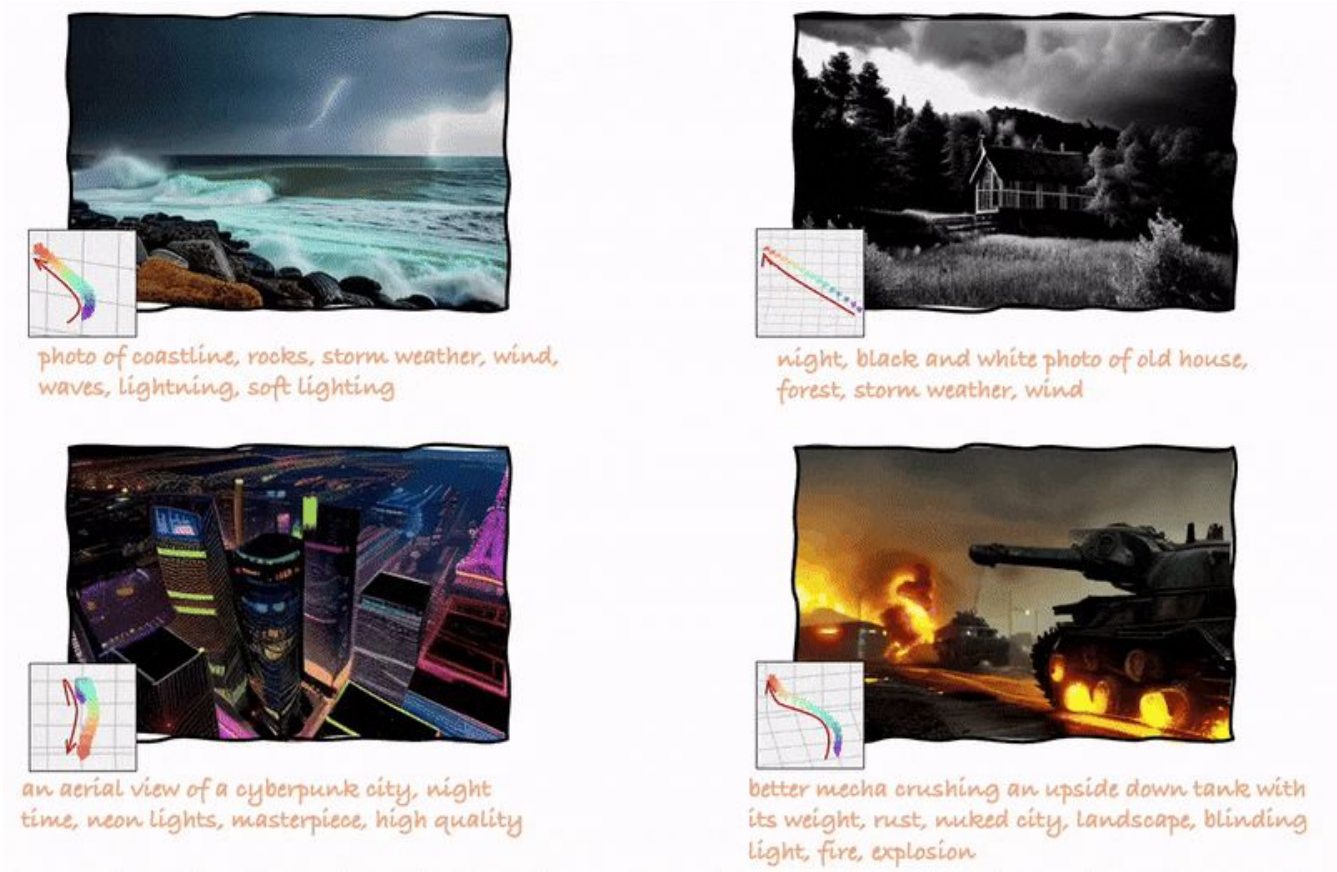
- [x] Looks real
- [ ] Follows the camera
- [ ] Same when we return
- [ ] Real 3D

MotionCtrl · 2023 →

CameraCtrl · 2024 →

DLS

Add the camera



Give the model a camera trajectory as a condition

CHECKLIST

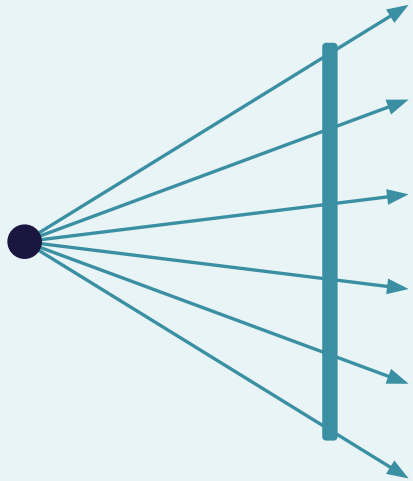
- [x] Looks real
- [ ] Follows the camera
- [ ] Same when we return
- [ ] Real 3D

How to add the camera

$$[R \mid \mathbf{t}]_f$$

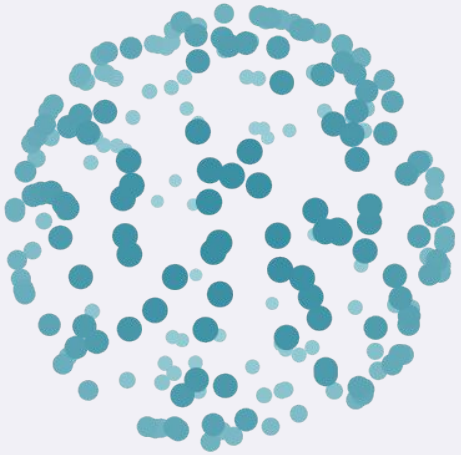
Camera parameters per frame

MotionCtrl



A ray for every pixel

CameraCtrl



Render of a coarse 3D scene

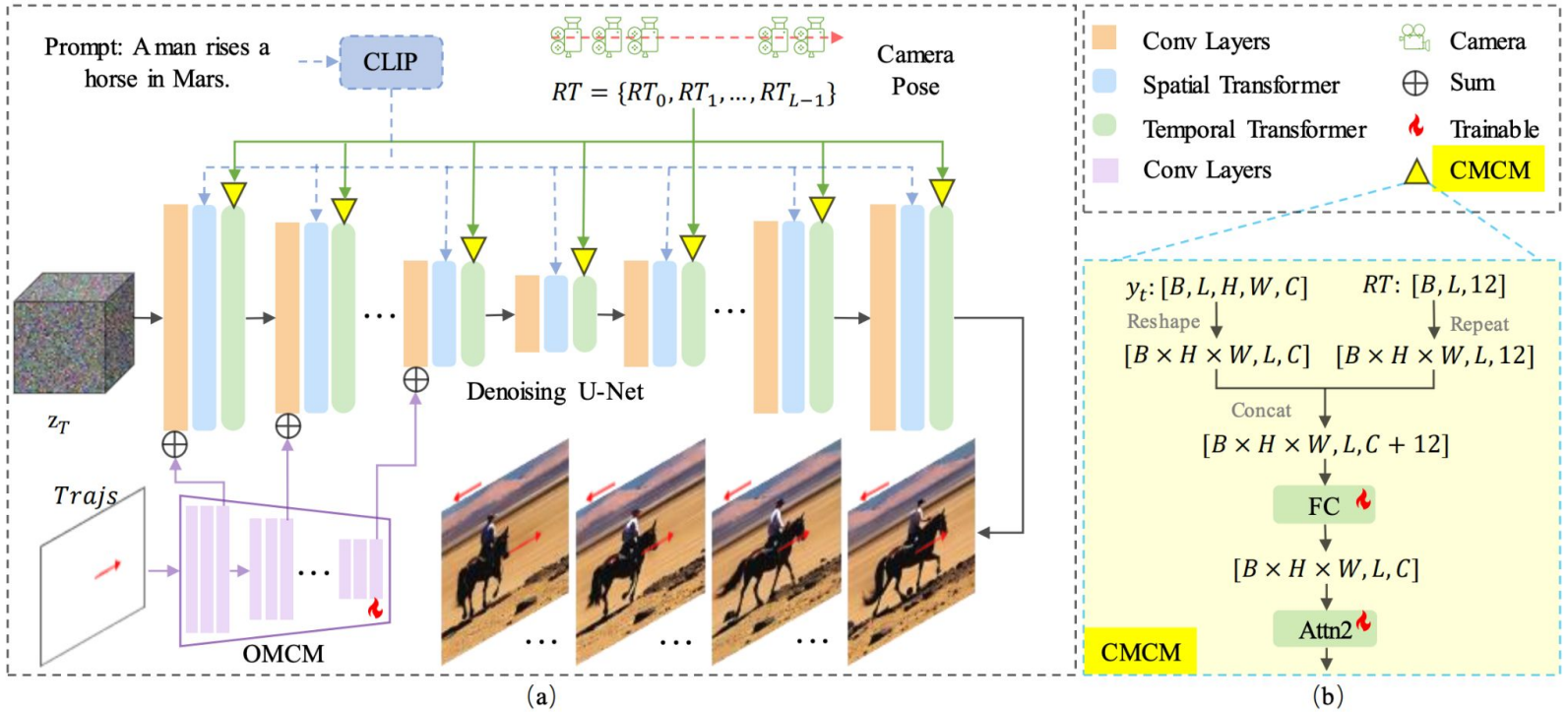
GEN3C

CHECKLIST

- [x] Looks real
- [ ] Follows the camera
- [ ] Same when we return
- [ ] Real 3D

MotionCtrl · 2023 →

MotionCtrl



$$\left[R \mid \mathbf{t} \right]_f$$

Camera parameters per frame

MotionCtrl

CHECKLIST

- [x] Looks real
- [ ] Follows the camera
- [ ] Same when we return
- [ ] Real 3D

Motion Ctrl · 2023 →

MotionCtrl

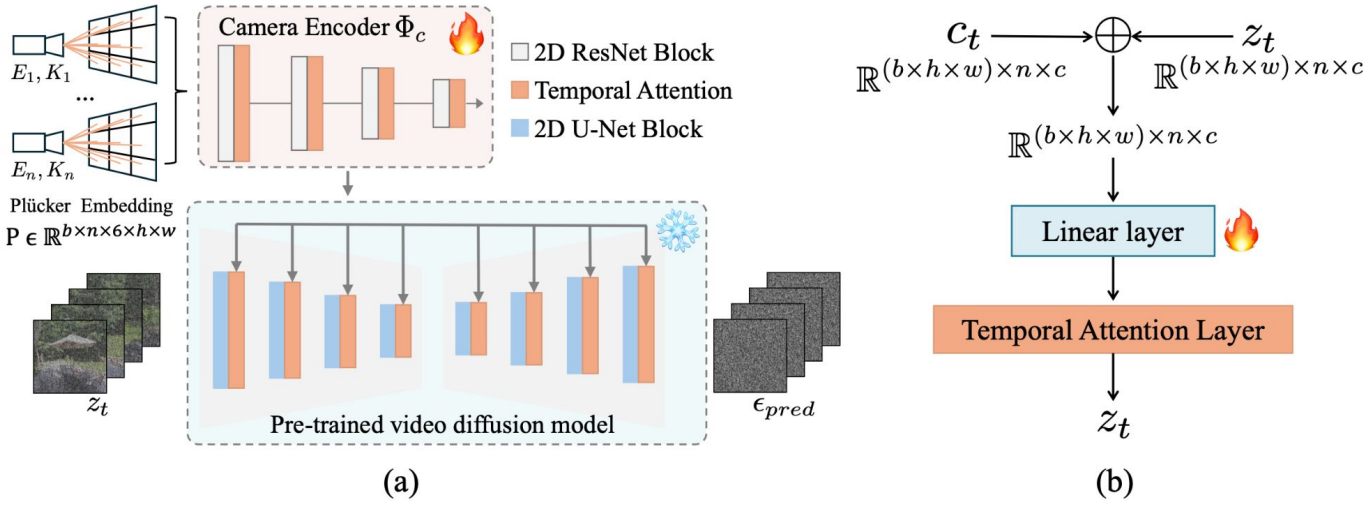


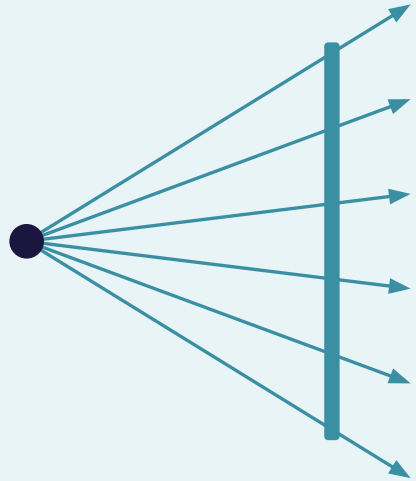
CHECKLIST

- [x] Looks real
- [ ] Follows the camera
- [ ] Same when we return
- [ ] Real 3D

CameraCtrl · 2024 →

CameraCtrl





A ray for every pixel

CameraCtrl

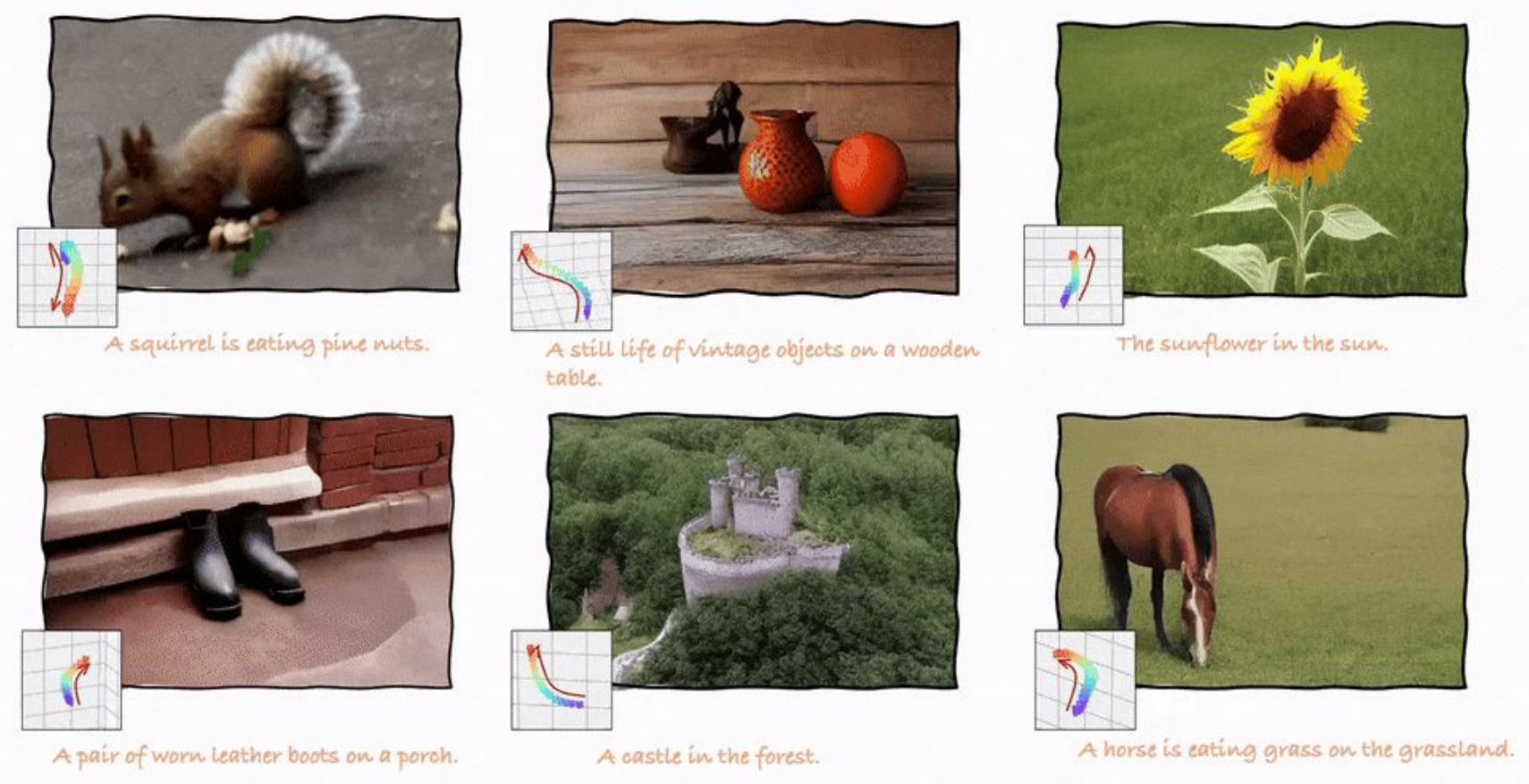
CHECKLIST

- [x] Looks real
- [ ] Follows the camera
- [ ] Same when we return
- [ ] Real 3D

CameraCtrl · 2024 →

DLS

CameraCtrl



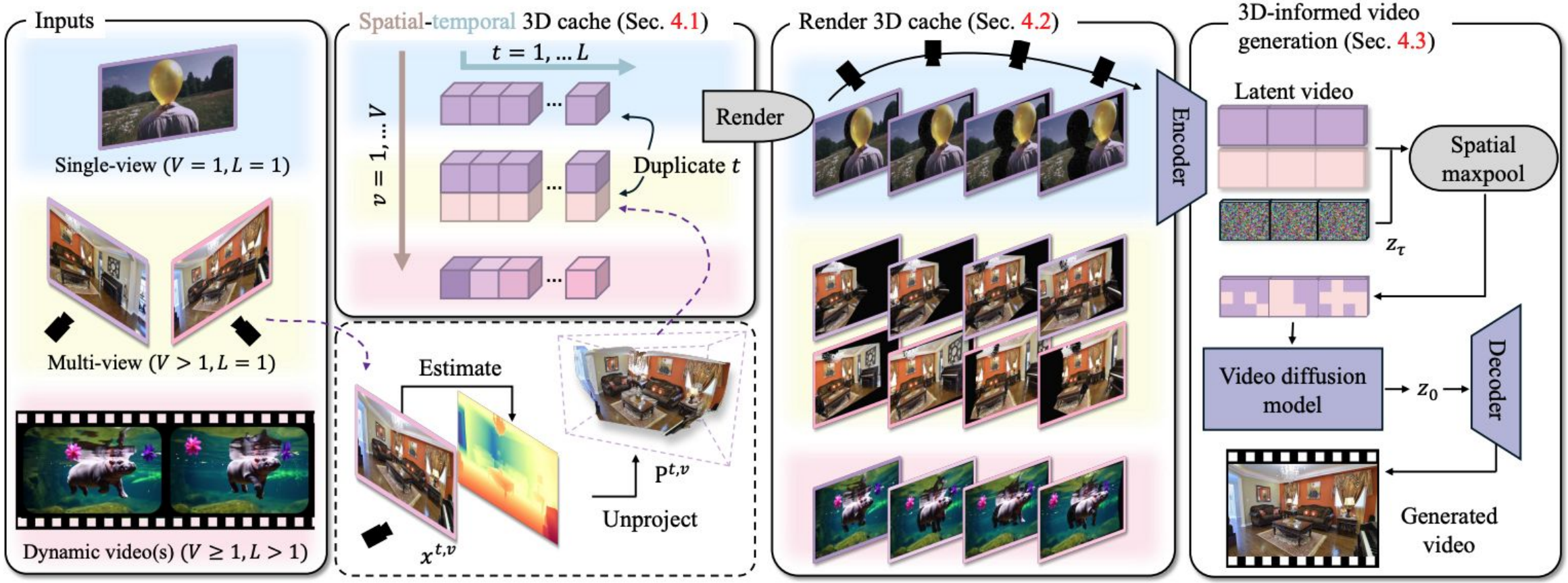
CHECKLIST

- [x] Looks real
- [ ] Follows the camera
- [ ] Same when we return
- [ ] Real 3D

GEN3C · 2025 →

DLS

GEN3C: the camera through 3D



CHECKLIST

- [x] Looks real
- [x] Follows the camera
- [ ] Same when we return
- [ ] Real 3D

GEN3C · 2025 →

GEN3C: the camera through 3D



CHECKLIST

- [x] Looks real
- [x] Follows the camera
- [ ] Same when we return
- [ ] Real 3D

DLS

What about memory?



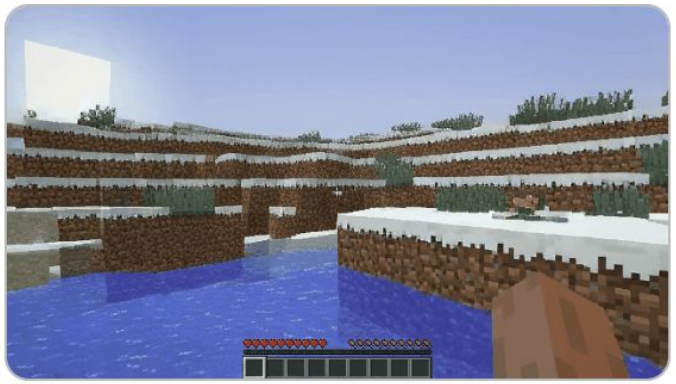
DLS

The world is big — the model forgets





Initial View



Revisited View

CHECKLIST

- [x] Looks real
- [x] Follows the camera
- [ ] Same when we return
- [ ] Real 3D

WorldMem · 2025 → Spatial Memory · 2025 → Voyager · 2025 →

Memory in practice

WorldMem

A bank of past frames with their poses and times; attention picks the relevant ones

Spatial Memory

Working (recent frames) + episodic (key frames) + spatial (a static point cloud)

Voyager

RGB and depth generated together, plus a cache of world points

[figure]

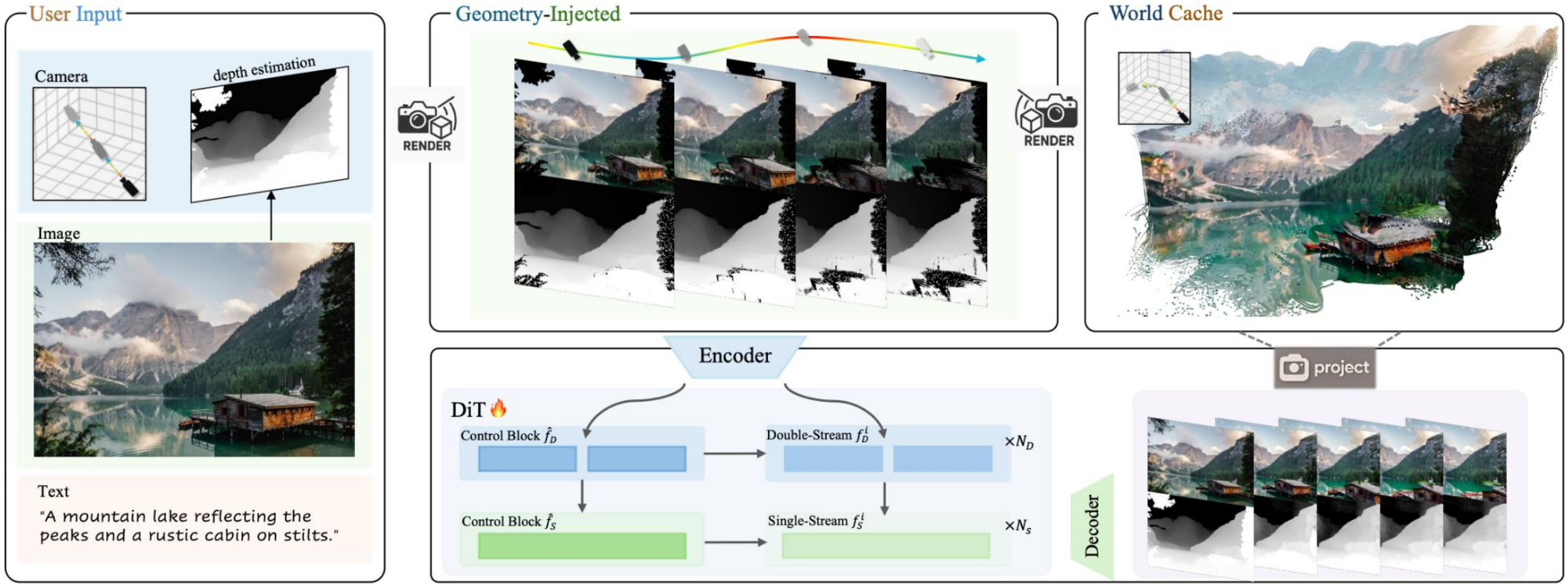
Paper figure: the three memories of Spatial Memory

CHECKLIST

- [x] **Looks real**
- [x] **Follows the camera**
- [x] **Same when we return**
- [ ] Real 3D

Voyager · 2025 →

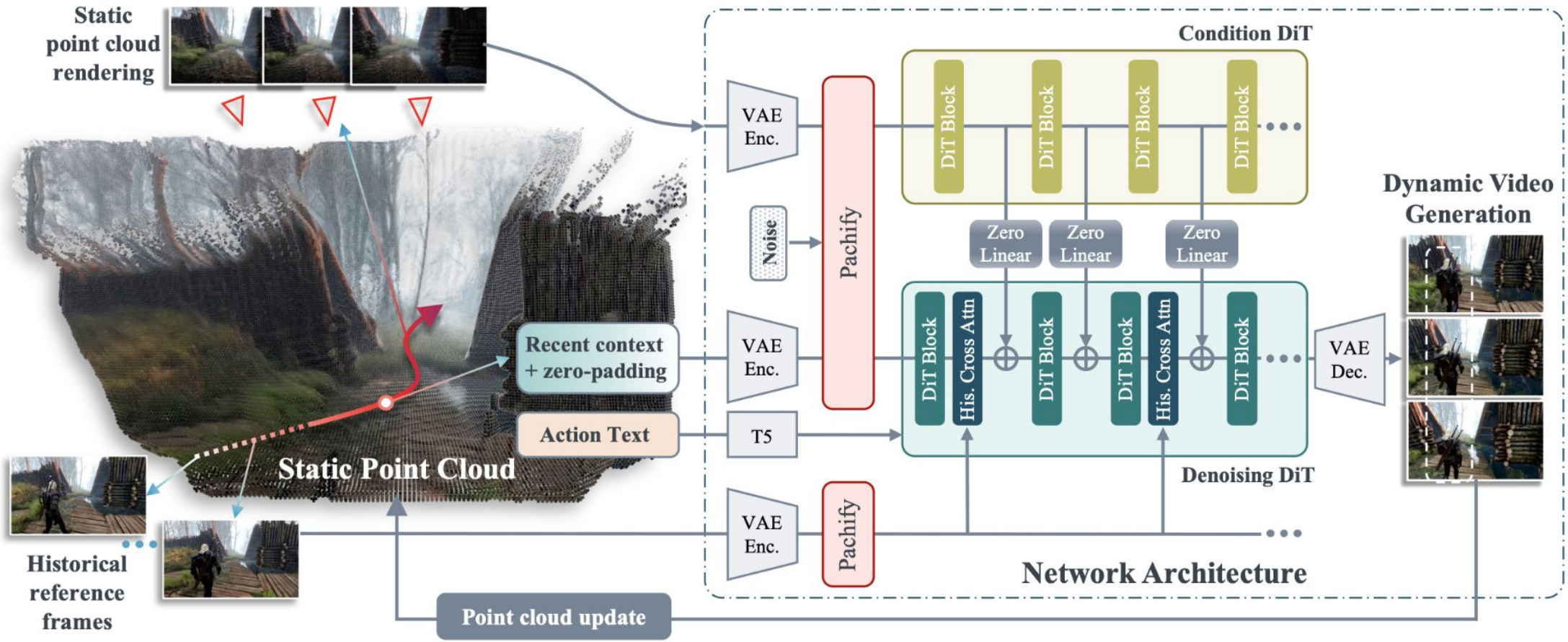
Memory in practice



Spatial Memory · 2025 →

DLS

Memory in practice



DLS

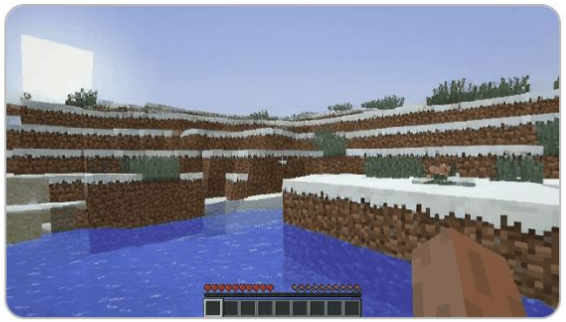
Do not forget!







Initial View



Revisited View



Initial View



Revisited View

CHECKLIST

- [x] Looks real
- [x] Follows the camera
- [ ] Same when we return
- [ ] Real 3D

Spatia →

Spatia

1 Build the 3D cache: rebuilt from scratch before every round

1. **History frames**
   All past frames as pixels (1248×704) + the given camera poses. The only memory kept between rounds
2. **Sample**
   Every 6th frame (map_fps 4), saved as a lossy mp4, resized to 518×294
3. **MapAnything**
   Told the given poses and intrinsics, metric scale. No Umeyama alignment, no confidence threshold
4. **Coloured point cloud**
   Point colour = pixel colour. 1 cm voxels average colours from all frames, so light from different moments is mixed
5. **Render into the round's cameras**
   Re-voxelise at 0.01 + 0.005·round; PyTorch3D, radius 0.01, 5 points/pixel → RGB render + hole mask for all 81 frames
6. **Pick 7 reference frames**
   Candidates: every 4th past frame except the last 9. Targets: every 12th camera (7). Shared front-most points → Hungarian, one frame per camera

2 What the generator sees in one round (rounds ≥ 1)

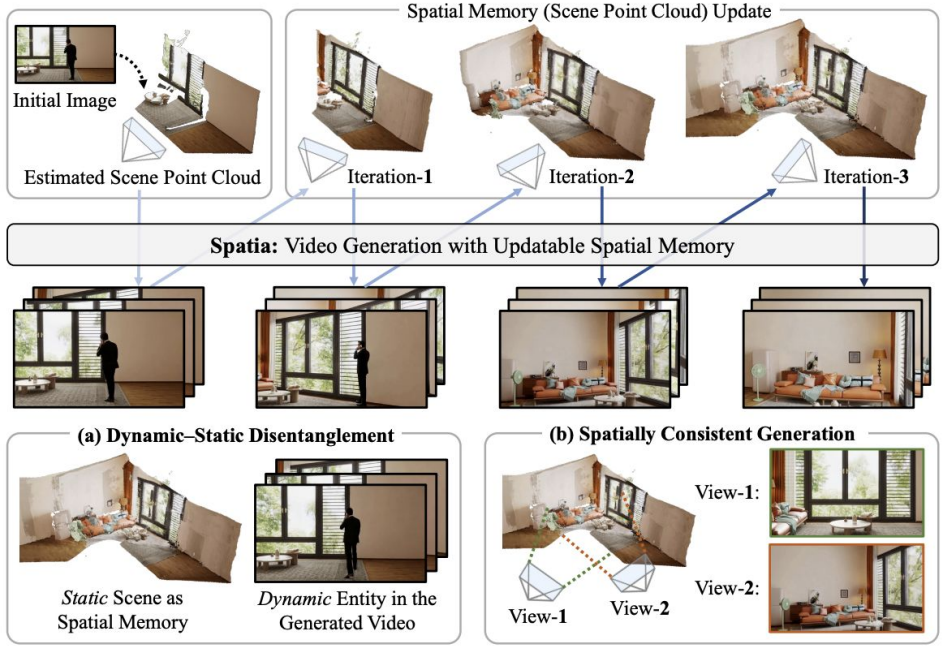
Latent sequence: 28 latents × 858 tokens (22×39) ≈ 24k tokens

7 refs	3 hist	18 new latents (72 frames)

7 refs: each reference frame VAE-encoded alone (1 latent), clean (t = 0), own RoPE placed off the target grid

3 hist: last 9 frames of the previous round, clean, written back after every step

New: denoised from noise; 40 steps, UniPC, CFG 3.5



Put it all together

video generator + camera + memory → long walk → 3D

One problem left: drift

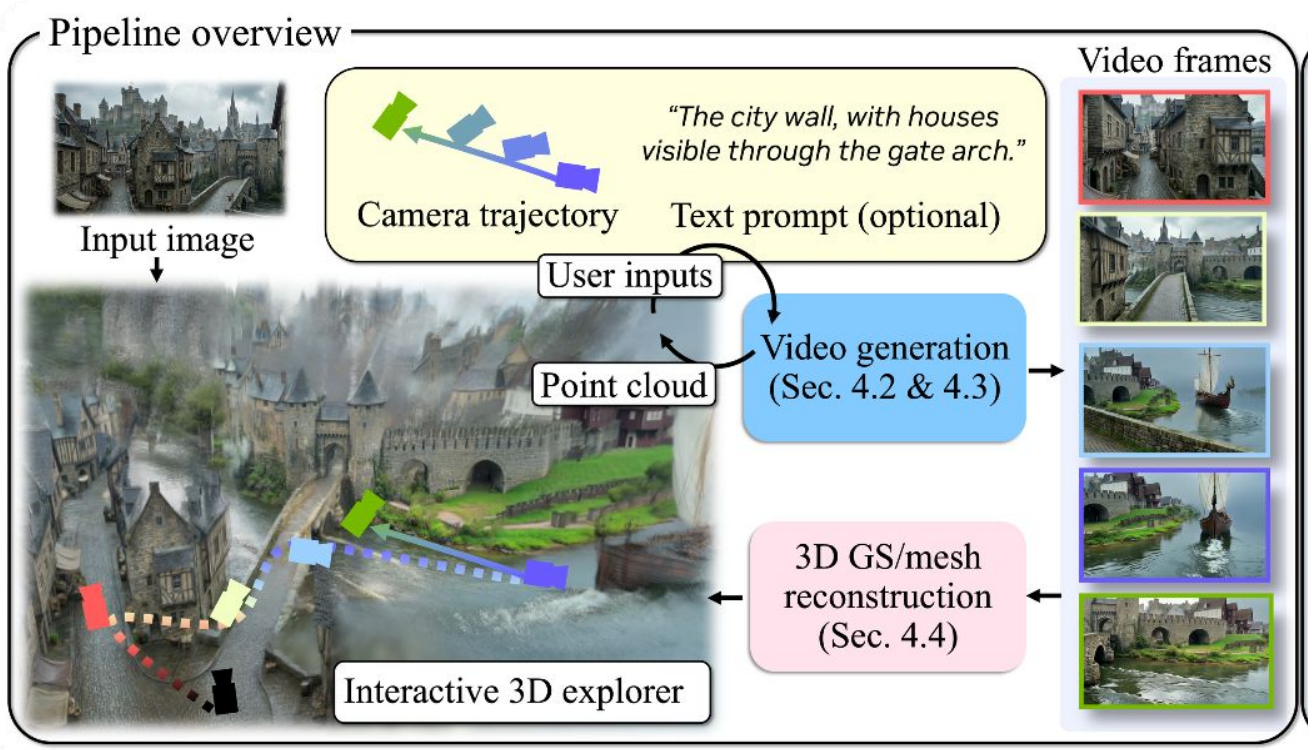
On a long walk, small errors pile up



Lyra 2.0 · 2026 → Project page →

DLS

Lyra 2.0: a world you can open



Walk, then lift

A long camera-controlled video from one image → 3D Gaussians + a mesh

Memory

Per-frame geometry, used only to find the right old frames — not to draw

Against drift

In training, the model sees its own corrupted frames and learns to fix them

Result

A real 3D world: export to a simulator

CHECKLIST

- ☑ Looks real
- ☑ Follows the camera
- ☑ Same when we return
- ☑ No drift
- ☑ Real 3D

Lyra 2.0 · 2026 →

Project page →

DLS

Lyra 2.0: a world you can open



CHECKLIST

- Looks real
- Follows the camera
- Same when we return
- No drift
- Real 3D

Lyra 2.0 · 2026 →

Project page →

DLS

Lyra 2.0: a world you can open



CHECKLIST

- [x] Looks real
- [x] Follows the camera
- [x] Same when we return
- [x] No drift
- [x] Real 3D

DLS



Nina Konovalova

@AfeliaN

@ReadingAI

Thank you for your attention!

@READING_AI